TCD1304 Controller

Generated by Doxygen 1.13.2

1 Firwmare for the TCD1304 using FlexPWM 1

1.0.1 In this directory: 1

2 Class Index 3

2.1 Class List 3

3 File Index 5

3.1 File List 5

4 Class Documentation 7

4.1 TCD1304Device::Frame_Header_struct Struct Reference 7

4.1.1 Member Data Documentation 7

4.1.1.1 buffer 7

4.1.1.2 error_flag 8

4.1.1.3 frame_counter 8

4.1.1.4 frame_elapsed_secs 8

4.1.1.5 frame_exposure_secs 8

4.1.1.6 frames_completed 8

4.1.1.7 frameset_counter 8

4.1.1.8 framesets_completed 8

4.1.1.9 mode 8

4.1.1.10 nbuffer 8

4.1.1.11 offset 8

4.1.1.12 offsetdone 9

4.1.1.13 oops_flag 9

4.1.1.14 pit_difference_secs 9

4.1.1.15 pit_elapsed_secs 9

4.1.1.16 rawoffset 9

4.1.1.17 ready_for_send 9

4.1.1.18 trigger_counter 9

4.1.1.19 trigger_difference_secs 9

4.1.1.20 trigger_elapsed_secs 9

4.1.1.21 trigger_mode 10

4.2 TCD1304Device::SubModule Struct Reference 10

4.2.1 Constructor & Destructor Documentation 11

4.2.1.1 SubModule() 11

4.2.2 Member Data Documentation 11

4.2.2.1 ctrl2_mask 11

4.2.2.2 divider 11

4.2.2.3 filler 11

4.2.2.4 flexpwm . . . 11
4.2.2.5 intena_mask . . . 11
4.2.2.6 interrupt_priority . . . 11
4.2.2.7 invertA . . . 12
4.2.2.8 invertB . . . 12
4.2.2.9 irq . . . 12
4.2.2.10 isr . . . 12
4.2.2.11 mask . . . 12
4.2.2.12 muxvalA . . . 12
4.2.2.13 muxvalB . . . 12
4.2.2.14 name . . . 12
4.2.2.15 newvals . . . 12
4.2.2.16 offA_counts . . . 12
4.2.2.17 offB_counts . . . 13
4.2.2.18 offset . . . 13
4.2.2.19 onA_counts . . . 13
4.2.2.20 onB_counts . . . 13
4.2.2.21 period_counts . . . 13
4.2.2.22 period_secs . . . 13
4.2.2.23 pinA . . . 13
4.2.2.24 pinB . . . 13
4.2.2.25 prescale . . . 13
4.2.2.26 submod . . . 14
4.3 TCD1304Device Class Reference . . . 14
4.3.1 Member Typedef Documentation . . . 19
4.3.1.1 Frame_Header . . . 19
4.3.2 Member Enumeration Documentation . . . 19
4.3.2.1 PULSE_LOOP_STATE . . . 19
4.3.3 Constructor & Destructor Documentation . . . 20
4.3.3.1 TCD1304Device() . . . 20
4.3.4 Member Function Documentation . . . 20
4.3.4.1 adc_blank_isr() . . . 20
4.3.4.2 attach_isr() . . . 21
4.3.4.3 begin_pit() . . . 21
4.3.4.4 check_submodule() . . . 22
4.3.4.5 clear_busypin() . . . 22
4.3.4.6 clear_error_flags() . . . 22
4.3.4.7 clear_frames_completed_callback() . . . 23
4.3.4.8 clear_framesets_completed_callback() . . . 23

4.3.4.9 clear_ldok() . . . 23
4.3.4.10 clear_mode() . . . 23
4.3.4.11 clear_run() . . . 23
4.3.4.12 clear_sync_busy_pins() . . . 24
4.3.4.13 clear_syncpin() . . . 24
4.3.4.14 close() . . . 24
4.3.4.15 cnvst_start() . . . 24
4.3.4.16 cycles64() . . . 25
4.3.4.17 disable_irqs() . . . 25
4.3.4.18 fill_frame_header() . . . 25
4.3.4.19 flexpwm_resetcounters() . . . 26
4.3.4.20 flexpwm_start() . . . 26
4.3.4.21 flexpwm_stop() . . . 26
4.3.4.22 flexpwm_wait() . . . 27
4.3.4.23 force() . . . 27
4.3.4.24 generic_wait() . . . 28
4.3.4.25 get_next_multiple() . . . 28
4.3.4.26 get_next_phase_tick() . . . 28
4.3.4.27 get_next_phase_time() . . . 29
4.3.4.28 load_frames_completed_callback() . . . 29
4.3.4.29 load_framesets_completed_callback() . . . 29
4.3.4.30 load_submodule() . . . 29
4.3.4.31 pit_difference_secs() . . . 30
4.3.4.32 pit_elapsed_secs() . . . 30
4.3.4.33 pit_isr() . . . 30
4.3.4.34 pit_start() . . . 31
4.3.4.35 pit_wait() . . . 31
4.3.4.36 print_and_check_submodule() . . . 32
4.3.4.37 print_counters() . . . 33
4.3.4.38 print_errormsg() [1/2] . . . 33
4.3.4.39 print_errormsg() [2/2] . . . 33
4.3.4.40 print_pulse_parameters() . . . 33
4.3.4.41 print_setup() . . . 34
4.3.4.42 print_setup_diagnostic() . . . 34
4.3.4.43 print_submodule() . . . 34
4.3.4.44 printbits16_() . . . 35
4.3.4.45 pulse_cnvst_isr() . . . 35
4.3.4.46 pulse_icg_isr() . . . 36
4.3.4.47 pulse_loop_arm() . . . 36

4.3.4.48 pulse_loop_cnvst_isr() . . . 36
4.3.4.49 pulse_loop_icg_isr() . . . 37
4.3.4.50 pulse_loop_sh_isr() . . . 37
4.3.4.51 pulse_loop_start() . . . 38
4.3.4.52 pulse_loop_stop() . . . 38
4.3.4.53 pulse_loop_wait() . . . 39
4.3.4.54 pulse_reset_and_arm() . . . 39
4.3.4.55 pulse_sh_isr() . . . 40
4.3.4.56 pulse_start() . . . 40
4.3.4.57 read() [1/2] . . . 40
4.3.4.58 read() [2/2] . . . 41
4.3.4.59 read_adc_offset() . . . 42
4.3.4.60 register_dump() . . . 43
4.3.4.61 set_clock_master() [1/2] . . . 43
4.3.4.62 set_clock_master() [2/2] . . . 44
4.3.4.63 set_clock_slave() [1/2] . . . 44
4.3.4.64 set_clock_slave() [2/2] . . . 45
4.3.4.65 set_clock_sync() [1/2] . . . 45
4.3.4.66 set_clock_sync() [2/2] . . . 45
4.3.4.67 set_cnt() . . . 46
4.3.4.68 set_init() . . . 46
4.3.4.69 set_ldok() . . . 46
4.3.4.70 set_outen() . . . 46
4.3.4.71 set_outen_off() . . . 46
4.3.4.72 set_outen_on() . . . 47
4.3.4.73 set_outenA_on() . . . 47
4.3.4.74 set_prescale() . . . 47
4.3.4.75 set_pulse_parameters() . . . 47
4.3.4.76 set_run() . . . 48
4.3.4.77 set_val0() . . . 48
4.3.4.78 set_val1() . . . 48
4.3.4.79 set_val2() . . . 48
4.3.4.80 set_val3() . . . 48
4.3.4.81 set_val4() . . . 48
4.3.4.82 set_val5() . . . 49
4.3.4.83 setup_digital_pins() . . . 49
4.3.4.84 setup_pit() . . . 49
4.3.4.85 setup_pulse() . . . 50
4.3.4.86 setup_pulse_loop() . . . 51

4.3.4.87 setup_submodule() . . . 52
4.3.4.88 setup_triggers() . . . 53
4.3.4.89 sh_difference_secs() . . . 54
4.3.4.90 sh_elapsed_secs() . . . 54
4.3.4.91 sh_exposure_secs() . . . 54
4.3.4.92 start_read() . . . 54
4.3.4.93 start_triggers() . . . 55
4.3.4.94 stop_all() . . . 56
4.3.4.95 stop_pit() . . . 56
4.3.4.96 stop_triggers() . . . 57
4.3.4.97 toggle_busypin() . . . 57
4.3.4.98 toggle_syncpin() . . . 57
4.3.4.99 trigger_difference_secs() . . . 57
4.3.4.100 trigger_elapsed_secs() . . . 58
4.3.4.101 trigger_isr() . . . 58
4.3.4.102 triggered_read() [1/2] . . . 59
4.3.4.103 triggered_read() [2/2] . . . 59
4.3.4.104 update_read_buffer() . . . 60
4.3.4.105 wait_read() . . . 60
4.3.4.106 wait_trigger() . . . 61
4.3.4.107 wait_triggered_read() . . . 62
4.3.5 Member Data Documentation . . . 62
4.3.5.1 adc_blank_counter . . . 62
4.3.5.2 adc_blank_running . . . 62
4.3.5.3 adc_offset . . . 62
4.3.5.4 adc_offset_done . . . 62
4.3.5.5 adc_raw_offset . . . 62
4.3.5.6 busytoggled . . . 63
4.3.5.7 clk . . . 63
4.3.5.8 cnvst . . . 63
4.3.5.9 cnvst_counter . . . 63
4.3.5.10 cyccnt64_prepare_exposure . . . 63
4.3.5.11 cyccnt64_prepare_readout . . . 63
4.3.5.12 error_flag . . . 63
4.3.5.13 flexpwm . . . 63
4.3.5.14 flexpwm_running . . . 63
4.3.5.15 frame_counter . . . 64
4.3.5.16 frame_counts . . . 64
4.3.5.17 frames_complete . . . 64

4.3.5.18 frames_completed_callback . . . 64
4.3.5.19 frameset_counter . . . 64
4.3.5.20 frameset_counts . . . 64
4.3.5.21 framesets_complete . . . 64
4.3.5.22 framesets_completed_callback . . . 64
4.3.5.23 icg . . . 64
4.3.5.24 icg_counter . . . 65
4.3.5.25 mode . . . 65
4.3.5.26 oops_flag . . . 65
4.3.5.27 oopsbuffer . . . 65
4.3.5.28 pit_begun . . . 65
4.3.5.29 pit_callback . . . 65
4.3.5.30 pit_counter . . . 65
4.3.5.31 pit_counts . . . 65
4.3.5.32 pit_cyccnt64_now . . . 65
4.3.5.33 pit_cyccnt64_prev . . . 65
4.3.5.34 pit_cyccnt64_start . . . 66
4.3.5.35 pit_delay_counts . . . 66
4.3.5.36 pit_delay_secs . . . 66
4.3.5.37 pit_interval_counts . . . 66
4.3.5.38 pit_interval_secs . . . 66
4.3.5.39 pit_running . . . 66
4.3.5.40 pit_state . . . 66
4.3.5.41 pulse_armed . . . 66
4.3.5.42 pulse_clk_secs . . . 66
4.3.5.43 pulse_cnvst_offset_secs . . . 67
4.3.5.44 pulse_cnvst_secs . . . 67
4.3.5.45 pulse_icg_offset_secs . . . 67
4.3.5.46 pulse_icg_secs . . . 67
4.3.5.47 pulse_interframe_min_secs . . . 67
4.3.5.48 pulse_loop_armed . . . 67
4.3.5.49 pulse_loop_frame_min_secs . . . 67
4.3.5.50 pulse_loop_period_secs . . . 67
4.3.5.51 pulse_loop_running . . . 67
4.3.5.52 pulse_loop_state . . . 67
4.3.5.53 pulse_sh_offset_secs . . . 68
4.3.5.54 pulse_sh_secs . . . 68
4.3.5.55 read_buffer . . . 68
4.3.5.56 read_callback . . . 68

4.3.5.57 read_counter . . . 68
4.3.5.58 read_counts . . . 68
4.3.5.59 read_expected_time . . . 68
4.3.5.60 read_pointer . . . 68
4.3.5.61 sh . . . 68
4.3.5.62 sh_clearing_counter . . . 69
4.3.5.63 sh_clearing_counts . . . 69
4.3.5.64 sh_counter . . . 69
4.3.5.65 sh_counts_per_icg . . . 69
4.3.5.66 sh_cyccnt64_exposure . . . 69
4.3.5.67 sh_cyccnt64_now . . . 69
4.3.5.68 sh_cyccnt64_prev . . . 69
4.3.5.69 sh_cyccnt64_start . . . 69
4.3.5.70 sh_short_period_counts . . . 69
4.3.5.71 sync_enabled . . . 69
4.3.5.72 sync_pin . . . 70
4.3.5.73 synctoggled . . . 70
4.3.5.74 trigger_attached . . . 70
4.3.5.75 trigger_busy . . . 70
4.3.5.76 trigger_callback . . . 70
4.3.5.77 trigger_counter . . . 70
4.3.5.78 trigger_counts . . . 70
4.3.5.79 trigger_cyccnt64_now . . . 70
4.3.5.80 trigger_cyccnt64_prev . . . 70
4.3.5.81 trigger_cyccnt64_start . . . 70
4.3.5.82 trigger_edge_mode . . . 71
4.3.5.83 trigger_mode . . . 71
4.3.5.84 trigger_pin . . . 71
4.3.5.85 trigger_pin_mode . . . 71
4.4 usb_string_descriptor_struct_manufacturer Struct Reference . . . 71
4.4.1 Member Data Documentation . . . 71
4.4.1.1 bDescriptorType . . . 71
4.4.1.2 bLength . . . 71
4.4.1.3 wString . . . 72
4.5 usb_string_descriptor_struct_product Struct Reference . . . 72
4.5.1 Member Data Documentation . . . 72
4.5.1.1 bDescriptorType . . . 72
4.5.1.2 bLength . . . 72
4.5.1.3 wString . . . 72

4.6 usb_string_descriptor_struct_serial_number Struct Reference . . . 72
4.6.1 Member Data Documentation . . . 73
4.6.1.1 bDescriptorType . . . 73
4.6.1.2 bLength . . . 73
4.6.1.3 wString . . . 73

5 File Documentation **75**
5.1 parselib2.cpp File Reference . . . 75
5.1.1 Function Documentation . . . 76
5.1.1.1 basenamef() . . . 76
5.1.1.2 countWords() . . . 77
5.1.1.3 eos() . . . 77
5.1.1.4 eow() . . . 77
5.1.1.5 nextWord() . . . 78
5.1.1.6 scaling() . . . 78
5.1.1.7 serialPrintf() . . . 79
5.1.1.8 serialPrintlnf() . . . 79
5.1.1.9 strBool() . . . 79
5.1.1.10 strFlt() . . . 80
5.1.1.11 strFlts() . . . 81
5.1.1.12 strMatch() . . . 82
5.1.1.13 strUint() . . . 82
5.1.1.14 strUint16() . . . 83
5.1.1.15 strUint32() . . . 83
5.1.1.16 strUint32s() . . . 84
5.1.1.17 strUint8() . . . 84
5.1.1.18 strUint8lim() . . . 85
5.1.1.19 strUints() . . . 85
5.1.1.20 wordLength() . . . 86
5.1.2 Variable Documentation . . . 86
5.1.2.1 nprintbuffer . . . 86
5.1.2.2 printbuffer . . . 86
5.2 parselib2.h File Reference . . . 86
5.2.1 Macro Definition Documentation . . . 87
5.2.1.1 PARSELIB_H . . . 87
5.2.2 Function Documentation . . . 87
5.2.2.1 basenamef() . . . 87
5.2.2.2 countWords() . . . 88
5.2.2.3 nextWord() . . . 88

5.2.2.4 serialPrintf() . . . 88
5.2.2.5 serialPrintlnf() . . . 89
5.2.2.6 strBool() . . . 89
5.2.2.7 strFlt() . . . 90
5.2.2.8 strFlts() . . . 90
5.2.2.9 strMatch() . . . 91
5.2.2.10 strUint() . . . 92
5.2.2.11 strUint16() . . . 93
5.2.2.12 strUint32() . . . 93
5.2.2.13 strUint32s() . . . 94
5.2.2.14 strUint8() . . . 94
5.2.2.15 strUint8lim() . . . 94
5.2.2.16 strUints() . . . 95
5.2.2.17 wordLength() . . . 95
5.3 parselib2.h . . . 96
5.4 README.md File Reference . . . 96
5.5 TCD1304Device2c.h File Reference . . . 96
5.5.1 Macro Definition Documentation . . . 99
5.5.1.1 BUSY_PIN . . . 99
5.5.1.2 BUSY_PIN_DEFAULT . . . 99
5.5.1.3 CLEARCNVST [1/2] . . . 100
5.5.1.4 CLEARCNVST [2/2] . . . 100
5.5.1.5 CLK_CHANNEL . . . 100
5.5.1.6 CLK_DEFAULT . . . 100
5.5.1.7 CLK_IRQ . . . 100
5.5.1.8 CLK_MASK . . . 100
5.5.1.9 CLK_MONITOR_PIN . . . 100
5.5.1.10 CLK_MUXVAL . . . 100
5.5.1.11 CLK_PIN . . . 100
5.5.1.12 CLK_SUBMODULE . . . 100
5.5.1.13 CMPF_MASKA_OFF . . . 101
5.5.1.14 CMPF_MASKA_ON . . . 101
5.5.1.15 CMPF_MASKA_ON_OFF . . . 101
5.5.1.16 CMPF_MASKB_OFF . . . 101
5.5.1.17 CMPF_MASKB_ON . . . 101
5.5.1.18 CMPF_MASKB_ON_OFF . . . 101
5.5.1.19 CNVST_CHANNEL . . . 101
5.5.1.20 CNVST_CLKS_FROM_ICG . . . 101
5.5.1.21 CNVST_IRQ . . . 101

5.5.1.22 CNVST_MASK . . . 101
5.5.1.23 CNVST_PIN . . . 102
5.5.1.24 CNVST_SECS . . . 102
5.5.1.25 CNVST_START_OVERHEAD . . . 102
5.5.1.26 CNVST_SUBMODULE . . . 102
5.5.1.27 COUNTER_MAX_SECS . . . 102
5.5.1.28 COUNTSCEIL . . . 102
5.5.1.29 COUNTSFLOOR . . . 102
5.5.1.30 CYCCNT2SECS . . . 102
5.5.1.31 DARKSTART . . . 103
5.5.1.32 DATASTART . . . 103
5.5.1.33 DATASTOP . . . 103
5.5.1.34 DEBUG . . . 103
5.5.1.35 DEBUGPRINTF . . . 103
5.5.1.36 ICG_CHANNEL . . . 103
5.5.1.37 ICG_IRQ . . . 103
5.5.1.38 ICG_MASK . . . 103
5.5.1.39 ICG_MUXVAL . . . 103
5.5.1.40 ICG_PIN . . . 104
5.5.1.41 ICG_SUBMODULE . . . 104
5.5.1.42 NBITS . . . 104
5.5.1.43 NDARK . . . 104
5.5.1.44 NPIXELS . . . 104
5.5.1.45 NREADOUT . . . 104
5.5.1.46 PINPULLS . . . 104
5.5.1.47 PIT_CHANNEL . . . 104
5.5.1.48 PIT_FREQ . . . 104
5.5.1.49 PITCOUNTS . . . 105
5.5.1.50 PITSECS . . . 105
5.5.1.51 PWM_CTRL2_CLOCK_MASTER . . . 105
5.5.1.52 PWM_CTRL2_CLOCK_SLAVE . . . 105
5.5.1.53 PWM_CTRL2_CLOCK_SYNC . . . 105
5.5.1.54 ROUNDTO . . . 105
5.5.1.55 ROUNDTOMOD . . . 105
5.5.1.56 ROUNDUP . . . 106
5.5.1.57 SAR_OFFSET . . . 106
5.5.1.58 SECONDS . . . 106
5.5.1.59 SECS2CYCCNT . . . 106
5.5.1.60 SENDLENGTHBYTES . . . 106

5.5.1.61 SENDLENGTHBYTES32 . . . 106
5.5.1.62 SENDLENGTHWORDS . . . 106
5.5.1.63 SENDSTART . . . 107
5.5.1.64 SETCNVST [1/2] . . . 107
5.5.1.65 SETCNVST [2/2] . . . 107
5.5.1.66 SH_CHANNEL . . . 107
5.5.1.67 SH_CLEARING_DEFAULT . . . 107
5.5.1.68 SH_IRQ . . . 107
5.5.1.69 SH_MASK . . . 107
5.5.1.70 SH_MUXVAL . . . 107
5.5.1.71 SH_PIN . . . 107
5.5.1.72 SH_SUBMODULE . . . 107
5.5.1.73 SHUTTERMIN . . . 108
5.5.1.74 SYNC_PIN . . . 108
5.5.1.75 SYNC_PIN_DEFAULT . . . 108
5.5.1.76 TCD1304_MAXCLKHZ . . . 108
5.5.1.77 TCD1304_MINCLKHZ . . . 108
5.5.1.78 TCD1304DEVICE_LIBARY_VERSION . . . 108
5.5.1.79 TDIFFSECS . . . 108
5.5.1.80 TRIGGER_PIN . . . 108
5.5.1.81 USBSPEED . . . 108
5.5.1.82 USBTRANSFERSECS . . . 109
5.5.1.83 VFS . . . 109
5.5.1.84 VPERBIT . . . 109
5.5.2 Enumeration Type Documentation . . . 109
5.5.2.1 TCD1304_Mode_t . . . 109
5.6 TCD1304Device2c.h . . . 109
5.7 TCD1304Device_Controller_260318.ino File Reference . . . 152
5.7.1 Macro Definition Documentation . . . 156
5.7.1.1 ADC_RESOLUTION . . . 156
5.7.1.2 ASCII . . . 156
5.7.1.3 BINARY . . . 156
5.7.1.4 CONTROLLER_CSPIN . . . 156
5.7.1.5 CONTROLLER_CSPIN2 . . . 156
5.7.1.6 CONTROLLER_OCPIN . . . 156
5.7.1.7 CYCLES_PER_USEC . . . 157
5.7.1.8 DRMCNELSONLAB . . . 157
5.7.1.9 EEPROM_COEFF_ADDR . . . 157
5.7.1.10 EEPROM_COEFF_LEN . . . 157

5.7.1.11 EEPROM_ID_ADDR . . . 157
5.7.1.12 EEPROM_ID_LEN . . . 157
5.7.1.13 EEPROM_NCOEFFS . . . 157
5.7.1.14 EEPROM_NUNITS . . . 157
5.7.1.15 EEPROM_SIZE . . . 157
5.7.1.16 EEPROM_UNITS_ADDR . . . 157
5.7.1.17 EEPROM_UNITS_LEN . . . 158
5.7.1.18 IMR_INDEX . . . 158
5.7.1.19 ISR_INDEX . . . 158
5.7.1.20 NADC_CHANNELS . . . 158
5.7.1.21 NBUFFERS . . . 158
5.7.1.22 RCVLEN . . . 158
5.7.1.23 SNDLEN . . . 158
5.7.1.24 thisMANUFACTURER_NAME . . . 158
5.7.1.25 thisMANUFACTURER_NAME_LEN . . . 158
5.7.1.26 thisPRODUCT_NAME . . . 158
5.7.1.27 thisPRODUCT_NAME_LEN . . . 159
5.7.1.28 thisPRODUCT_SERIAL_NUMBER . . . 159
5.7.1.29 thisPRODUCT_SERIAL_NUMBER_LEN . . . 159
5.7.2 Function Documentation . . . 159
5.7.2.1 blink() . . . 159
5.7.2.2 calculateCRC() . . . 159
5.7.2.3 clock_isr() . . . 159
5.7.2.4 clock_setup() . . . 160
5.7.2.5 clock_start() . . . 160
5.7.2.6 clock_stop() . . . 161
5.7.2.7 cycles64() . . . 161
5.7.2.8 disablePinInterrupts() . . . 161
5.7.2.9 eeErase() . . . 161
5.7.2.10 eeread() . . . 162
5.7.2.11 eereadUntil() . . . 162
5.7.2.12 eewrite() . . . 162
5.7.2.13 eewriteUntil() . . . 163
5.7.2.14 eraseCoefficients() . . . 163
5.7.2.15 eraseIdentifier() . . . 164
5.7.2.16 eraseUnits() . . . 164
5.7.2.17 fastAnalogRead() . . . 165
5.7.2.18 frame_callback() . . . 165
5.7.2.19 help() . . . 166

5.7.2.20 loop() . . . 166
5.7.2.21 ocpinAttach() . . . 167
5.7.2.22 ocpinDetach() . . . 168
5.7.2.23 ocpinISR() . . . 168
5.7.2.24 ocpinRead() . . . 169
5.7.2.25 printCoefficients() . . . 169
5.7.2.26 printIdentifier() . . . 170
5.7.2.27 printTriggerSetup() . . . 170
5.7.2.28 printUnits() . . . 171
5.7.2.29 pulsePin() . . . 171
5.7.2.30 readCoefficients() . . . 172
5.7.2.31 readIdentifier() . . . 172
5.7.2.32 readUnits() . . . 173
5.7.2.33 resumePinInterrupts() . . . 174
5.7.2.34 send_frame() . . . 174
5.7.2.35 send_frames() . . . 174
5.7.2.36 send_header() . . . 175
5.7.2.37 sendADCs() . . . 175
5.7.2.38 sendBuffer_Binary() . . . 176
5.7.2.39 sendBuffer_Formatted() . . . 176
5.7.2.40 sendChipTemperature() . . . 177
5.7.2.41 sendData() . . . 177
5.7.2.42 sendDataCRC() . . . 178
5.7.2.43 sendDataReady() . . . 178
5.7.2.44 sendDataSum() . . . 179
5.7.2.45 sendTestData() . . . 179
5.7.2.46 setup() . . . 180
5.7.2.47 spi_settings() . . . 180
5.7.2.48 storeCoefficients() . . . 181
5.7.2.49 storeIdentifier() . . . 181
5.7.2.50 storeUnits() . . . 182
5.7.2.51 strPin() . . . 182
5.7.2.52 strStartNoStart() . . . 183
5.7.2.53 strSubModule() . . . 184
5.7.2.54 strTimings() . . . 185
5.7.2.55 strTrigger() . . . 185
5.7.2.56 strWaitParameters() . . . 186
5.7.2.57 sumData() . . . 187
5.7.2.58 tempmonGetTemp() . . . 187

5.7.3 Variable Documentation . . . 188
5.7.3.1 adc . . . 188
5.7.3.2 adc_averages . . . 188
5.7.3.3 adc_data . . . 188
5.7.3.4 analogPin . . . 188
5.7.3.5 authorstr . . . 188
5.7.3.6 buffer_index . . . 188
5.7.3.7 buffer_ring . . . 188
5.7.3.8 bufferp . . . 188
5.7.3.9 busyPin . . . 188
5.7.3.10 chipTemp_averages . . . 189
5.7.3.11 CLKPin . . . 189
5.7.3.12 clock_callback . . . 189
5.7.3.13 clock_counter . . . 189
5.7.3.14 clock_counts . . . 189
5.7.3.15 clock_mode . . . 189
5.7.3.16 clock_timer . . . 189
5.7.3.17 clock_usecs . . . 189
5.7.3.18 CNVSTPin . . . 189
5.7.3.19 counter . . . 189
5.7.3.20 crc_enable . . . 190
5.7.3.21 crctable . . . 190
5.7.3.22 data_async . . . 190
5.7.3.23 dataformat . . . 190
5.7.3.24 diagnostic_usecs . . . 190
5.7.3.25 diagnostics . . . 190
5.7.3.26 elapsed_usecs . . . 190
5.7.3.27 fMPin . . . 191
5.7.3.28 frame_header_ring . . . 191
5.7.3.29 frame_index . . . 191
5.7.3.30 frame_send_index . . . 191
5.7.3.31 framep . . . 191
5.7.3.32 ICGPin . . . 191
5.7.3.33 nrcvbuf . . . 191
5.7.3.34 ocpinattached . . . 191
5.7.3.35 ocpinstate . . . 191
5.7.3.36 rcvbuffer . . . 191
5.7.3.37 SDIPin . . . 192
5.7.3.38 SDOPin . . . 192

5.7.3.39 sensorstr 192
5.7.3.40 SHPin 192
5.7.3.41 sndbuffer 192
5.7.3.42 srcfilestr 192
5.7.3.43 sum_enable 192
5.7.3.44 syncPin 192
5.7.3.45 tcd1304device 192
5.7.3.46 triggerPin 192
5.7.3.47 usb_string_manufacturer_name 193
5.7.3.48 usb_string_product_name 193
5.7.3.49 usb_string_serial_number 193
5.7.3.50 versionstr 193

Index **195**

Chapter 1

Firwmare for the TCD1304 using FlexPWM

This directory contains an Arduino sketch file and libraries to operate the TCD1304 Sensor using the Teensy 4 in either (a) the two board system, the TCD1304 SPI board plus Teensy 4 Controller Rev 3 or (b) the All-In-One board with FlexPWM support.

Features in this implementation include ring buffering, and cnvst idling to keep the ADC and its voltage reference in a ready steady state.

To setup your system to install the firmware, see the `Teensyduino website and setup instructions`

See the `Python directory in this repo` for an interactive user utility with real time graphics. The Python program uses Python3, Numpy, Matplotlib, PySerial and SciPY.

In the interactive user utility, use the command "help" to see a list of commands.

1.0.1 In this directory:

The file TCD1304Device2b.h is a header only library for the TCD1304 using the NXP iMXRT1060 FlexPWM.

The file TCD1304Device_Controller_250213.ino implements the user command interface and instantiates the ring buffer system.

The files parselib2.cpp and parselib2.h provides an api for parsing commands and is used in the controller code to implement the command language for the device.

The ring buffer system is implemented in the controller code, using the frame completion callback interface provide by the TCD1304Device class along with a frame structure also provided in the device library.

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

TCD1304Device::Frame_Header_struct 7
TCD1304Device::SubModule 10
TCD1304Device 14
usb_string_descriptor_struct_manufacturer 71
usb_string_descriptor_struct_product 72
usb_string_descriptor_struct_serial_number 72

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

parselib2.cpp 75
parselib2.h 86
TCD1304Device2c.h 96
TCD1304Device_Controller_260318.ino 152

Chapter 4

Class Documentation

4.1 TCD1304Device::Frame_Header_struct Struct Reference

```
#include <TCD1304Device2c.h>
```

Public Attributes

- uint16_t * buffer
- unsigned int nbuffer
- unsigned int rawoffset
- float offset
- bool offsetdone
- float frame_elapsed_secs
- float frame_exposure_secs
- float pit_elapsed_secs
- float pit_difference_secs
- float trigger_elapsed_secs
- float trigger_difference_secs
- unsigned int frame_counter
- unsigned int frameset_counter
- unsigned int trigger_counter
- TCD1304_Mode_t mode
- bool trigger_mode
- bool error_flag
- bool oops_flag
- bool frames_completed
- bool framesets_completed
- bool ready_for_send

4.1.1 Member Data Documentation

4.1.1.1 buffer

```
uint16_t* TCD1304Device::Frame_Header_struct::buffer
```

4.1.1.2 error_flag

```
bool TCD1304Device::Frame_Header_struct::error_flag
```

4.1.1.3 frame_counter

```
unsigned int TCD1304Device::Frame_Header_struct::frame_counter
```

4.1.1.4 frame_elapsed_secs

```
float TCD1304Device::Frame_Header_struct::frame_elapsed_secs
```

4.1.1.5 frame_exposure_secs

```
float TCD1304Device::Frame_Header_struct::frame_exposure_secs
```

4.1.1.6 frames_completed

```
bool TCD1304Device::Frame_Header_struct::frames_completed
```

4.1.1.7 frameset_counter

```
unsigned int TCD1304Device::Frame_Header_struct::frameset_counter
```

4.1.1.8 framesets_completed

```
bool TCD1304Device::Frame_Header_struct::framesets_completed
```

4.1.1.9 mode

```
TCD1304_Mode_t TCD1304Device::Frame_Header_struct::mode
```

4.1.1.10 nbuffer

```
unsigned int TCD1304Device::Frame_Header_struct::nbuffer
```

4.1.1.11 offset

```
float TCD1304Device::Frame_Header_struct::offset
```

4.1.1.12 offsetdone

```
bool TCD1304Device::Frame_Header_struct::offsetdone
```

4.1.1.13 oops_flag

```
bool TCD1304Device::Frame_Header_struct::oops_flag
```

4.1.1.14 pit_difference_secs

```
float TCD1304Device::Frame_Header_struct::pit_difference_secs
```

4.1.1.15 pit_elapsed_secs

```
float TCD1304Device::Frame_Header_struct::pit_elapsed_secs
```

4.1.1.16 rawoffset

```
unsigned int TCD1304Device::Frame_Header_struct::rawoffset
```

4.1.1.17 ready_for_send

```
bool TCD1304Device::Frame_Header_struct::ready_for_send
```

4.1.1.18 trigger_counter

```
unsigned int TCD1304Device::Frame_Header_struct::trigger_counter
```

4.1.1.19 trigger_difference_secs

```
float TCD1304Device::Frame_Header_struct::trigger_difference_secs
```

4.1.1.20 trigger_elapsed_secs

```
float TCD1304Device::Frame_Header_struct::trigger_elapsed_secs
```

4.1.1.21 trigger_mode

```
bool TCD1304Device::Frame_Header_struct::trigger_mode
```

The documentation for this struct was generated from the following file:

- TCD1304Device2c.h

4.2 TCD1304Device::SubModule Struct Reference

```
#include <TCD1304Device2c.h>
```

Public Member Functions

- SubModule (const char *name_, uint8_t submod_, uint8_t mask_, uint8_t pinA_, uint8_t muxvalA_, uint8_t pin↩ B_, uint8_t muxvalB_, IRQ_NUMBER_t irq_, IMXRT_FLEXPWM_t *flexpwm_)

Public Attributes

- const char * name
- const uint8_t submod
- const uint8_t mask
- const uint8_t pinA
- const uint8_t muxvalA
- const uint8_t pinB
- const uint8_t muxvalB
- IRQ_NUMBER_t irq
- IMXRT_FLEXPWM_t * flexpwm
- uint32_t period_counts = 0
- uint16_t onA_counts = 0
- uint16_t offA_counts = 0
- uint16_t onB_counts = 0
- uint16_t offB_counts = 0
- uint16_t ctrl2_mask = 0
- uint16_t intena_mask = 0
- uint16_t divider = 1
- uint8_t prescale = 0
- uint8_t filler = 0
- float period_secs = 0
- float offset = 0
- bool invertA = false
- bool invertB = false
- bool newvals = false
- void(* isr)() = nullptr
- uint8_t interrupt_priority = 128

4.2.1 Constructor & Destructor Documentation

4.2.1.1 SubModule()

```
TCD1304Device::SubModule::SubModule (
            const char * name_,
            uint8_t submod_,
            uint8_t mask_,
            uint8_t pinA_,
            uint8_t muxvalA_,
            uint8_t pinB_,
            uint8_t muxvalB_,
            IRQ_NUMBER_t irq_,
            IMXRT_FLEXPWM_t * flexpwm_)  [inline]
```

4.2.2 Member Data Documentation

4.2.2.1 ctrl2_mask

```
uint16_t TCD1304Device::SubModule::ctrl2_mask = 0
```

4.2.2.2 divider

```
uint16_t TCD1304Device::SubModule::divider = 1
```

4.2.2.3 filler

```
uint8_t TCD1304Device::SubModule::filler = 0
```

4.2.2.4 flexpwm

```
IMXRT_FLEXPWM_t* TCD1304Device::SubModule::flexpwm
```

4.2.2.5 intena_mask

```
uint16_t TCD1304Device::SubModule::intena_mask = 0
```

4.2.2.6 interrupt_priority

```
uint8_t TCD1304Device::SubModule::interrupt_priority = 128
```

4.2.2.7 invertA

```
bool TCD1304Device::SubModule::invertA = false
```

4.2.2.8 invertB

```
bool TCD1304Device::SubModule::invertB = false
```

4.2.2.9 irq

```
IRQ_NUMBER_t TCD1304Device::SubModule::irq
```

4.2.2.10 isr

```
void(* TCD1304Device::SubModule::isr) () = nullptr
```

4.2.2.11 mask

```
const uint8_t TCD1304Device::SubModule::mask
```

4.2.2.12 muxvalA

```
const uint8_t TCD1304Device::SubModule::muxvalA
```

4.2.2.13 muxvalB

```
const uint8_t TCD1304Device::SubModule::muxvalB
```

4.2.2.14 name

```
const char* TCD1304Device::SubModule::name
```

4.2.2.15 newvals

```
bool TCD1304Device::SubModule::newvals = false
```

4.2.2.16 offA_counts

```
uint16_t TCD1304Device::SubModule::offA_counts = 0
```

4.2.2.17 offB_counts

```
uint16_t TCD1304Device::SubModule::offB_counts = 0
```

4.2.2.18 offset

```
float TCD1304Device::SubModule::offset = 0
```

4.2.2.19 onA_counts

```
uint16_t TCD1304Device::SubModule::onA_counts = 0
```

4.2.2.20 onB_counts

```
uint16_t TCD1304Device::SubModule::onB_counts = 0
```

4.2.2.21 period_counts

```
uint32_t TCD1304Device::SubModule::period_counts = 0
```

4.2.2.22 period_secs

```
float TCD1304Device::SubModule::period_secs = 0
```

4.2.2.23 pinA

```
const uint8_t TCD1304Device::SubModule::pinA
```

4.2.2.24 pinB

```
const uint8_t TCD1304Device::SubModule::pinB
```

4.2.2.25 prescale

```
uint8_t TCD1304Device::SubModule::prescale = 0
```

4.2.2.26 submod

```
const uint8_t TCD1304Device::SubModule::submod
```

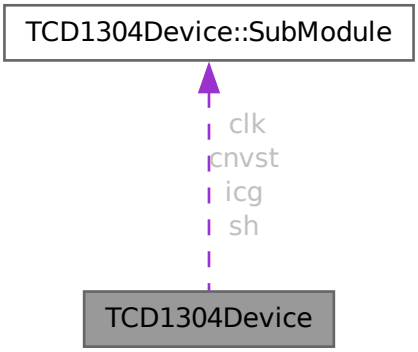
The documentation for this struct was generated from the following file:

- TCD1304Device2c.h

4.3 TCD1304Device Class Reference

```
#include <TCD1304Device2c.h>
```

Collaboration diagram for TCD1304Device:



Classes

- struct Frame_Header_struct
- struct SubModule

Public Types

- enum PULSE_LOOP_STATE {
  IDLE , START_EXPOSURE , WAIT_EXPOSURE , WAIT_FRAME ,
  READOUT }
- typedef struct TCD1304Device::Frame_Header_struct Frame_Header

Public Member Functions

- TCD1304Device (unsigned int period=CLK_DEFAULT)
- double get_next_phase_time (double t, double p, double phase_fraction)
- uint16_t get_next_phase_tick (uint16_t t, uint32_t p, double phase_fraction)
- uint32_t get_next_multiple (uint32_t a, uint32_t b)
- void print_pulse_parameters ()
- bool set_pulse_parameters (float clock_period_secs=0.4E-6, float period_secs=10.E-6)
- bool wait_read (float timeout=0., float timestep=0.01, bool interruptible=false, bool verbose=false)
- bool read (uint nframes, float exposure, uint16_t *bufferp, void(*frame_callbackf)(), void(*frameset_callbackf)(), void(*completion_callbackf)(), void(*setup_callbackf)(), bool start=true)
- bool read (uint nframes, float exposure, float frame_interval, uint16_t *bufferp, void(*frame_callbackf)(), void(*frameset_callbackf)(), void(*completion_callbackf)(), void(*setup_callbackf)(), bool start=true)
- bool triggered_read (uint ntriggers, uint nframes, float exposure, uint16_t *bufferp, void(*frame_callbackf)(), void(*frameset_callbackf)(), void(*completion_callbackf)(), void(*setup_callbackf)(), bool start=true)
- bool triggered_read (uint ntriggers, uint nframes, float exposure, float interval, uint16_t *bufferp, void(*frame_callbackf)(), void(*frameset_callbackf)(), void(*completion_callbackf)(), void(*setup_callbackf)(), bool start=true)
- bool wait_triggered_read (float timeout=1., float timestep=0.01, bool interruptible=true, bool verbose=false)
- void print_setup_diagnostic (SubModule *p)
- void print_setup ()
- void print_submodule (SubModule *p)
- bool check_submodule (SubModule *p)
- bool print_and_check_submodule (SubModule *p)
- void load_submodule (SubModule *p)
- bool setup_submodule (SubModule *p, uint8_t prescale, uint32_t period_counts, uint16_t onA_counts, uint16_t offA_counts, bool invertA, uint16_t onB_counts, uint16_t offB_counts, bool invertB, uint16_t ctrl2_mask)
- void attach_isr (SubModule *p, uint16_t cmpf_mask, void(*isrf)(), uint8_t priority=0, bool enable=true)
- void clear_frames_completed_callback ()
- void load_frames_completed_callback (void(*callback)(), unsigned int nframes=0)
- void clear_framesets_completed_callback ()
- void load_framesets_completed_callback (void(*callback)(), unsigned int nsets=0)
- bool generic_wait (bool *flag, float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
- bool flexpwm_wait (float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
- bool setup_pulse (float clk_secs, float sh_secs, float sh_offset_secs, float icg_secs, float icg_offset_secs, uint16_t *buffer, unsigned int nbuffer, void(*callbackf)())
- bool read_adc_offset ()
- bool pulse_loop_wait (float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
- bool setup_pulse_loop (float clk_secs, float sh_secs, float sh_offset_secs, float icg_secs, float icg_offset_secs, float period_secs, float exposure_secs, float frame_secs, uint nframes, uint16_t *buffer, unsigned int nbuffer, void(*callbackf)())
- void begin_pit ()
- bool pit_wait (float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
- bool setup_pit (float exposure_secs, float delay_secs=1.E-6, unsigned int ncounts=0)
- bool start_triggers ()
- bool wait_trigger (float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
- bool setup_triggers (unsigned int ncounts=0)
- void set_clock_master (uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)

  Low level functions for command line register access.
- void set_clock_master (SubModule *p)
- void set_clock_slave (uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_clock_slave (SubModule *p)

- void set_clock_sync (uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_clock_sync (SubModule *p)
- void clear_ldok (uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_ldok (uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm)
- void clear_run (uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_run (uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm)
- void force (uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_prescale (uint8_t submod, uint8_t prescale=0, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_init (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_cnt (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val0 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val1 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val2 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val3 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val4 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- void set_val5 (uint8_t submod, uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm)
- uint16_t set_outen (uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm)
- uint16_t set_outen_on (uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm)
- uint16_t set_outen_off (uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm)
- uint16_t set_outenA_on (uint8_t mask8, IMXRT_FLEXPWM_t *q=flexpwm)
- void printbits16_ (uint16_t u16, int bits)
- void register_dump (IMXRT_FLEXPWM_t *const q=flexpwm, uint8_t mask=0xFF)

Static Public Member Functions

- static void stop_all ()
- static void disable_irqs ()
- static void clear_error_flags ()
- static void clear_mode ()
- static void clear_sync_busy_pins ()
- static void toggle_busypin ()
- static void toggle_syncpin ()
- static void clear_busypin ()
- static void clear_syncpin ()
- static void update_read_buffer (uint16_t *buffer)
- static void fill_frame_header (Frame_Header *p)
- static void start_read ()
- static uint64_t cycles64 ()
- static double sh_elapsed_secs ()
- static double sh_difference_secs ()
- static double sh_exposure_secs ()
- static double pit_elapsed_secs ()
- static double pit_difference_secs ()
- static double trigger_elapsed_secs ()
- static double trigger_difference_secs ()
- static void print_errormsg (const char *name, const char *errmsg)
- static void print_errormsg (SubModule *p, const char *errmsg)
- static void print_counters ()
- static void cnvst_start ()
- static void flexpwm_stop (uint8_t mask=0xF)

- static void flexpwm_start (uint8_t mask=0xF)
- static void flexpwm_resetcounters (uint8_t mask=0xFF)
- static void pulse_sh_isr ()
- static void pulse_icg_isr ()
- static void pulse_cnvst_isr ()
- static void pulse_start ()
- static void pulse_reset_and_arm ()
- static void adc_blank_isr ()
- static void pulse_loop_sh_isr ()
- static void pulse_loop_icg_isr ()
- static void pulse_loop_cnvst_isr ()
- static void pulse_loop_stop ()
- static void pulse_loop_start ()
- static void pulse_loop_arm ()
- static void pit_isr ()
- static void pit_start ()
- static void stop_pit ()
- static void trigger_isr ()
- static void stop_triggers ()
- static void setup_digital_pins ()
- static void close ()

Public Attributes

- char oopsbuffer [128] = { 0 }

Static Public Attributes

- static uint64_t sh_cyccnt64_start = 0
- static uint64_t sh_cyccnt64_now = 0
- static uint64_t sh_cyccnt64_prev = 0
- static uint64_t sh_cyccnt64_exposure = 0
- static uint64_t pit_cyccnt64_start = 0
- static uint64_t pit_cyccnt64_now = 0
- static uint64_t pit_cyccnt64_prev = 0
- static uint64_t trigger_cyccnt64_start = 0
- static uint64_t trigger_cyccnt64_now = 0
- static uint64_t trigger_cyccnt64_prev = 0
- static TCD1304_Mode_t mode = NOTCONFIGURED
- static bool trigger_mode = false
- static bool trigger_attached = false
- static unsigned int sh_counter = 0
- static unsigned int icg_counter = 0
- static unsigned int cnvst_counter = 0
- static unsigned int sh_counts_per_icg = 0
- static unsigned int sh_clearing_counts = SH_CLEARING_DEFAULT
- static unsigned int sh_clearing_counter = 0
- static unsigned int sh_short_period_counts = 0
- static unsigned int read_counter = 0

- static unsigned int read_counts = 0
- static uint16_t ∗ read_buffer = 0
- static uint16_t ∗ read_pointer = 0
- static void(∗ read_callback)()=0
- static unsigned int frame_counter = 0
- static unsigned int frame_counts = 0
- static void(∗ frames_completed_callback)()=0
- static unsigned int frameset_counter = 0
- static unsigned int frameset_counts = 0
- static void(∗ framesets_completed_callback)()=0
- static float pulse_interframe_min_secs = 0.
- static unsigned int trigger_counter = 0
- static unsigned int trigger_counts = 1
- static void(∗ trigger_callback)()=0
- static uint8_t trigger_pin = TRIGGER_PIN
- static uint8_t trigger_edge_mode = RISING
- static uint8_t trigger_pin_mode = INPUT
- static bool busytoggled = false
- static uint8_t sync_pin = SYNC_PIN
- static bool synctoggled = false
- static bool sync_enabled = true
- static IMXRT_FLEXPWM_t ∗const flexpwm = &IMXRT_FLEXPWM2
- static SubModule clk = {"clk",CLK_SUBMODULE, CLK_MASK, CLK_PIN,CLK_MUXVAL, 0xFF,0, CLK_IRQ, &IMXRT_FLEXPWM2}
- static SubModule sh = {"sh", SH_SUBMODULE, SH_MASK, SH_PIN,SH_MUXVAL, 0xFF,0, SH_IRQ, &IMXRT↩ _FLEXPWM2}
- static SubModule icg = {"icg", ICG_SUBMODULE, ICG_MASK, ICG_PIN,ICG_MUXVAL, 0xFF,0, ICG_IRQ, &IMXRT_FLEXPWM2}
- static SubModule cnvst = {"cnvst", CNVST_SUBMODULE, CNVST_MASK, 0xFF,0, 0xFF,0, CNVST_IRQ, &IMXRT_FLEXPWM2}
- static bool error_flag = false
- static bool oops_flag = false
- static bool frames_complete = false
- static bool framesets_complete = false
- static float pulse_clk_secs = 0.
- static float pulse_sh_offset_secs = 0.
- static float pulse_sh_secs = 0.
- static float pulse_icg_offset_secs = 0.
- static float pulse_icg_secs = 0.
- static float pulse_cnvst_offset_secs = 0.
- static float pulse_cnvst_secs = 0.
- static float pulse_loop_period_secs = 0.
- static float read_expected_time = 0.
- static bool flexpwm_running = false
- static bool pulse_armed = false

  Configure the flexpwm for single pulse, use this with a timer or interrupt.
- static bool adc_blank_running = false
- static uint adc_blank_counter = 0
- static uint adc_raw_offset = 0
- static double adc_offset = 0.
- static bool adc_offset_done = false

- static bool pulse_loop_armed = false
- static bool pulse_loop_running = false
- static uint64_t cyccnt64_prepare_readout = 0
- static uint64_t cyccnt64_prepare_exposure = 0
- static PULSE_LOOP_STATE pulse_loop_state = IDLE
- static float pulse_loop_frame_min_secs = 0.
- static float pit_delay_secs = 1.E-6
- static float pit_interval_secs = 0.1
- static uint32_t pit_delay_counts = 0
- static uint32_t pit_interval_counts = 0
- static uint8_t pit_state = 0
- static unsigned int pit_counts = 2
- static unsigned int pit_counter = 0
- static bool pit_running = false
- static bool pit_begun = false
- static void(∗ pit_callback)()=0
- static bool trigger_busy = false

4.3.1 Member Typedef Documentation

4.3.1.1 Frame_Header

```
typedef struct TCD1304Device::Frame_Header_struct TCD1304Device::Frame_Header
```

4.3.2 Member Enumeration Documentation

4.3.2.1 PULSE_LOOP_STATE

```
enum TCD1304Device::PULSE_LOOP_STATE
```

Enumerator

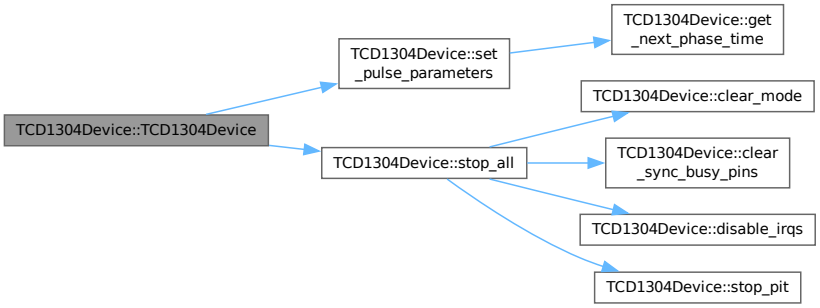
IDLE	
START_EXPOSURE	
WAIT_EXPOSURE	
WAIT_FRAME	
READOUT	

4.3.3 Constructor & Destructor Documentation

4.3.3.1 TCD1304Device()

```
TCD1304Device::TCD1304Device (
            unsigned int period = CLK_DEFAULT)  [inline]
```

Here is the call graph for this function:

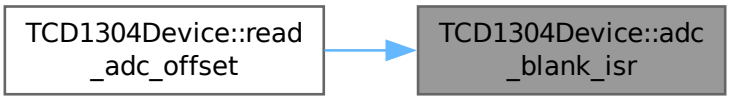


4.3.4 Member Function Documentation

4.3.4.1 adc_blank_isr()

```
static void TCD1304Device::adc_blank_isr ()  [inline], [static]
```

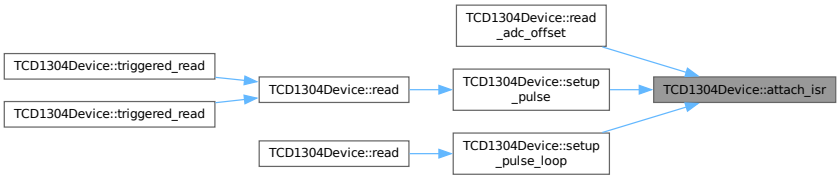
Here is the caller graph for this function:



4.3.4.2 attach_isr()

```
void TCD1304Device::attach_isr (
            SubModule * p,
            uint16_t cmpf_mask,
            void(* isrf )(),
            uint8_t priority = 0,
            bool enable = true)  [inline]
```

Here is the caller graph for this function:

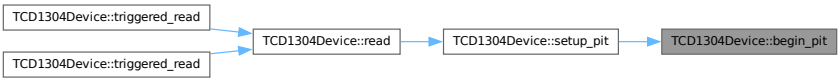


4.3.4.3 begin_pit()

```
void TCD1304Device::begin_pit ()  [inline]
```

Here is the call graph for this function:

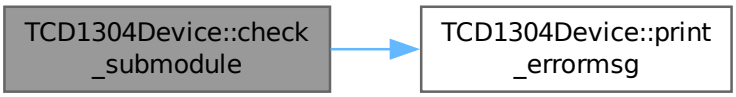
Here is the caller graph for this function:



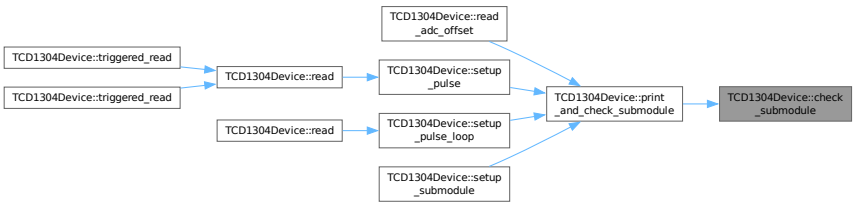
4.3.4.4 check_submodule()

```
bool TCD1304Device::check_submodule (
            SubModule * p)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



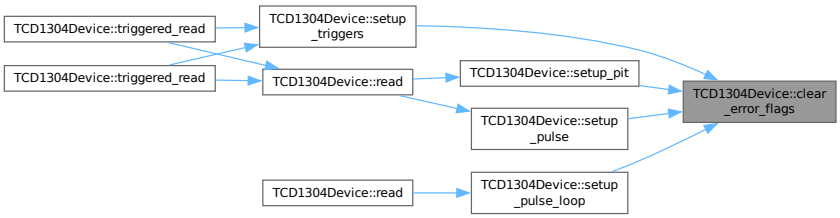
4.3.4.5 clear_busypin()

```
static void TCD1304Device::clear_busypin ()  [inline], [static]
```

4.3.4.6 clear_error_flags()

```
static void TCD1304Device::clear_error_flags ()  [inline], [static]
```

Here is the caller graph for this function:



4.3.4.7 clear_frames_completed_callback()

```
void TCD1304Device::clear_frames_completed_callback () [inline]
```

4.3.4.8 clear_framesets_completed_callback()

```
void TCD1304Device::clear_framesets_completed_callback () [inline]
```

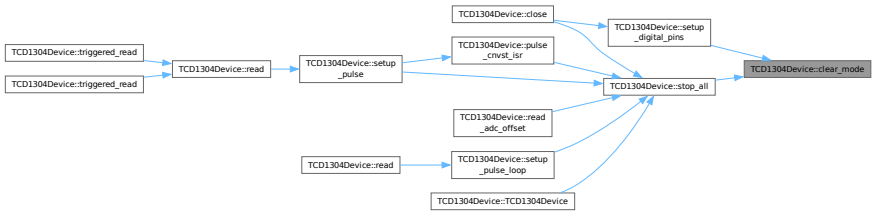
4.3.4.9 clear_ldok()

```
void TCD1304Device::clear_ldok (
            uint8_t mask,
            IMXRT_FLEXPWM_t * q = flexpwm) [inline]
```

4.3.4.10 clear_mode()

```
static void TCD1304Device::clear_mode () [inline], [static]
```

Here is the caller graph for this function:



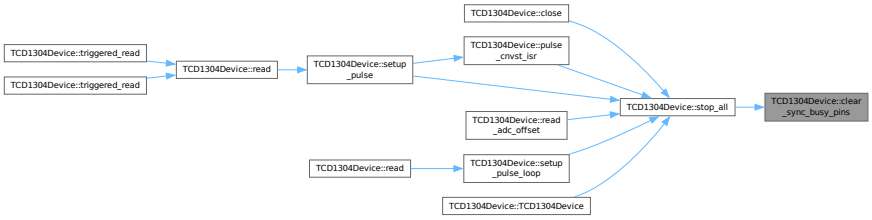
4.3.4.11 clear_run()

```
void TCD1304Device::clear_run (
            uint8_t mask,
            IMXRT_FLEXPWM_t * q = flexpwm) [inline]
```

4.3.4.12 clear_sync_busy_pins()

```
static void TCD1304Device::clear_sync_busy_pins () [inline], [static]
```

Here is the caller graph for this function:



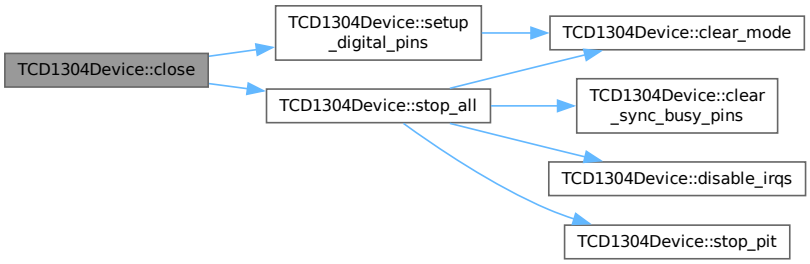
4.3.4.13 clear_syncpin()

```
static void TCD1304Device::clear_syncpin () [inline], [static]
```

4.3.4.14 close()

```
static void TCD1304Device::close () [inline], [static]
```

Here is the call graph for this function:



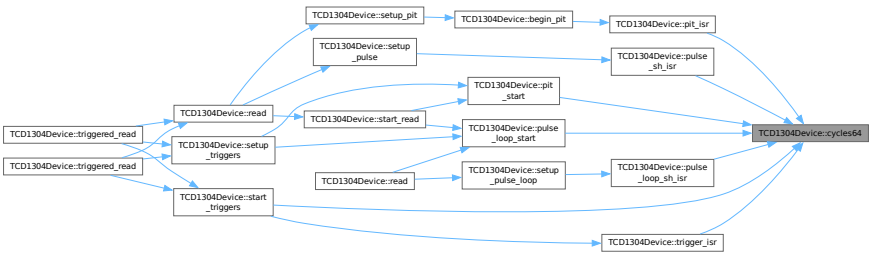
4.3.4.15 cnvst_start()

```
static void TCD1304Device::cnvst_start () [inline], [static]
```

4.3.4.16 cycles64()

```
static uint64_t TCD1304Device::cycles64 () [inline], [static]
```

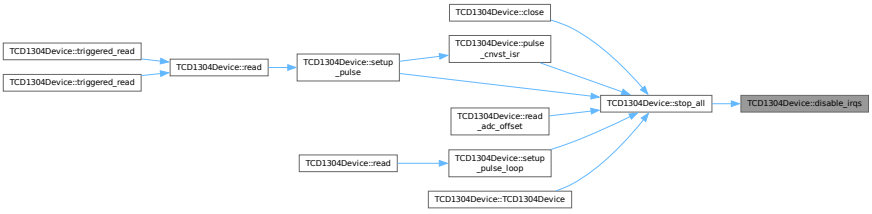
Here is the caller graph for this function:



4.3.4.17 disable_irqs()

```
static void TCD1304Device::disable_irqs () [inline], [static]
```

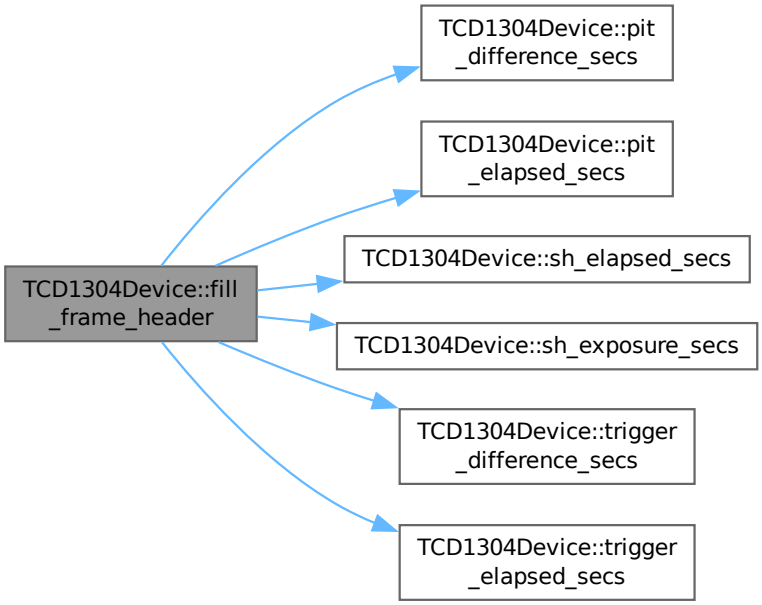
Here is the caller graph for this function:



4.3.4.18 fill_frame_header()

```
static void TCD1304Device::fill_frame_header (
            Frame_Header * p) [inline], [static]
```

Here is the call graph for this function:



4.3.4.19 flexpwm_resetcounters()

```
static void TCD1304Device::flexpwm_resetcounters (
            uint8_t mask = 0xFF)  [inline], [static]
```

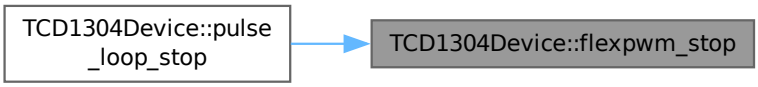
4.3.4.20 flexpwm_start()

```
static void TCD1304Device::flexpwm_start (
            uint8_t mask = 0xF)  [inline], [static]
```

4.3.4.21 flexpwm_stop()

```
static void TCD1304Device::flexpwm_stop (
            uint8_t mask = 0xF)  [inline], [static]
```

Here is the caller graph for this function:

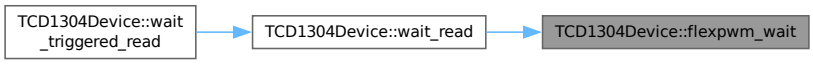


4.3.4.22 flexpwm_wait()

```
bool TCD1304Device::flexpwm_wait (
            float timeout_ = 1.,
            float timestep_ = 0.01,
            bool interruptible = false,
            bool verbose = false)  [inline]
```

Here is the call graph for this function:

Here is the caller graph for this function:



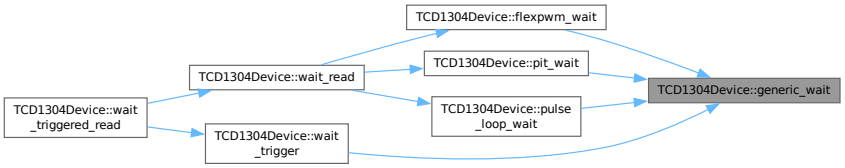
4.3.4.23 force()

```
void TCD1304Device::force (
            uint8_t submod,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.24 generic_wait()

```
bool TCD1304Device::generic_wait (
            bool * flag,
            float timeout_ = 1.,
            float timestep_ = 0.01,
            bool interruptible = false,
            bool verbose = false)  [inline]
```

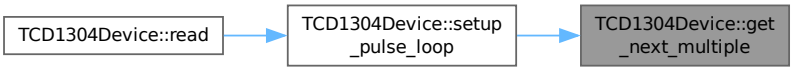
Here is the caller graph for this function:



4.3.4.25 get_next_multiple()

```
uint32_t TCD1304Device::get_next_multiple (
            uint32_t a,
            uint32_t b)  [inline]
```

Here is the caller graph for this function:



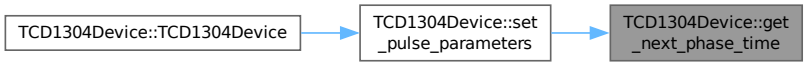
4.3.4.26 get_next_phase_tick()

```
uint16_t TCD1304Device::get_next_phase_tick (
            uint16_t t,
            uint32_t p,
            double phase_fraction)  [inline]
```

4.3.4.27 get_next_phase_time()

```
double TCD1304Device::get_next_phase_time (
            double t,
            double p,
            double phase_fraction)  [inline]
```

Here is the caller graph for this function:



4.3.4.28 load_frames_completed_callback()

```
void TCD1304Device::load_frames_completed_callback (
            void(* callback )(),
            unsigned int nframes = 0)  [inline]
```

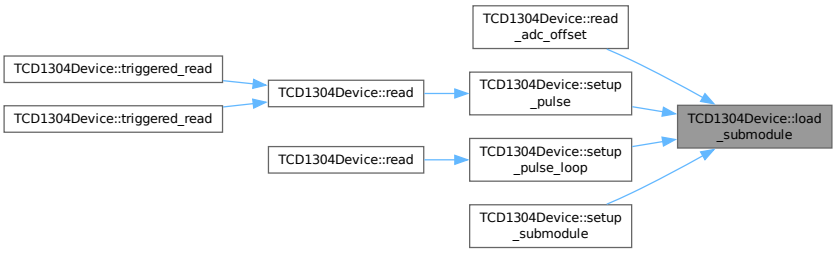
4.3.4.29 load_framesets_completed_callback()

```
void TCD1304Device::load_framesets_completed_callback (
            void(* callback )(),
            unsigned int nsets = 0)  [inline]
```

4.3.4.30 load_submodule()

```
void TCD1304Device::load_submodule (
            SubModule * p)  [inline]
```

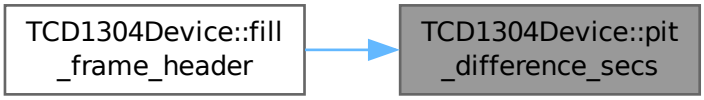
Here is the caller graph for this function:



4.3.4.31 pit_difference_secs()

```
static double TCD1304Device::pit_difference_secs () [inline], [static]
```

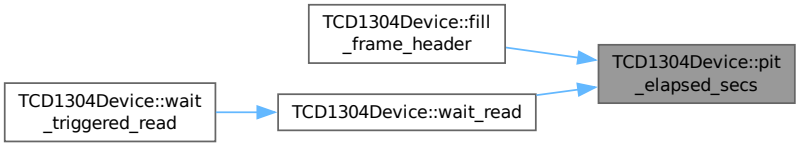
Here is the caller graph for this function:



4.3.4.32 pit_elapsed_secs()

```
static double TCD1304Device::pit_elapsed_secs () [inline], [static]
```

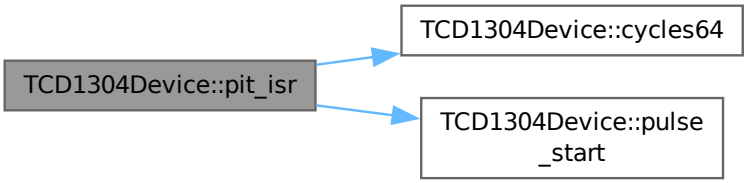
Here is the caller graph for this function:



4.3.4.33 pit_isr()

```
static void TCD1304Device::pit_isr () [inline], [static]
```

Here is the call graph for this function:

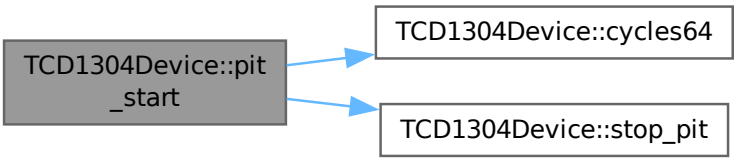


Here is the caller graph for this function:

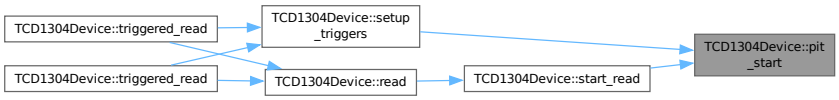
4.3.4.34 pit_start()

```
static void TCD1304Device::pit_start () [inline], [static]
```

Here is the call graph for this function:



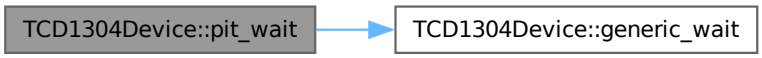
Here is the caller graph for this function:



4.3.4.35 pit_wait()

```
bool TCD1304Device::pit_wait (
            float timeout_ = 1.,
            float timestep_ = 0.01,
            bool interruptible = false,
            bool verbose = false) [inline]
```

Here is the call graph for this function:

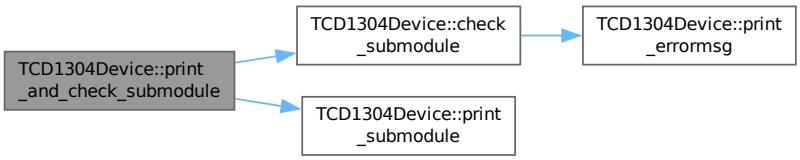


Here is the caller graph for this function:

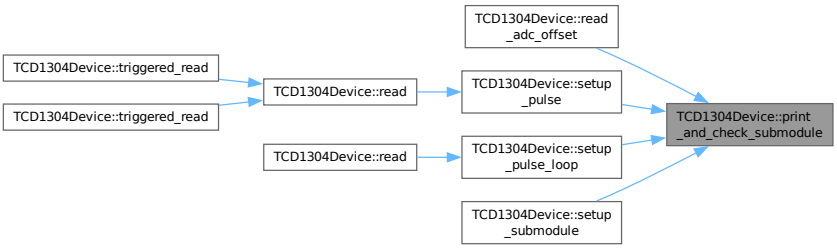
4.3.4.36 print_and_check_submodule()

```
bool TCD1304Device::print_and_check_submodule (
            SubModule * p)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



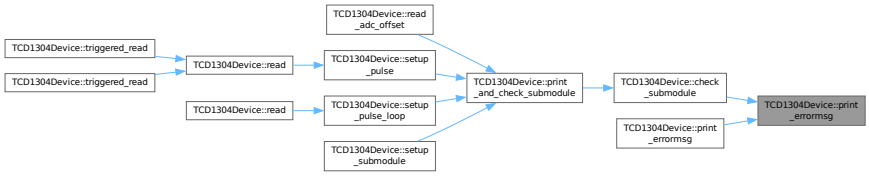
4.3.4.37 print_counters()

```
static void TCD1304Device::print_counters ()  [inline], [static]
```

4.3.4.38 print_errormsg() [1/2]

```
static void TCD1304Device::print_errormsg (
            const char * name,
            const char * errmsg)  [inline], [static]
```

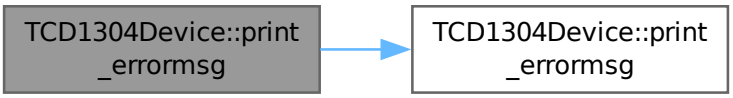
Here is the caller graph for this function:



4.3.4.39 print_errormsg() [2/2]

```
static void TCD1304Device::print_errormsg (
            SubModule * p,
            const char * errmsg)  [inline], [static]
```

Here is the call graph for this function:



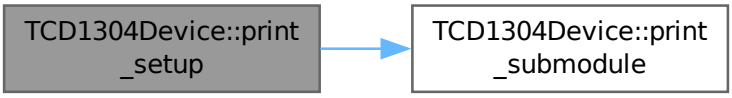
4.3.4.40 print_pulse_parameters()

```
void TCD1304Device::print_pulse_parameters ()  [inline]
```

4.3.4.41 print_setup()

```
void TCD1304Device::print_setup () [inline]
```

Here is the call graph for this function:



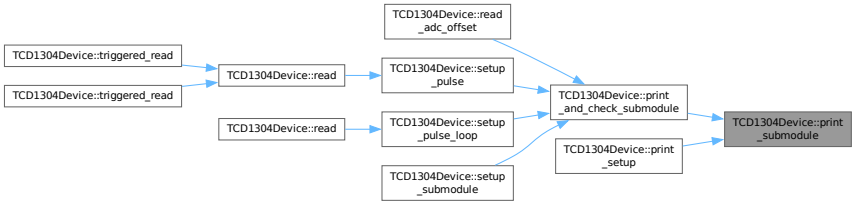
4.3.4.42 print_setup_diagnostic()

```
void TCD1304Device::print_setup_diagnostic (
            SubModule * p) [inline]
```

4.3.4.43 print_submodule()

```
void TCD1304Device::print_submodule (
            SubModule * p) [inline]
```

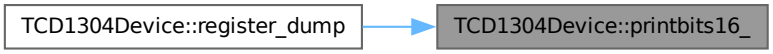
Here is the caller graph for this function:



4.3.4.44 printbits16_()

```
void TCD1304Device::printbits16_ (
            uint16_t u16,
            int bits)  [inline]
```

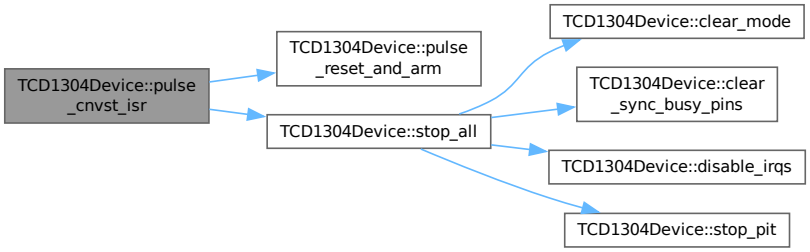
Here is the caller graph for this function:



4.3.4.45 pulse_cnvst_isr()

```
static void TCD1304Device::pulse_cnvst_isr ()  [inline], [static]
```

Here is the call graph for this function:



Here is the caller graph for this function:

4.3.4.46 pulse_icg_isr()

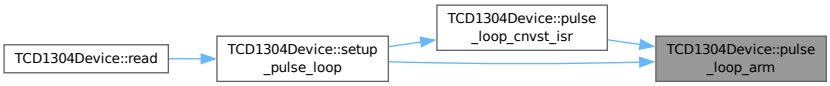
```
static void TCD1304Device::pulse_icg_isr () [inline], [static]
```

Here is the caller graph for this function:

4.3.4.47 pulse_loop_arm()

```
static void TCD1304Device::pulse_loop_arm () [inline], [static]
```

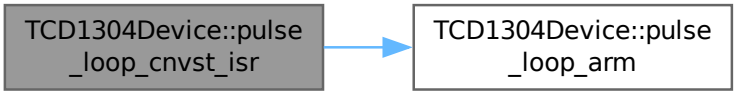
Here is the caller graph for this function:



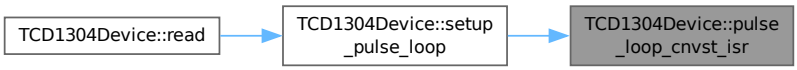
4.3.4.48 pulse_loop_cnvst_isr()

```
static void TCD1304Device::pulse_loop_cnvst_isr () [inline], [static]
```

Here is the call graph for this function:



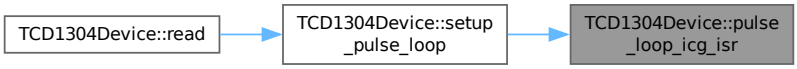
Here is the caller graph for this function:



4.3.4.49 pulse_loop_icg_isr()

```
static void TCD1304Device::pulse_loop_icg_isr ()  [inline], [static]
```

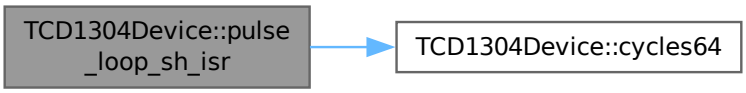
Here is the caller graph for this function:



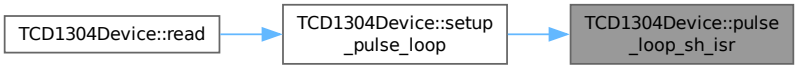
4.3.4.50 pulse_loop_sh_isr()

```
static void TCD1304Device::pulse_loop_sh_isr ()  [inline], [static]
```

Here is the call graph for this function:



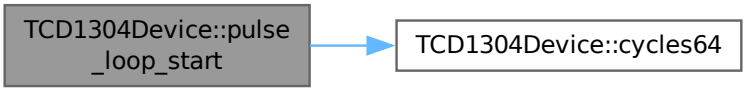
Here is the caller graph for this function:



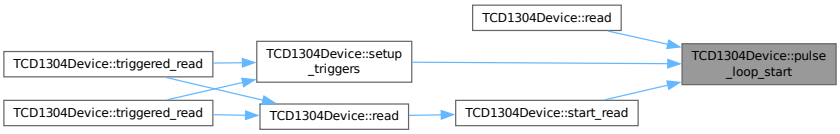
4.3.4.51 pulse_loop_start()

```
static void TCD1304Device::pulse_loop_start () [inline], [static]
```

Here is the call graph for this function:



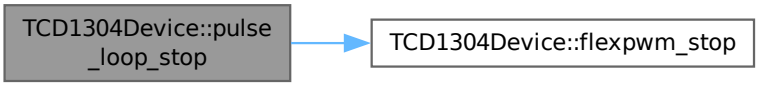
Here is the caller graph for this function:



4.3.4.52 pulse_loop_stop()

```
static void TCD1304Device::pulse_loop_stop () [inline], [static]
```

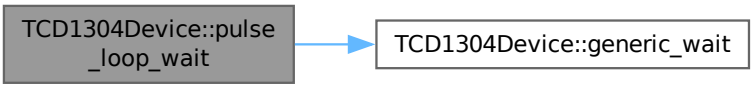
Here is the call graph for this function:



4.3.4.53 pulse_loop_wait()

```
bool TCD1304Device::pulse_loop_wait (
            float timeout_ = 1.,
            float timestep_ = 0.01,
            bool interruptible = false,
            bool verbose = false)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:

4.3.4.54 pulse_reset_and_arm()

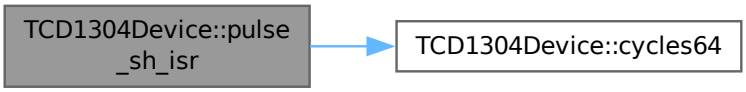
```
static void TCD1304Device::pulse_reset_and_arm ()  [inline], [static]
```

Here is the caller graph for this function:

4.3.4.55 pulse_sh_isr()

```
static void TCD1304Device::pulse_sh_isr () [inline], [static]
```

Here is the call graph for this function:

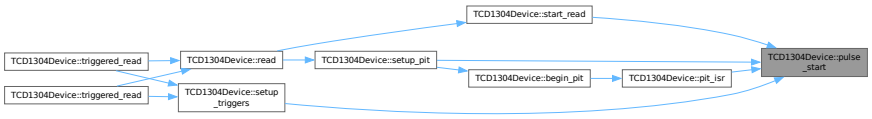


Here is the caller graph for this function:

4.3.4.56 pulse_start()

```
static void TCD1304Device::pulse_start () [inline], [static]
```

Here is the caller graph for this function:

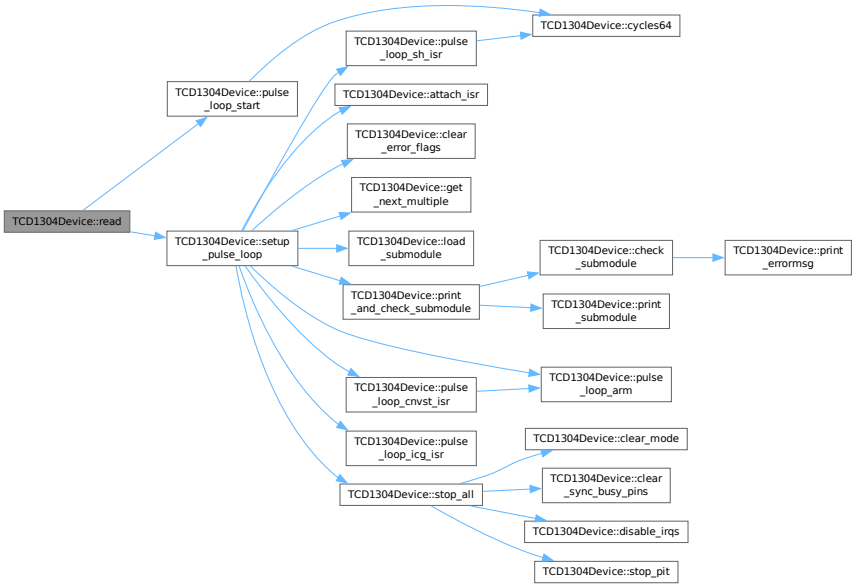


4.3.4.57 read() [1/2]

```
bool TCD1304Device::read (
            uint nframes,
            float exposure,
            float frame_interval,
            uint16_t * bufferp,
```

```
            void(* frame_callbackf )(),
            void(* frameset_callbackf )(),
            void(* completion_callbackf )(),
            void(* setup_callbackf )(),
            bool start = true)  [inline]
```

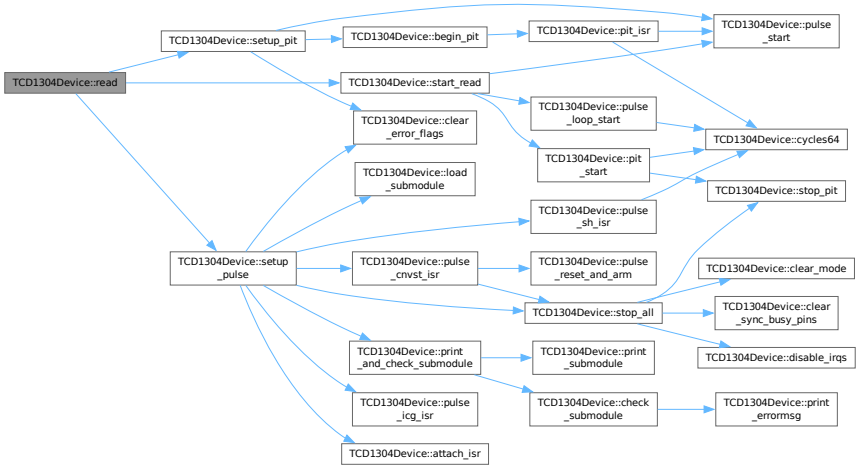
Here is the call graph for this function:



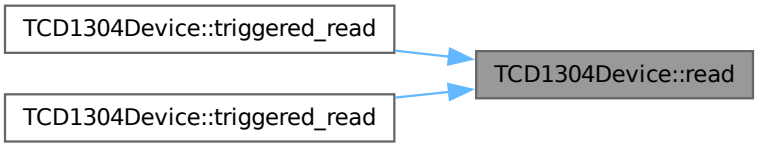
4.3.4.58 read() [2/2]

```
bool TCD1304Device::read (
            uint nframes,
            float exposure,
            uint16_t * bufferp,
            void(* frame_callbackf )(),
            void(* frameset_callbackf )(),
            void(* completion_callbackf )(),
            void(* setup_callbackf )(),
            bool start = true)  [inline]
```

Here is the call graph for this function:



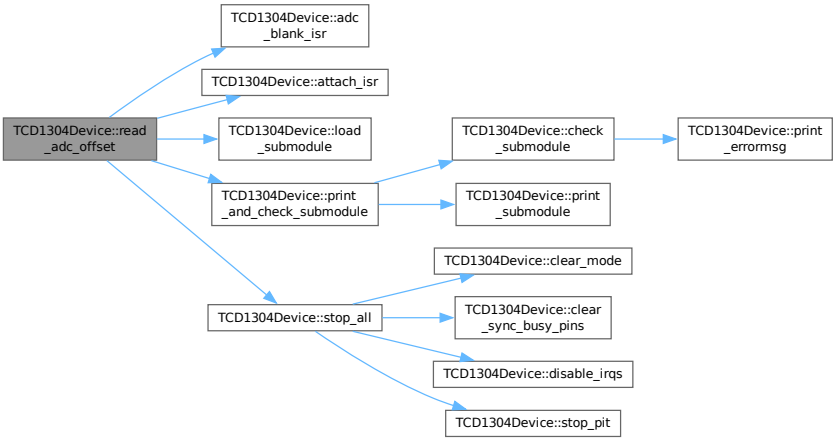
Here is the caller graph for this function:



4.3.4.59 read_adc_offset()

```
bool TCD1304Device::read_adc_offset () [inline]
```

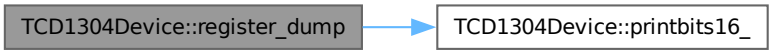
Here is the call graph for this function:



4.3.4.60 register_dump()

```
void TCD1304Device::register_dump (
            IMXRT_FLEXPWM_t *const q = flexpwm,
            uint8_t mask = 0xFF)  [inline]
```

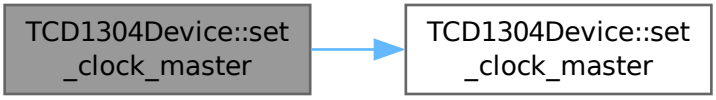
Here is the call graph for this function:



4.3.4.61 set_clock_master() [1/2]

```
void TCD1304Device::set_clock_master (
            SubModule * p)  [inline]
```

Here is the call graph for this function:

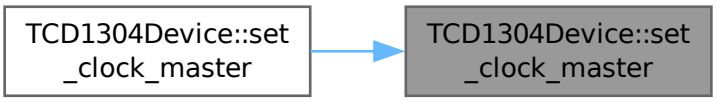


4.3.4.62 set_clock_master() [2/2]

```
void TCD1304Device::set_clock_master (
            uint8_t submod,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

Low level functions for command line register access.

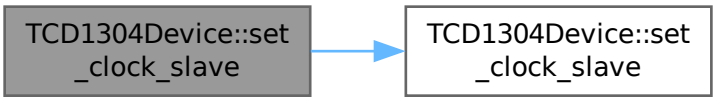
Here is the caller graph for this function:



4.3.4.63 set_clock_slave() [1/2]

```
void TCD1304Device::set_clock_slave (
            SubModule * p)  [inline]
```

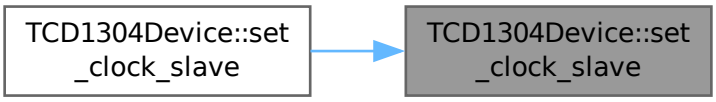
Here is the call graph for this function:



4.3.4.64 set_clock_slave() [2/2]

```
void TCD1304Device::set_clock_slave (
            uint8_t submod,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

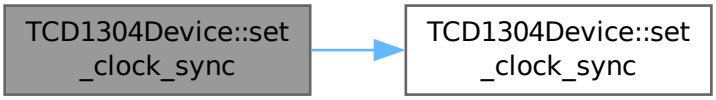
Here is the caller graph for this function:



4.3.4.65 set_clock_sync() [1/2]

```
void TCD1304Device::set_clock_sync (
            SubModule * p)  [inline]
```

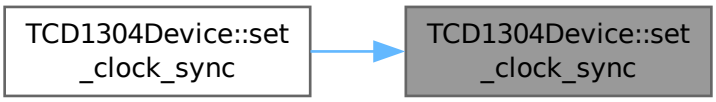
Here is the call graph for this function:



4.3.4.66 set_clock_sync() [2/2]

```
void TCD1304Device::set_clock_sync (
            uint8_t submod,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

Here is the caller graph for this function:



4.3.4.67 set_cnt()

```
void TCD1304Device::set_cnt (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.68 set_init()

```
void TCD1304Device::set_init (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.69 set_ldok()

```
void TCD1304Device::set_ldok (
            uint8_t mask,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.70 set_outen()

```
uint16_t TCD1304Device::set_outen (
            uint16_t mask16,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.71 set_outen_off()

```
uint16_t TCD1304Device::set_outen_off (
            uint16_t mask16,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.72 set_outen_on()

```
uint16_t TCD1304Device::set_outen_on (
            uint16_t mask16,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.73 set_outenA_on()

```
uint16_t TCD1304Device::set_outenA_on (
            uint8_t mask8,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

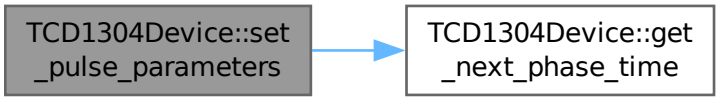
4.3.4.74 set_prescale()

```
void TCD1304Device::set_prescale (
            uint8_t submod,
            uint8_t prescale = 0,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

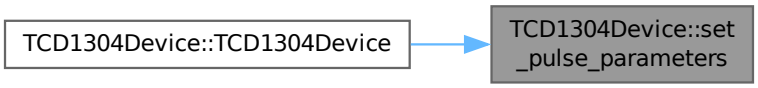
4.3.4.75 set_pulse_parameters()

```
bool TCD1304Device::set_pulse_parameters (
            float clock_period_secs = 0.4E-6,
            float period_secs = 10.E-6)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4.76 set_run()

```
void TCD1304Device::set_run (
            uint8_t mask,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.77 set_val0()

```
void TCD1304Device::set_val0 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.78 set_val1()

```
void TCD1304Device::set_val1 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.79 set_val2()

```
void TCD1304Device::set_val2 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.80 set_val3()

```
void TCD1304Device::set_val3 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

4.3.4.81 set_val4()

```
void TCD1304Device::set_val4 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

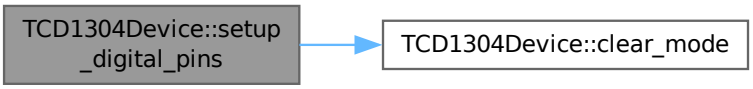
4.3.4.82 set_val5()

```
void TCD1304Device::set_val5 (
            uint8_t submod,
            uint16_t value,
            IMXRT_FLEXPWM_t * q = flexpwm)  [inline]
```

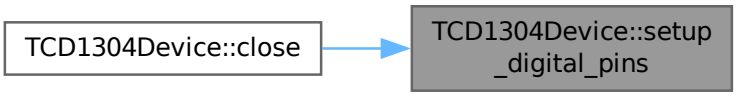
4.3.4.83 setup_digital_pins()

```
static void TCD1304Device::setup_digital_pins ()  [inline], [static]
```

Here is the call graph for this function:



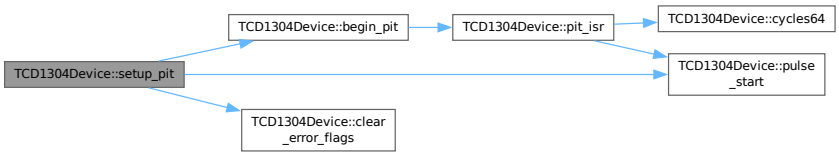
Here is the caller graph for this function:



4.3.4.84 setup_pit()

```
bool TCD1304Device::setup_pit (
            float exposure_secs,
            float delay_secs = 1.E-6,
            unsigned int ncounts = 0)  [inline]
```

Here is the call graph for this function:

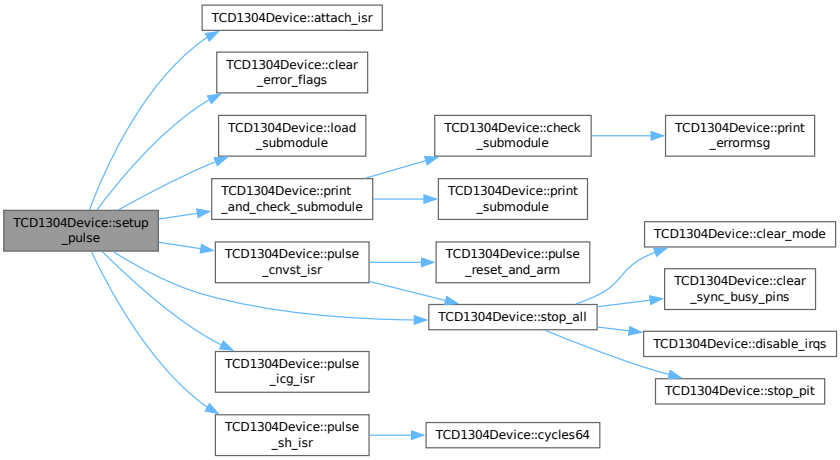


Here is the caller graph for this function:

4.3.4.85 setup_pulse()

```
bool TCD1304Device::setup_pulse (
            float clk_secs,
            float sh_secs,
            float sh_offset_secs,
            float icg_secs,
            float icg_offset_secs,
            uint16_t * buffer,
            unsigned int nbuffer,
            void(* callbackf )())  [inline]
```

Here is the call graph for this function:

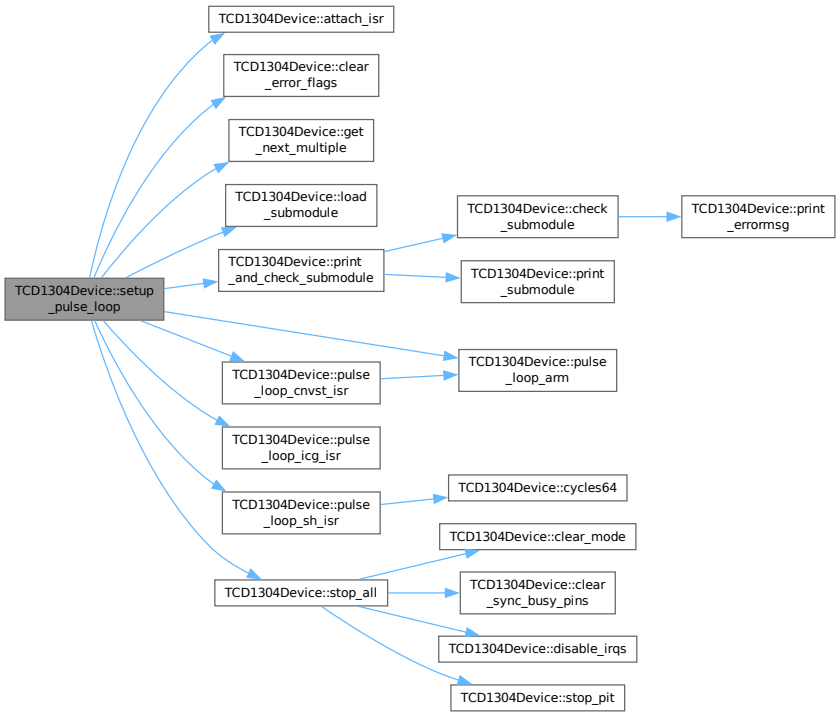


Here is the caller graph for this function:

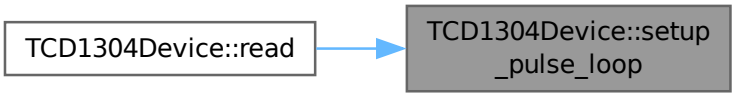
4.3.4.86 setup_pulse_loop()

```
bool TCD1304Device::setup_pulse_loop (
            float clk_secs,
            float sh_secs,
            float sh_offset_secs,
            float icg_secs,
            float icg_offset_secs,
            float period_secs,
            float exposure_secs,
            float frame_secs,
            uint nframes,
            uint16_t * buffer,
            unsigned int nbuffer,
            void(* callbackf )())  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:

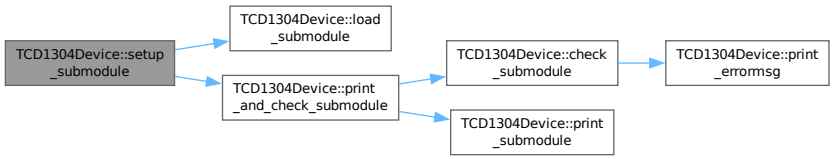


4.3.4.87 setup_submodule()

```
bool TCD1304Device::setup_submodule (
            SubModule * p,
            uint8_t prescale,
            uint32_t period_counts,
            uint16_t onA_counts,
            uint16_t offA_counts,
```

```
            bool invertA,
            uint16_t onB_counts,
            uint16_t offB_counts,
            bool invertB,
            uint16_t ctrl2_mask)  [inline]
```

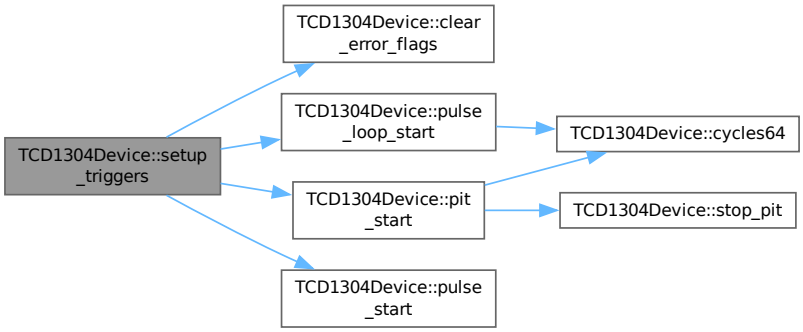
Here is the call graph for this function:



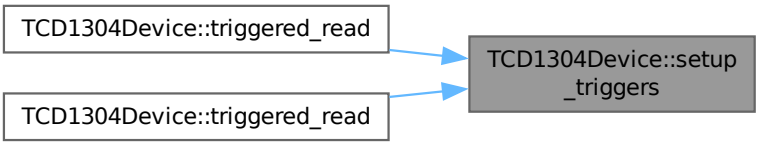
4.3.4.88 setup_triggers()

```
bool TCD1304Device::setup_triggers (
            unsigned int ncounts = 0)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4.89 sh_difference_secs()

```
static double TCD1304Device::sh_difference_secs () [inline], [static]
```

4.3.4.90 sh_elapsed_secs()

```
static double TCD1304Device::sh_elapsed_secs () [inline], [static]
```

Here is the caller graph for this function:

4.3.4.91 sh_exposure_secs()

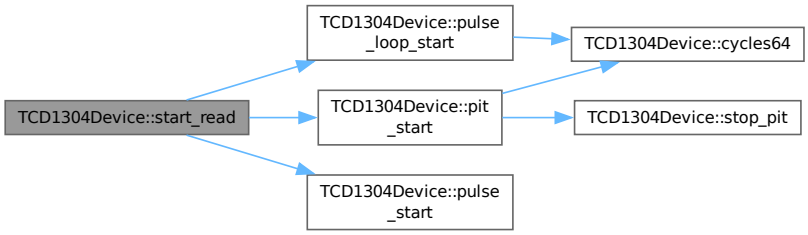
```
static double TCD1304Device::sh_exposure_secs () [inline], [static]
```

Here is the caller graph for this function:

4.3.4.92 start_read()

```
static void TCD1304Device::start_read () [inline], [static]
```

Here is the call graph for this function:

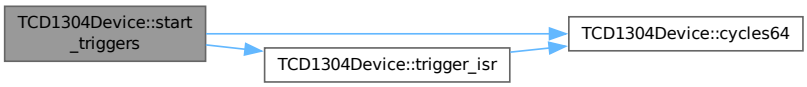


Here is the caller graph for this function:

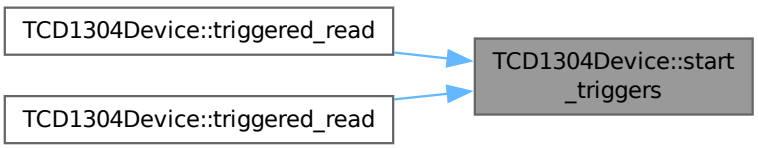
4.3.4.93 start_triggers()

```
bool TCD1304Device::start_triggers () [inline]
```

Here is the call graph for this function:



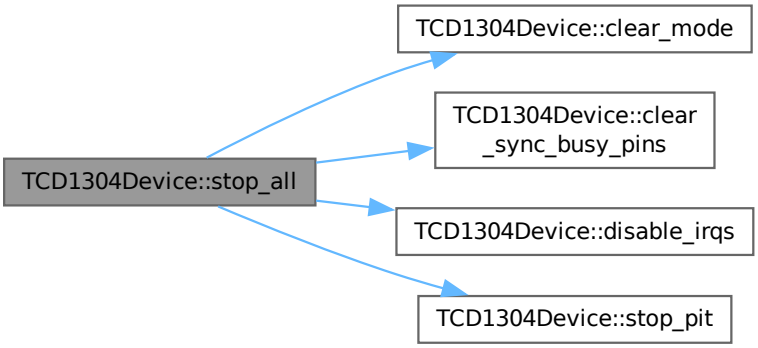
Here is the caller graph for this function:



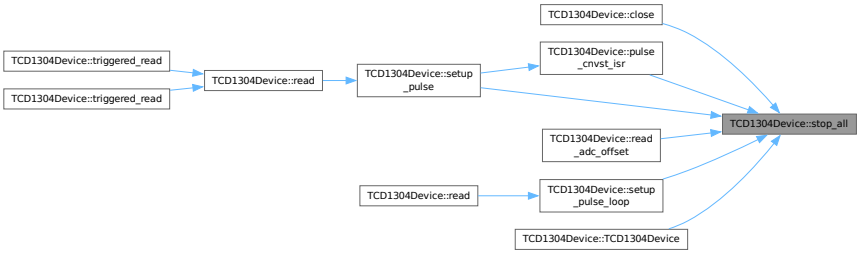
4.3.4.94 stop_all()

```
static void TCD1304Device::stop_all () [inline], [static]
```

Here is the call graph for this function:



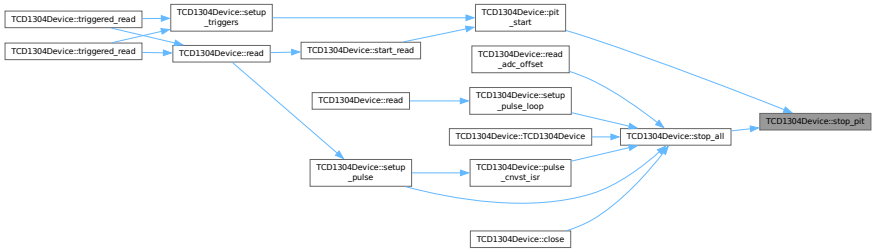
Here is the caller graph for this function:



4.3.4.95 stop_pit()

```
static void TCD1304Device::stop_pit () [inline], [static]
```

Here is the caller graph for this function:



4.3.4.96 stop_triggers()

```
static void TCD1304Device::stop_triggers () [inline], [static]
```

4.3.4.97 toggle_busypin()

```
static void TCD1304Device::toggle_busypin () [inline], [static]
```

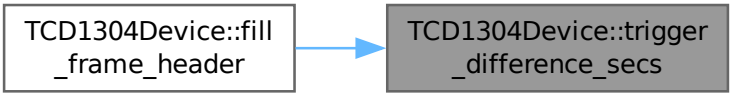
4.3.4.98 toggle_syncpin()

```
static void TCD1304Device::toggle_syncpin () [inline], [static]
```

4.3.4.99 trigger_difference_secs()

```
static double TCD1304Device::trigger_difference_secs () [inline], [static]
```

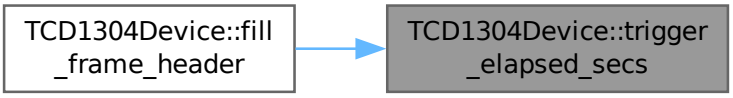
Here is the caller graph for this function:



4.3.4.100 trigger_elapsed_secs()

```
static double TCD1304Device::trigger_elapsed_secs () [inline], [static]
```

Here is the caller graph for this function:

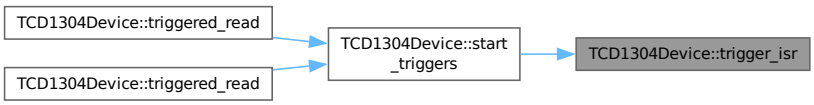


4.3.4.101 trigger_isr()

```
static void TCD1304Device::trigger_isr () [inline], [static]
```

Here is the call graph for this function:

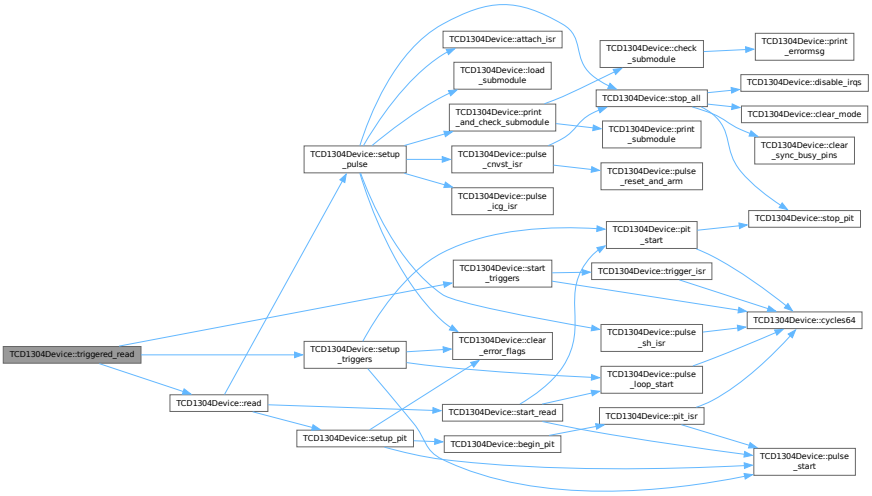
Here is the caller graph for this function:



4.3.4.102 triggered_read() [1/2]

```
bool TCD1304Device::triggered_read (
            uint ntriggers,
            uint nframes,
            float exposure,
            float interval,
            uint16_t * bufferp,
            void(* frame_callbackf )(),
            void(* frameset_callbackf )(),
            void(* completion_callbackf )(),
            void(* setup_callbackf )(),
            bool start = true)  [inline]
```

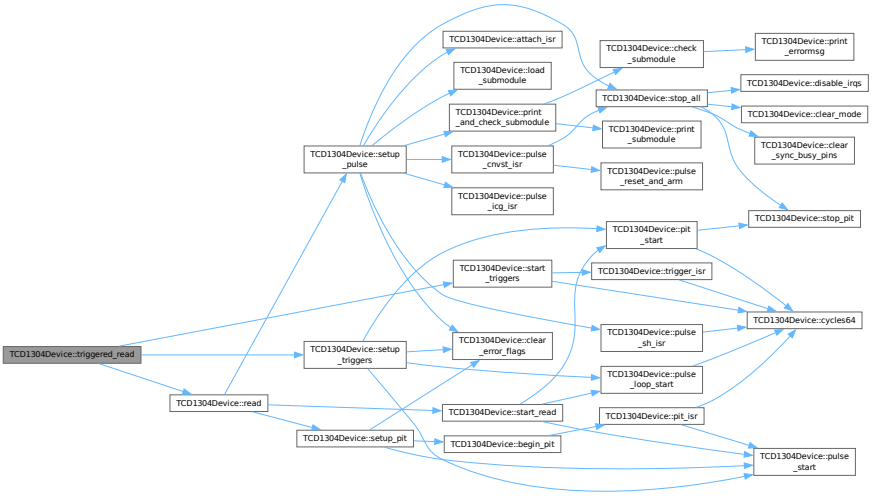
Here is the call graph for this function:



4.3.4.103 triggered_read() [2/2]

```
bool TCD1304Device::triggered_read (
            uint ntriggers,
            uint nframes,
            float exposure,
            uint16_t * bufferp,
            void(* frame_callbackf )(),
            void(* frameset_callbackf )(),
            void(* completion_callbackf )(),
            void(* setup_callbackf )(),
            bool start = true)  [inline]
```

Here is the call graph for this function:



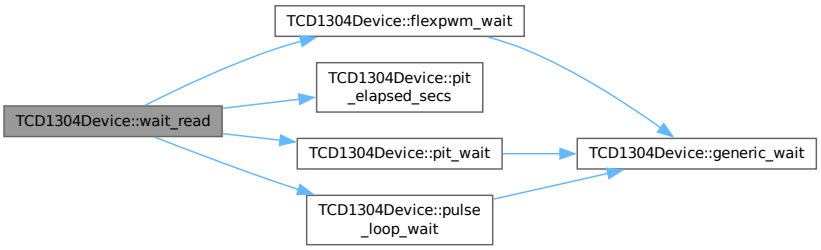
4.3.4.104 update_read_buffer()

```
static void TCD1304Device::update_read_buffer (
            uint16_t * buffer)  [inline], [static]
```

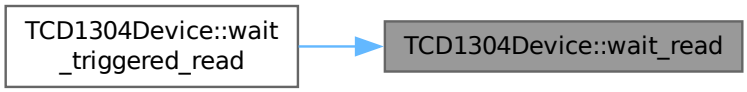
4.3.4.105 wait_read()

```
bool TCD1304Device::wait_read (
            float timeout = 0.,
            float timestep = 0.01,
            bool interruptible = false,
            bool verbose = false)  [inline]
```

Here is the call graph for this function:



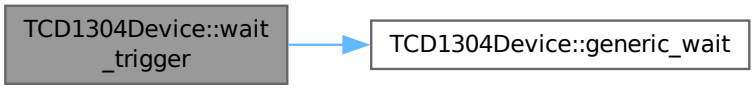
Here is the caller graph for this function:



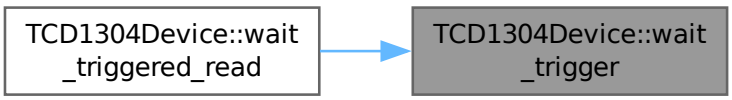
4.3.4.106 wait_trigger()

```
bool TCD1304Device::wait_trigger (
            float timeout_ = 1.,
            float timestep_ = 0.01,
            bool interruptible = false,
            bool verbose = false)  [inline]
```

Here is the call graph for this function:



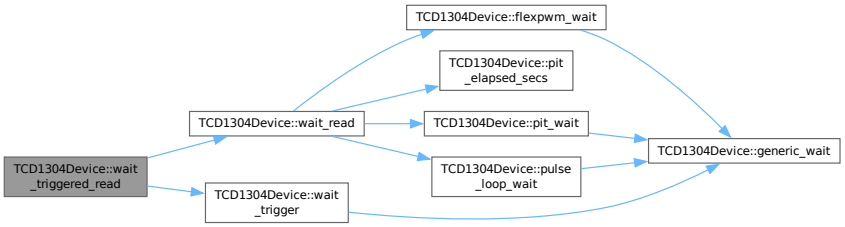
Here is the caller graph for this function:



4.3.4.107 wait_triggered_read()

```
bool TCD1304Device::wait_triggered_read (
            float timeout = 1.,
            float timestep = 0.01,
            bool interruptible = true,
            bool verbose = false)  [inline]
```

Here is the call graph for this function:



4.3.5 Member Data Documentation

4.3.5.1 adc_blank_counter

```
uint TCD1304Device::adc_blank_counter = 0  [inline], [static]
```

4.3.5.2 adc_blank_running

```
bool TCD1304Device::adc_blank_running = false  [inline], [static]
```

4.3.5.3 adc_offset

```
double TCD1304Device::adc_offset = 0.  [inline], [static]
```

4.3.5.4 adc_offset_done

```
bool TCD1304Device::adc_offset_done = false  [inline], [static]
```

4.3.5.5 adc_raw_offset

```
uint TCD1304Device::adc_raw_offset = 0  [inline], [static]
```

4.3.5.6 busytoggled

```
bool TCD1304Device::busytoggled = false  [inline], [static]
```

4.3.5.7 clk

```
SubModule TCD1304Device::clk = {"clk",CLK_SUBMODULE, CLK_MASK, CLK_PIN,CLK_MUXVAL, 0xFF,0, CLK_IRQ,
&IMXRT_FLEXPWM2}  [inline], [static]
```

4.3.5.8 cnvst

```
SubModule TCD1304Device::cnvst = {"cnvst", CNVST_SUBMODULE, CNVST_MASK, 0xFF,0, 0xFF,0, CNVST_IRQ,
&IMXRT_FLEXPWM2}  [inline], [static]
```

4.3.5.9 cnvst_counter

```
unsigned int TCD1304Device::cnvst_counter = 0  [inline], [static]
```

4.3.5.10 cyccnt64_prepare_exposure

```
uint64_t TCD1304Device::cyccnt64_prepare_exposure = 0  [inline], [static]
```

4.3.5.11 cyccnt64_prepare_readout

```
uint64_t TCD1304Device::cyccnt64_prepare_readout = 0  [inline], [static]
```

4.3.5.12 error_flag

```
bool TCD1304Device::error_flag = false  [inline], [static]
```

4.3.5.13 flexpwm

```
IMXRT_FLEXPWM_t* const TCD1304Device::flexpwm = &IMXRT_FLEXPWM2  [inline], [static]
```

4.3.5.14 flexpwm_running

```
bool TCD1304Device::flexpwm_running = false  [inline], [static]
```

4.3.5.15 frame_counter

```
unsigned int TCD1304Device::frame_counter = 0  [inline], [static]
```

4.3.5.16 frame_counts

```
unsigned int TCD1304Device::frame_counts = 0  [inline], [static]
```

4.3.5.17 frames_complete

```
bool TCD1304Device::frames_complete = false  [inline], [static]
```

4.3.5.18 frames_completed_callback

```
void(* TCD1304Device::frames_completed_callback) ()=0  [inline], [static]
```

4.3.5.19 frameset_counter

```
unsigned int TCD1304Device::frameset_counter = 0  [inline], [static]
```

4.3.5.20 frameset_counts

```
unsigned int TCD1304Device::frameset_counts = 0  [inline], [static]
```

4.3.5.21 framesets_complete

```
bool TCD1304Device::framesets_complete = false  [inline], [static]
```

4.3.5.22 framesets_completed_callback

```
void(* TCD1304Device::framesets_completed_callback) ()=0  [inline], [static]
```

4.3.5.23 icg

```
SubModule TCD1304Device::icg = {"icg", ICG_SUBMODULE, ICG_MASK, ICG_PIN,ICG_MUXVAL, 0xFF,0, ICG_IRQ,
&IMXRT_FLEXPWM2}  [inline], [static]
```

4.3.5.24 icg_counter

```
unsigned int TCD1304Device::icg_counter = 0  [inline], [static]
```

4.3.5.25 mode

```
TCD1304_Mode_t TCD1304Device::mode = NOTCONFIGURED  [inline], [static]
```

4.3.5.26 oops_flag

```
bool TCD1304Device::oops_flag = false  [inline], [static]
```

4.3.5.27 oopsbuffer

```
char TCD1304Device::oopsbuffer[128] = { 0 }
```

4.3.5.28 pit_begun

```
bool TCD1304Device::pit_begun = false  [inline], [static]
```

4.3.5.29 pit_callback

```
void(* TCD1304Device::pit_callback) ()=0  [inline], [static]
```

4.3.5.30 pit_counter

```
unsigned int TCD1304Device::pit_counter = 0  [inline], [static]
```

4.3.5.31 pit_counts

```
unsigned int TCD1304Device::pit_counts = 2  [inline], [static]
```

4.3.5.32 pit_cyccnt64_now

```
uint64_t TCD1304Device::pit_cyccnt64_now = 0  [inline], [static]
```

4.3.5.33 pit_cyccnt64_prev

```
uint64_t TCD1304Device::pit_cyccnt64_prev = 0  [inline], [static]
```

4.3.5.34 pit_cyccnt64_start

```
uint64_t TCD1304Device::pit_cyccnt64_start = 0  [inline], [static]
```

4.3.5.35 pit_delay_counts

```
uint32_t TCD1304Device::pit_delay_counts = 0  [inline], [static]
```

4.3.5.36 pit_delay_secs

```
float TCD1304Device::pit_delay_secs = 1.E-6  [inline], [static]
```

4.3.5.37 pit_interval_counts

```
uint32_t TCD1304Device::pit_interval_counts = 0  [inline], [static]
```

4.3.5.38 pit_interval_secs

```
float TCD1304Device::pit_interval_secs = 0.1  [inline], [static]
```

4.3.5.39 pit_running

```
bool TCD1304Device::pit_running = false  [inline], [static]
```

4.3.5.40 pit_state

```
uint8_t TCD1304Device::pit_state = 0  [inline], [static]
```

4.3.5.41 pulse_armed

```
bool TCD1304Device::pulse_armed = false  [inline], [static]
```

Configure the flexpwm for single pulse, use this with a timer or interrupt.

4.3.5.42 pulse_clk_secs

```
float TCD1304Device::pulse_clk_secs = 0.  [inline], [static]
```

4.3.5.43 pulse_cnvst_offset_secs

```
float TCD1304Device::pulse_cnvst_offset_secs = 0.  [inline], [static]
```

4.3.5.44 pulse_cnvst_secs

```
float TCD1304Device::pulse_cnvst_secs = 0.  [inline], [static]
```

4.3.5.45 pulse_icg_offset_secs

```
float TCD1304Device::pulse_icg_offset_secs = 0.  [inline], [static]
```

4.3.5.46 pulse_icg_secs

```
float TCD1304Device::pulse_icg_secs = 0.  [inline], [static]
```

4.3.5.47 pulse_interframe_min_secs

```
float TCD1304Device::pulse_interframe_min_secs = 0.  [inline], [static]
```

4.3.5.48 pulse_loop_armed

```
bool TCD1304Device::pulse_loop_armed = false  [inline], [static]
```

4.3.5.49 pulse_loop_frame_min_secs

```
float TCD1304Device::pulse_loop_frame_min_secs = 0.  [inline], [static]
```

4.3.5.50 pulse_loop_period_secs

```
float TCD1304Device::pulse_loop_period_secs = 0.  [inline], [static]
```

4.3.5.51 pulse_loop_running

```
bool TCD1304Device::pulse_loop_running = false  [inline], [static]
```

4.3.5.52 pulse_loop_state

```
PULSE_LOOP_STATE TCD1304Device::pulse_loop_state = IDLE  [inline], [static]
```

4.3.5.53 pulse_sh_offset_secs

```
float TCD1304Device::pulse_sh_offset_secs = 0.  [inline], [static]
```

4.3.5.54 pulse_sh_secs

```
float TCD1304Device::pulse_sh_secs = 0.  [inline], [static]
```

4.3.5.55 read_buffer

```
uint16_t* TCD1304Device::read_buffer = 0  [inline], [static]
```

4.3.5.56 read_callback

```
void(* TCD1304Device::read_callback) ()=0  [inline], [static]
```

4.3.5.57 read_counter

```
unsigned int TCD1304Device::read_counter = 0  [inline], [static]
```

4.3.5.58 read_counts

```
unsigned int TCD1304Device::read_counts = 0  [inline], [static]
```

4.3.5.59 read_expected_time

```
float TCD1304Device::read_expected_time = 0.  [inline], [static]
```

4.3.5.60 read_pointer

```
uint16_t* TCD1304Device::read_pointer = 0  [inline], [static]
```

4.3.5.61 sh

```
SubModule TCD1304Device::sh = {"sh", SH_SUBMODULE, SH_MASK, SH_PIN,SH_MUXVAL, 0xFF,0, SH_IRQ,
&IMXRT_FLEXPWM2}  [inline], [static]
```

4.3.5.62 sh_clearing_counter

```
unsigned int TCD1304Device::sh_clearing_counter = 0  [inline], [static]
```

4.3.5.63 sh_clearing_counts

```
unsigned int TCD1304Device::sh_clearing_counts = SH_CLEARING_DEFAULT  [inline], [static]
```

4.3.5.64 sh_counter

```
unsigned int TCD1304Device::sh_counter = 0  [inline], [static]
```

4.3.5.65 sh_counts_per_icg

```
unsigned int TCD1304Device::sh_counts_per_icg = 0  [inline], [static]
```

4.3.5.66 sh_cyccnt64_exposure

```
uint64_t TCD1304Device::sh_cyccnt64_exposure = 0  [inline], [static]
```

4.3.5.67 sh_cyccnt64_now

```
uint64_t TCD1304Device::sh_cyccnt64_now = 0  [inline], [static]
```

4.3.5.68 sh_cyccnt64_prev

```
uint64_t TCD1304Device::sh_cyccnt64_prev = 0  [inline], [static]
```

4.3.5.69 sh_cyccnt64_start

```
uint64_t TCD1304Device::sh_cyccnt64_start = 0  [inline], [static]
```

4.3.5.70 sh_short_period_counts

```
unsigned int TCD1304Device::sh_short_period_counts = 0  [inline], [static]
```

4.3.5.71 sync_enabled

```
bool TCD1304Device::sync_enabled = true  [inline], [static]
```

4.3.5.72 sync_pin

```
uint8_t TCD1304Device::sync_pin = SYNC_PIN  [inline], [static]
```

4.3.5.73 synctoggled

```
bool TCD1304Device::synctoggled = false  [inline], [static]
```

4.3.5.74 trigger_attached

```
bool TCD1304Device::trigger_attached = false  [inline], [static]
```

4.3.5.75 trigger_busy

```
bool TCD1304Device::trigger_busy = false  [inline], [static]
```

4.3.5.76 trigger_callback

```
void(* TCD1304Device::trigger_callback) ()=0  [inline], [static]
```

4.3.5.77 trigger_counter

```
unsigned int TCD1304Device::trigger_counter = 0  [inline], [static]
```

4.3.5.78 trigger_counts

```
unsigned int TCD1304Device::trigger_counts = 1  [inline], [static]
```

4.3.5.79 trigger_cyccnt64_now

```
uint64_t TCD1304Device::trigger_cyccnt64_now = 0  [inline], [static]
```

4.3.5.80 trigger_cyccnt64_prev

```
uint64_t TCD1304Device::trigger_cyccnt64_prev = 0  [inline], [static]
```

4.3.5.81 trigger_cyccnt64_start

```
uint64_t TCD1304Device::trigger_cyccnt64_start = 0  [inline], [static]
```

4.3.5.82 trigger_edge_mode

```
uint8_t TCD1304Device::trigger_edge_mode = RISING  [inline], [static]
```

4.3.5.83 trigger_mode

```
bool TCD1304Device::trigger_mode = false  [inline], [static]
```

4.3.5.84 trigger_pin

```
uint8_t TCD1304Device::trigger_pin = TRIGGER_PIN  [inline], [static]
```

4.3.5.85 trigger_pin_mode

```
uint8_t TCD1304Device::trigger_pin_mode = INPUT  [inline], [static]
```

The documentation for this class was generated from the following file:

- TCD1304Device2c.h

4.4 usb_string_descriptor_struct_manufacturer Struct Reference

Public Attributes

- uint8_t bLength
- uint8_t bDescriptorType
- uint16_t wString [thisMANUFACTURER_NAME_LEN]

4.4.1 Member Data Documentation

4.4.1.1 bDescriptorType

```
uint8_t usb_string_descriptor_struct_manufacturer::bDescriptorType
```

4.4.1.2 bLength

```
uint8_t usb_string_descriptor_struct_manufacturer::bLength
```

4.4.1.3 wString

```
uint16_t usb_string_descriptor_struct_manufacturer::wString[thisMANUFACTURER_NAME_LEN]
```

The documentation for this struct was generated from the following file:

- TCD1304Device_Controller_260318.ino

4.5 usb_string_descriptor_struct_product Struct Reference

Public Attributes

- uint8_t bLength
- uint8_t bDescriptorType
- uint16_t wString [thisPRODUCT_NAME_LEN]

4.5.1 Member Data Documentation

4.5.1.1 bDescriptorType

```
uint8_t usb_string_descriptor_struct_product::bDescriptorType
```

4.5.1.2 bLength

```
uint8_t usb_string_descriptor_struct_product::bLength
```

4.5.1.3 wString

```
uint16_t usb_string_descriptor_struct_product::wString[thisPRODUCT_NAME_LEN]
```

The documentation for this struct was generated from the following file:

- TCD1304Device_Controller_260318.ino

4.6 usb_string_descriptor_struct_serial_number Struct Reference

Public Attributes

- uint8_t bLength
- uint8_t bDescriptorType
- uint16_t wString [thisPRODUCT_SERIAL_NUMBER_LEN]

4.6.1 Member Data Documentation

4.6.1.1 bDescriptorType

```
uint8_t usb_string_descriptor_struct_serial_number::bDescriptorType
```

4.6.1.2 bLength

```
uint8_t usb_string_descriptor_struct_serial_number::bLength
```

4.6.1.3 wString

```
uint16_t usb_string_descriptor_struct_serial_number::wString[thisPRODUCT_SERIAL_NUMBER_LEN]
```

The documentation for this struct was generated from the following file:

- TCD1304Device_Controller_260318.ino

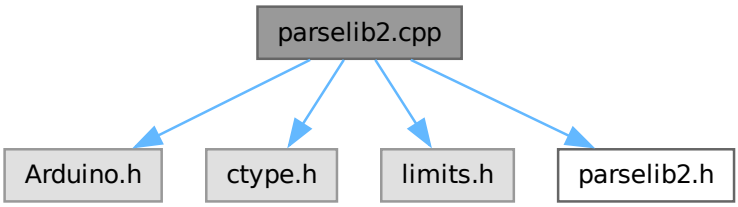
Chapter 5

File Documentation

5.1 parselib2.cpp File Reference

```
#include "Arduino.h"
#include <ctype.h>
#include <limits.h>
#include "parselib2.h"
```

Include dependency graph for parselib2.cpp:



Functions

- int serialPrintf (const char ∗format,...)
- int serialPrintlnf (const char ∗format,...)
- char ∗ eow (char ∗s)
- char ∗ eos (char ∗s)
- unsigned int wordLength (char ∗s)
- char ∗ nextWord (char ∗s)
- char ∗ basenamef (const char ∗cs)
- unsigned int countWords (char ∗s)
- bool strMatch (const char ∗cs, const char ∗key, char ∗∗next=NULL)

- bool strBool (const char ∗cs, bool ∗b, char ∗∗next)
- bool strUint8lim (const char ∗cs, uint8_t ∗u, char ∗∗next, uint8_t ulim)
- bool strUint8 (const char ∗cs, uint8_t ∗u, char ∗∗next)
- bool strUint (const char ∗cs, unsigned int ∗u, char ∗∗next)
- bool strUint16 (const char ∗cs, uint16_t ∗u, char ∗∗next)
- bool strUint32 (const char ∗cs, uint32_t ∗u, char ∗∗next)
- char ∗ scaling (float ∗f, char ∗s)
- bool strFlt (const char ∗cs, float ∗p, char ∗∗next)
- unsigned int strUints (const char ∗cs, unsigned int ∗p, unsigned int nmax, char ∗∗next)
- unsigned int strUint32s (const char ∗cs, uint32_t ∗p, unsigned int nmax, char ∗∗next)
- unsigned int strFlts (const char ∗pc, float ∗p, unsigned int nmax, char ∗∗next)

Variables

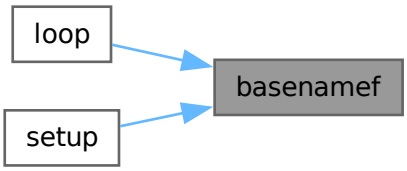
- char printbuffer [256]
- unsigned int nprintbuffer = 0

5.1.1 Function Documentation

5.1.1.1 basenamef()

```
char * basenamef (
            const char * cs)
```

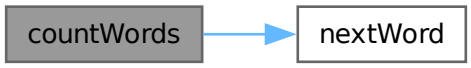
Here is the caller graph for this function:



5.1.1.2 countWords()

```
unsigned int countWords (
            char * s)
```

Here is the call graph for this function:



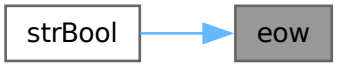
5.1.1.3 eos()

```
char * eos (
            char * s)
```

5.1.1.4 eow()

```
char * eow (
            char * s)
```

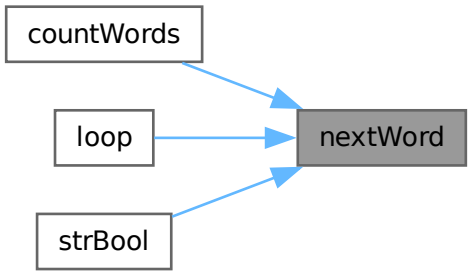
Here is the caller graph for this function:



5.1.1.5 nextWord()

```
char * nextWord (
            char * s)
```

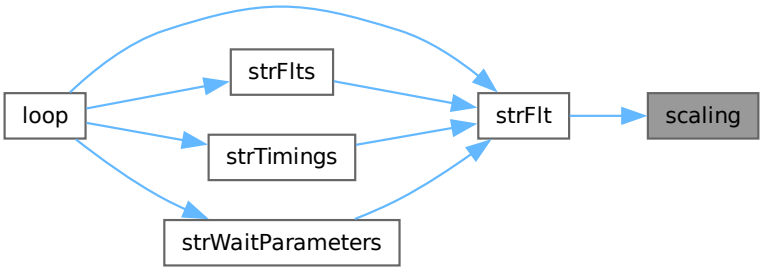
Here is the caller graph for this function:



5.1.1.6 scaling()

```
char * scaling (
            float * f,
            char * s)
```

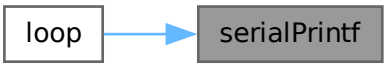
Here is the caller graph for this function:



5.1.1.7 serialPrintf()

```
int serialPrintf (
            const char * format,
             ...)
```

Here is the caller graph for this function:



5.1.1.8 serialPrintlnf()

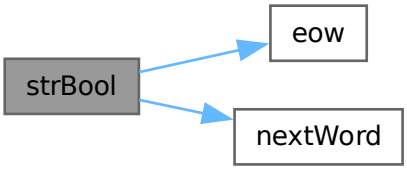
```
int serialPrintlnf (
            const char * format,
             ...)
```

Here is the caller graph for this function:

5.1.1.9 strBool()

```
bool strBool (
            const char * cs,
            bool * b,
            char ** next)
```

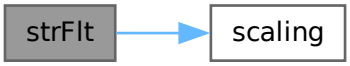
Here is the call graph for this function:



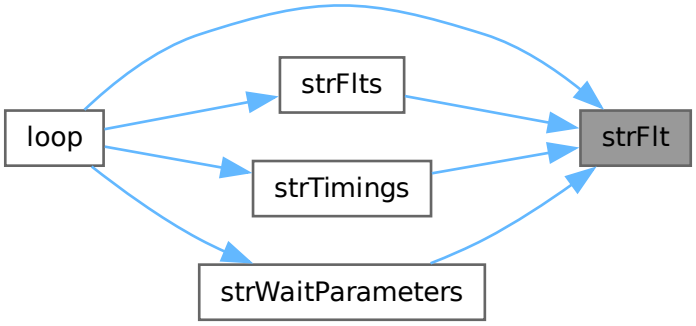
5.1.1.10 strFlt()

```
bool strFlt (
            const char * cs,
            float * p,
            char ** next)
```

Here is the call graph for this function:



Here is the caller graph for this function:

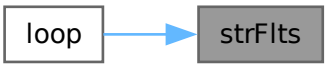


5.1.1.11 strFlts()

```
unsigned int strFlts (
            const char * pc,
            float * p,
            unsigned int nmax,
            char ** next)
```

Here is the call graph for this function:

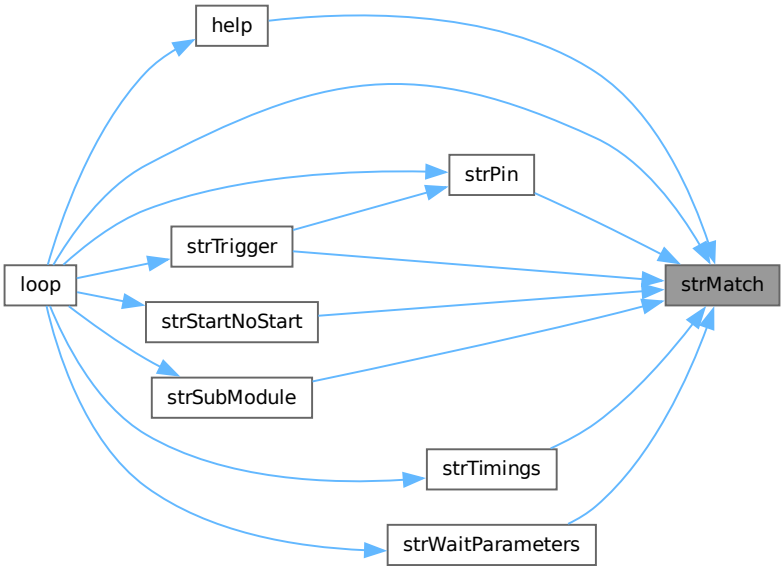
Here is the caller graph for this function:



5.1.1.12 strMatch()

```
bool strMatch (
            const char * cs,
            const char * key,
            char ** next = NULL)
```

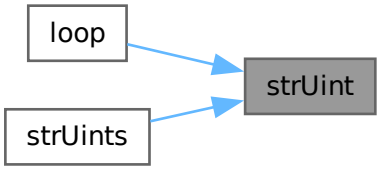
Here is the caller graph for this function:



5.1.1.13 strUint()

```
bool strUint (
            const char * cs,
            unsigned int * u,
            char ** next)
```

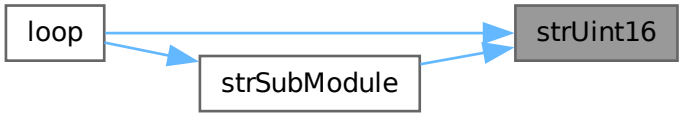
Here is the caller graph for this function:



5.1.1.14 strUint16()

```
bool strUint16 (
            const char * cs,
            uint16_t * u,
            char ** next)
```

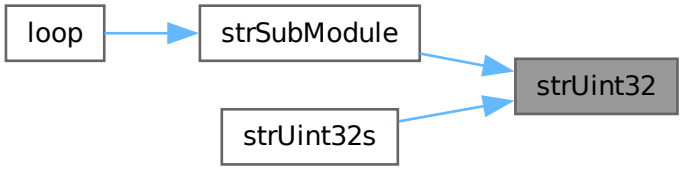
Here is the caller graph for this function:



5.1.1.15 strUint32()

```
bool strUint32 (
            const char * cs,
            uint32_t * u,
            char ** next)
```

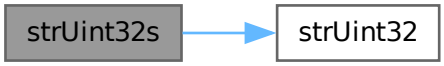
Here is the caller graph for this function:



5.1.1.16 strUint32s()

```
unsigned int strUint32s (
            const char * cs,
            uint32_t * p,
            unsigned int nmax,
            char ** next)
```

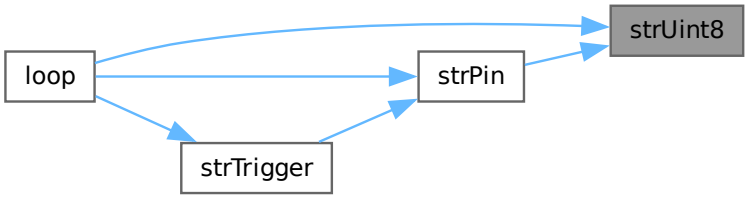
Here is the call graph for this function:



5.1.1.17 strUint8()

```
bool strUint8 (
            const char * cs,
            uint8_t * u,
            char ** next)
```

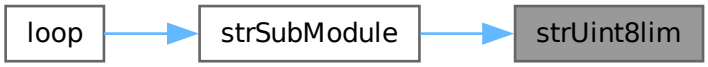
Here is the caller graph for this function:



5.1.1.18 strUint8lim()

```
bool strUint8lim (
            const char * cs,
            uint8_t * u,
            char ** next,
            uint8_t ulim)
```

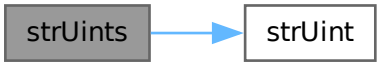
Here is the caller graph for this function:



5.1.1.19 strUints()

```
unsigned int strUints (
            const char * cs,
            unsigned int * p,
            unsigned int nmax,
            char ** next)
```

Here is the call graph for this function:



5.1.1.20 wordLength()

```
unsigned int wordLength (
            char * s)
```

5.1.2 Variable Documentation

5.1.2.1 nprintbuffer

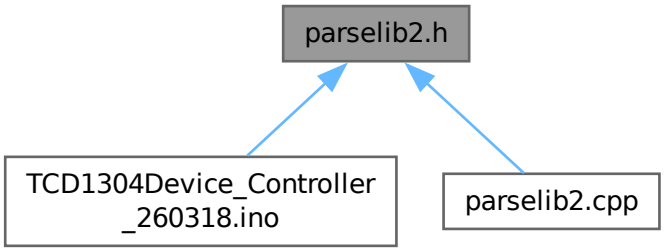
```
unsigned int nprintbuffer = 0
```

5.1.2.2 printbuffer

```
char printbuffer[256]
```

5.2 parselib2.h File Reference

This graph shows which files directly or indirectly include this file:



Macros

- #define PARSELIB_H

Functions

- int serialPrintf (const char ∗fmt,...)
- int serialPrintlnf (const char ∗fmt,...)
- unsigned int wordLength (char ∗s)
- char ∗ nextWord (char ∗s)
- unsigned int countWords (char ∗s)
- char ∗ basenamef (const char ∗cs)
- bool strMatch (const char ∗s, const char ∗key, char ∗∗next)
- bool strBool (const char ∗s, bool ∗b, char ∗∗next=0)
- bool strUint8 (const char ∗s, uint8_t ∗u, char ∗∗next)
- bool strUint8lim (const char ∗s, uint8_t ∗u, char ∗∗next, uint8_t ulim)
- bool strUint (const char ∗s, unsigned int ∗u, char ∗∗next)
- bool strUint16 (const char ∗s, uint16_t ∗u, char ∗∗next)
- bool strUint32 (const char ∗s, uint32_t ∗u, char ∗∗next)
- bool strFlt (const char ∗s, float ∗p, char ∗∗next)
- unsigned int strUints (const char ∗pc, unsigned int ∗p, unsigned int nmax, char ∗∗next)
- unsigned int strUint32s (const char ∗pc, uint32_t ∗p, unsigned int nmax, char ∗∗next)
- unsigned int strFlts (const char ∗pc, float ∗p, unsigned int nmax, char ∗∗next)

5.2.1 Macro Definition Documentation

5.2.1.1 PARSELIB_H

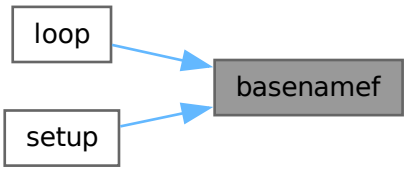
```
#define PARSELIB_H
```

5.2.2 Function Documentation

5.2.2.1 basenamef()

```
char * basenamef (
            const char * cs)
```

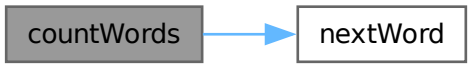
Here is the caller graph for this function:



5.2.2.2 countWords()

```
unsigned int countWords (
            char * s)
```

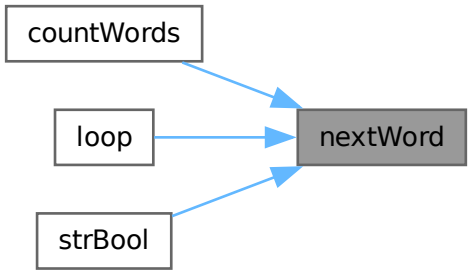
Here is the call graph for this function:



5.2.2.3 nextWord()

```
char * nextWord (
            char * s)
```

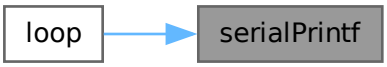
Here is the caller graph for this function:



5.2.2.4 serialPrintf()

```
int serialPrintf (
            const char * fmt,
             ...)
```

Here is the caller graph for this function:



5.2.2.5 serialPrintlnf()

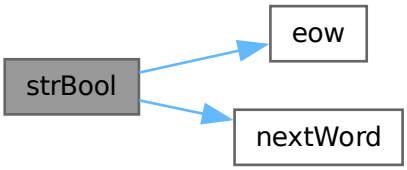
```
int serialPrintlnf (
            const char * fmt,
             ...)
```

Here is the caller graph for this function:

5.2.2.6 strBool()

```
bool strBool (
            const char * s,
            bool * b,
            char ** next = 0)
```

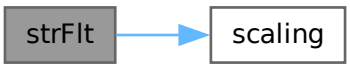
Here is the call graph for this function:



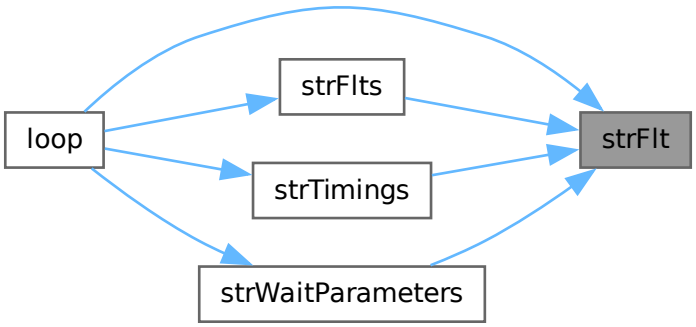
5.2.2.7 strFlt()

```
bool strFlt (
            const char * s,
            float * p,
            char ** next)
```

Here is the call graph for this function:



Here is the caller graph for this function:

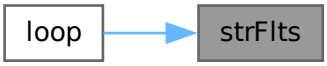


5.2.2.8 strFlts()

```
unsigned int strFlts (
            const char * pc,
            float * p,
            unsigned int nmax,
            char ** next)
```

Here is the call graph for this function:

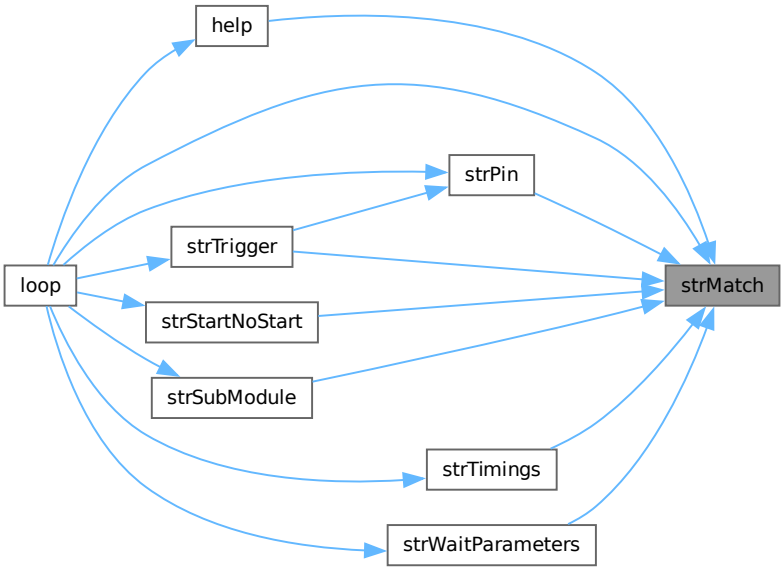
Here is the caller graph for this function:



5.2.2.9 strMatch()

```
bool strMatch (
            const char * s,
            const char * key,
            char ** next)
```

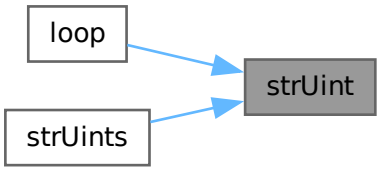
Here is the caller graph for this function:



5.2.2.10 strUint()

```
bool strUint (
            const char * s,
            unsigned int * u,
            char ** next)
```

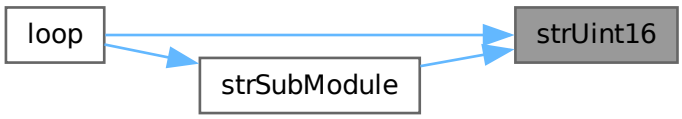
Here is the caller graph for this function:



5.2.2.11 strUint16()

```
bool strUint16 (
            const char * s,
            uint16_t * u,
            char ** next)
```

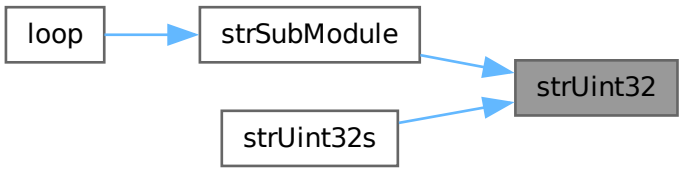
Here is the caller graph for this function:



5.2.2.12 strUint32()

```
bool strUint32 (
            const char * s,
            uint32_t * u,
            char ** next)
```

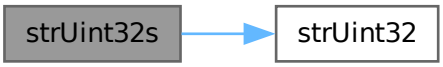
Here is the caller graph for this function:



5.2.2.13 strUint32s()

```
unsigned int strUint32s (
            const char * pc,
            uint32_t * p,
            unsigned int nmax,
            char ** next)
```

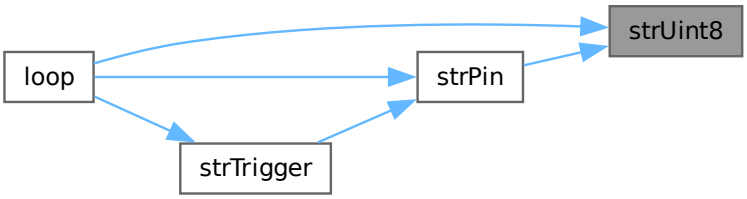
Here is the call graph for this function:



5.2.2.14 strUint8()

```
bool strUint8 (
            const char * s,
            uint8_t * u,
            char ** next)
```

Here is the caller graph for this function:

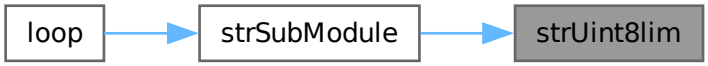


5.2.2.15 strUint8lim()

```
bool strUint8lim (
            const char * s,
            uint8_t * u,
```

```
            char ** next,
            uint8_t ulim)
```

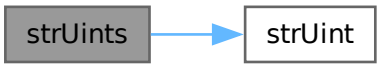
Here is the caller graph for this function:



5.2.2.16 strUints()

```
unsigned int strUints (
            const char * pc,
            unsigned int * p,
            unsigned int nmax,
            char ** next)
```

Here is the call graph for this function:



5.2.2.17 wordLength()

```
unsigned int wordLength (
            char * s)
```

5.3 parselib2.h

Go to the documentation of this file.

```
/* ====================================================================
   Parse srings for words, numbers, etc

   Apart from wordLength(), these return a pointer to the
   next character in the string, or null if they fail

*/

#ifndef PARSELIB_h
#define PARSELIB_H

/*
#ifdef __cplusplus
extern "C" {
#endif
*/

int serialPrintf( const char *fmt, ... );
int serialPrintlnf( const char *fmt, ... );

unsigned int wordLength( char *s );

char *nextWord( char *s );

unsigned int countWords( char *s );

char *basenamef(const char *cs);

bool strMatch(const char *s, const char *key, char **next);

bool strBool(const char *s, bool *b, char **next=0);

bool strUint8(const char *s, uint8_t *u, char **next );

bool strUint8lim(const char *s, uint8_t *u, char **next, uint8_t ulim);

bool strUint(const char *s, unsigned int *u, char **next);

bool strUint16(const char *s, uint16_t *u, char **next);

bool strUint32(const char *s, uint32_t *u, char **next);

bool strFlt(const char *s, float *p, char **next);

unsigned int strUints(const char *pc, unsigned int *p, unsigned int nmax, char **next);

unsigned int strUint32s(const char *pc, uint32_t *p, unsigned int nmax, char **next);

unsigned int strFlts(const char *pc, float *p, unsigned int nmax, char **next);

/*
  #ifdef __cplusplus
  }
  #endif
*/

#endif
```

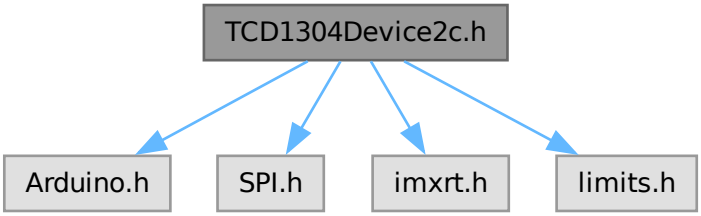
5.4 README.md File Reference

5.5 TCD1304Device2c.h File Reference

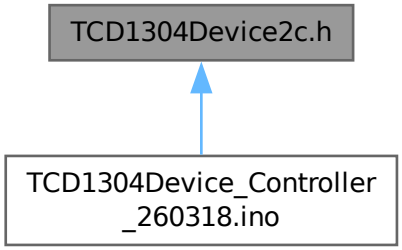
```
#include "Arduino.h"
#include <SPI.h>
#include "imxrt.h"
```

```
#include <limits.h>
```

Include dependency graph for TCD1304Device2c.h:



This graph shows which files directly or indirectly include this file:



Classes

- class TCD1304Device
- struct TCD1304Device::SubModule
- struct TCD1304Device::Frame_Header_struct

Macros

- #define TCD1304DEVICE_LIBARY_VERSION 0X002
- #define PINPULLS
- #define SH_CLEARING_DEFAULT 0
- #define TCD1304_MAXCLKHZ 4.E6
- #define TCD1304_MINCLKHZ 0.8E6
- #define COUNTSCEIL(a)

- #define COUNTSFLOOR(a)
- #define SECONDS(a)
- #define ROUNDUP(a, b)
- #define ROUNDTO(a, b)
- #define ROUNDTOMOD(a, b, c)
- #define CYCCNT2SECS(a)
- #define SECS2CYCCNT(a)
- #define TDIFFSECS(a, b)
- #define PWM_CTRL2_CLOCK_MASTER FLEXPWM_SMCTRL2_FRCEN
- #define PWM_CTRL2_CLOCK_SLAVE (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | (FLEXPWM_SMCTRL2_↩ FRCEN | FLEXPWM_SMCTRL2_FORCE_SEL(0X3)))
- #define PWM_CTRL2_CLOCK_SYNC (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | FLEXPWM_SMCTRL2_↩ FRCEN)
- #define CMPF_MASKA_ON (1<<2)
- #define CMPF_MASKA_OFF (1<<3)
- #define CMPF_MASKA_ON_OFF (CMPF_MASKA_ON|CMPF_MASKA_OFF)
- #define CMPF_MASKB_ON (1<<4)
- #define CMPF_MASKB_OFF (1<<5)
- #define CMPF_MASKB_ON_OFF (CMPF_MASKB_ON|CMPF_MASKB_OFF)
- #define CLK_DEFAULT 64
- #define CLK_SUBMODULE 0
- #define CLK_MASK 0x1
- #define CLK_CHANNEL 1
- #define CLK_PIN 4
- #define CLK_MUXVAL 1
- #define CLK_IRQ IRQ_FLEXPWM2_0
- #define ICG_SUBMODULE 1
- #define ICG_MASK 0x2
- #define ICG_CHANNEL 1
- #define ICG_PIN 5
- #define ICG_MUXVAL 1
- #define ICG_IRQ IRQ_FLEXPWM2_1
- #define SH_SUBMODULE 2
- #define SH_MASK 0x4
- #define SH_CHANNEL 1
- #define SH_PIN 6
- #define SH_MUXVAL 2
- #define SH_IRQ IRQ_FLEXPWM2_2
- #define CNVST_SUBMODULE 3
- #define CNVST_MASK 0x8
- #define CNVST_CHANNEL 1
- #define CNVST_IRQ IRQ_FLEXPWM2_3
- #define CNVST_PIN 10
- #define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)
- #define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)
- #define CLK_MONITOR_PIN 3
- #define SYNC_PIN 0
- #define BUSY_PIN 1
- #define TRIGGER_PIN 2
- #define SYNC_PIN_DEFAULT LOW
- #define BUSY_PIN_DEFAULT HIGH

- #define NREADOUT 3694
- #define DATASTART 16
- #define DATASTOP 3680
- #define NPIXELS NREADOUT
- #define SENDSTART 0
- #define SENDLENGTHWORDS NREADOUT
- #define SENDLENGTHBYTES (2∗NREADOUT)
- #define SENDLENGTHBYTES32 (4∗NREADOUT)
- #define DARKSTART (DATASTART-SENDSTART)
- #define NDARK 13
- #define NBITS 16
- #define VFS (4.096/5)
- #define VPERBIT (VFS/(1<<NBITS))
- #define SHUTTERMIN 5
- #define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)
- #define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)
- #define USBSPEED 480.0E6
- #define USBTRANSFERSECS ((float)NREADOUT∗(16/USBSPEED))
- #define COUNTER_MAX_SECS ((float)(65535 ∗ 128) / F_BUS_ACTUAL)
- #define DEBUGPRINTF(...)
- #define CNVST_SECS 630.E-9
- #define CNVST_START_OVERHEAD 300.0E-9
- #define CNVST_CLKS_FROM_ICG 3.75
- #define SAR_OFFSET 100.E-9
- #define DEBUG
- #define PIT_CHANNEL 0
- #define PIT_FREQ 24000000.0f
- #define PITSECS(a)
- #define PITCOUNTS(a)

Enumerations

- enum TCD1304_Mode_t {
  NOTCONFIGURED , PULSE , PULSE_LOOP , PIT ,
  FLEXPWM }

5.5.1 Macro Definition Documentation

5.5.1.1 BUSY_PIN

```
#define BUSY_PIN 1
```

5.5.1.2 BUSY_PIN_DEFAULT

```
#define BUSY_PIN_DEFAULT HIGH
```

5.5.1.3 CLEARCNVST [1/2]

```
#define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)
```

5.5.1.4 CLEARCNVST [2/2]

```
#define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)
```

5.5.1.5 CLK_CHANNEL

```
#define CLK_CHANNEL 1
```

5.5.1.6 CLK_DEFAULT

```
#define CLK_DEFAULT 64
```

5.5.1.7 CLK_IRQ

```
#define CLK_IRQ IRQ_FLEXPWM2_0
```

5.5.1.8 CLK_MASK

```
#define CLK_MASK 0x1
```

5.5.1.9 CLK_MONITOR_PIN

```
#define CLK_MONITOR_PIN 3
```

5.5.1.10 CLK_MUXVAL

```
#define CLK_MUXVAL 1
```

5.5.1.11 CLK_PIN

```
#define CLK_PIN 4
```

5.5.1.12 CLK_SUBMODULE

```
#define CLK_SUBMODULE 0
```

5.5.1.13 CMPF_MASKA_OFF

```
#define CMPF_MASKA_OFF (1<<3)
```

5.5.1.14 CMPF_MASKA_ON

```
#define CMPF_MASKA_ON (1<<2)
```

5.5.1.15 CMPF_MASKA_ON_OFF

```
#define CMPF_MASKA_ON_OFF (CMPF_MASKA_ON|CMPF_MASKA_OFF)
```

5.5.1.16 CMPF_MASKB_OFF

```
#define CMPF_MASKB_OFF (1<<5)
```

5.5.1.17 CMPF_MASKB_ON

```
#define CMPF_MASKB_ON (1<<4)
```

5.5.1.18 CMPF_MASKB_ON_OFF

```
#define CMPF_MASKB_ON_OFF (CMPF_MASKB_ON|CMPF_MASKB_OFF)
```

5.5.1.19 CNVST_CHANNEL

```
#define CNVST_CHANNEL 1
```

5.5.1.20 CNVST_CLKS_FROM_ICG

```
#define CNVST_CLKS_FROM_ICG 3.75
```

5.5.1.21 CNVST_IRQ

```
#define CNVST_IRQ IRQ_FLEXPWM2_3
```

5.5.1.22 CNVST_MASK

```
#define CNVST_MASK 0x8
```

5.5.1.23 CNVST_PIN

```
#define CNVST_PIN 10
```

5.5.1.24 CNVST_SECS

```
#define CNVST_SECS 630.E-9
```

5.5.1.25 CNVST_START_OVERHEAD

```
#define CNVST_START_OVERHEAD 300.0E-9
```

5.5.1.26 CNVST_SUBMODULE

```
#define CNVST_SUBMODULE 3
```

5.5.1.27 COUNTER_MAX_SECS

```
#define COUNTER_MAX_SECS ((float)(65535 * 128) / F_BUS_ACTUAL)
```

5.5.1.28 COUNTSCEIL

```
#define COUNTSCEIL(
            a)
```

Value:

```
(a<1.?(int)ceil((a)*F_BUS_ACTUAL):(int)a)
```

5.5.1.29 COUNTSFLOOR

```
#define COUNTSFLOOR(
            a)
```

Value:

```
(a<1.?(int)floor((a)*F_BUS_ACTUAL):(int)a)
```

5.5.1.30 CYCCNT2SECS

```
#define CYCCNT2SECS(
            a)
```

Value:

```
(((double)(a))/F_CPU)
```

5.5.1.31 DARKSTART

```
#define DARKSTART (DATASTART-SENDSTART)
```

5.5.1.32 DATASTART

```
#define DATASTART 16
```

5.5.1.33 DATASTOP

```
#define DATASTOP 3680
```

5.5.1.34 DEBUG

```
#define DEBUG
```

5.5.1.35 DEBUGPRINTF

```
#define DEBUGPRINTF(
              ...)
```

5.5.1.36 ICG_CHANNEL

```
#define ICG_CHANNEL 1
```

5.5.1.37 ICG_IRQ

```
#define ICG_IRQ IRQ_FLEXPWM2_1
```

5.5.1.38 ICG_MASK

```
#define ICG_MASK 0x2
```

5.5.1.39 ICG_MUXVAL

```
#define ICG_MUXVAL 1
```

5.5.1.40 ICG_PIN

```
#define ICG_PIN 5
```

5.5.1.41 ICG_SUBMODULE

```
#define ICG_SUBMODULE 1
```

5.5.1.42 NBITS

```
#define NBITS 16
```

5.5.1.43 NDARK

```
#define NDARK 13
```

5.5.1.44 NPIXELS

```
#define NPIXELS NREADOUT
```

5.5.1.45 NREADOUT

```
#define NREADOUT 3694
```

5.5.1.46 PINPULLS

```
#define PINPULLS
```

5.5.1.47 PIT_CHANNEL

```
#define PIT_CHANNEL 0
```

5.5.1.48 PIT_FREQ

```
#define PIT_FREQ 24000000.0f
```

5.5.1.49 PITCOUNTS

```
#define PITCOUNTS(
            a)
```

Value:

```
(uint32_t)((a)*PIT_FREQ))
```

5.5.1.50 PITSECS

```
#define PITSECS(
            a)
```

Value:

```
((a)/PIT_FREQ)
```

5.5.1.51 PWM_CTRL2_CLOCK_MASTER

```
#define PWM_CTRL2_CLOCK_MASTER FLEXPWM_SMCTRL2_FRCEN
```

5.5.1.52 PWM_CTRL2_CLOCK_SLAVE

```
#define PWM_CTRL2_CLOCK_SLAVE (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | (FLEXPWM_SMCTRL2_FRCEN | FLEXPWM_↩
SMCTRL2_FORCE_SEL(0X3)))
```

5.5.1.53 PWM_CTRL2_CLOCK_SYNC

```
#define PWM_CTRL2_CLOCK_SYNC (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | FLEXPWM_SMCTRL2_FRCEN)
```

5.5.1.54 ROUNDTO

```
#define ROUNDTO(
            a,
            b)
```

Value:

```
((a/b)*b)
```

5.5.1.55 ROUNDTOMOD

```
#define ROUNDTOMOD(
            a,
            b,
            c)
```

Value:

```
(((a/b)*b)%c)
```

5.5.1.56 ROUNDUP

```
#define ROUNDUP(
            a,
            b)
```

Value:

```
(ceil((float)a/b)*b)
```

5.5.1.57 SAR_OFFSET

```
#define SAR_OFFSET 100.E-9
```

5.5.1.58 SECONDS

```
#define SECONDS(
            a)
```

Value:

```
((float)(a)/F_BUS_ACTUAL)
```

5.5.1.59 SECS2CYCCNT

```
#define SECS2CYCCNT(
            a)
```

Value:

```
((uint64_t)(((double)(a))*F_CPU))
```

5.5.1.60 SENDLENGTHBYTES

```
#define SENDLENGTHBYTES (2*NREADOUT)
```

5.5.1.61 SENDLENGTHBYTES32

```
#define SENDLENGTHBYTES32 (4*NREADOUT)
```

5.5.1.62 SENDLENGTHWORDS

```
#define SENDLENGTHWORDS NREADOUT
```

5.5.1.63 SENDSTART

```
#define SENDSTART 0
```

5.5.1.64 SETCNVST [1/2]

```
#define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)
```

5.5.1.65 SETCNVST [2/2]

```
#define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)
```

5.5.1.66 SH_CHANNEL

```
#define SH_CHANNEL 1
```

5.5.1.67 SH_CLEARING_DEFAULT

```
#define SH_CLEARING_DEFAULT 0
```

5.5.1.68 SH_IRQ

```
#define SH_IRQ IRQ_FLEXPWM2_2
```

5.5.1.69 SH_MASK

```
#define SH_MASK 0x4
```

5.5.1.70 SH_MUXVAL

```
#define SH_MUXVAL 2
```

5.5.1.71 SH_PIN

```
#define SH_PIN 6
```

5.5.1.72 SH_SUBMODULE

```
#define SH_SUBMODULE 2
```

5.5.1.73 SHUTTERMIN

```
#define SHUTTERMIN 5
```

5.5.1.74 SYNC_PIN

```
#define SYNC_PIN 0
```

5.5.1.75 SYNC_PIN_DEFAULT

```
#define SYNC_PIN_DEFAULT LOW
```

5.5.1.76 TCD1304_MAXCLKHZ

```
#define TCD1304_MAXCLKHZ 4.E6
```

5.5.1.77 TCD1304_MINCLKHZ

```
#define TCD1304_MINCLKHZ 0.8E6
```

5.5.1.78 TCD1304DEVICE_LIBARY_VERSION

```
#define TCD1304DEVICE_LIBARY_VERSION 0X002
```

5.5.1.79 TDIFFSECS

```
#define TDIFFSECS(
            a,
            b)
```

Value:

```
(((double)(a-b))/F_CPU)
```

5.5.1.80 TRIGGER_PIN

```
#define TRIGGER_PIN 2
```

5.5.1.81 USBSPEED

```
#define USBSPEED 480.0E6
```

5.5.1.82 USBTRANSFERSECS

```
#define USBTRANSFERSECS ((float)NREADOUT*(16/USBSPEED))
```

5.5.1.83 VFS

```
#define VFS (4.096/5)
```

5.5.1.84 VPERBIT

```
#define VPERBIT (VFS/(1<<NBITS))
```

5.5.2 Enumeration Type Documentation

5.5.2.1 TCD1304_Mode_t

```
enum TCD1304_Mode_t
```

Enumerator

NOTCONFIGURED	
PULSE	
PULSE_LOOP	
PIT	
FLEXPWM	

5.6 TCD1304Device2c.h

Go to the documentation of this file.

```
/* tcd1304 flexpwm library

   Author Mitchell C Nelson, PhD
   Copyright 2025

   Free for non-commercial use.

   No warranty and no representation of suitability for any purpose whatsoever

*/

#ifndef TCD1304DEVICE_H
#define TCD1304DEVICE_H

#define TCD1304DEVICE_LIBARY_VERSION 0X002

#include "Arduino.h"
#include <SPI.h>

#include "imxrt.h"

```

```
#include <limits.h>

// ========================================
// All-In-One Board or other that uses analog input
// #define ALLINONEBOARD

// CONFIGURE PULLUPS/PULLDOWNS
#define PINPULLS

// For PIT mode, the CNVST idle is our virtual buffer for vref - leave it on!
// #define CNVST_IDLE_DISABLE

// For PIT mode, clearing pulses default
#define SH_CLEARING_DEFAULT 0

// Do voltage offset calculation
//#define TCD13404_OFFSET_ENABLED

// debug statements in setup
//#define DEBUG

// ========================================

#ifdef  ALLINONEBOARD
#include <ADC.h>
#include <ADC_util.h>
extern ADC *adc;
#define ANALOGPIN A0
#define TCD1304_MAXCLKHZ 2.35E6
#else
#define TCD1304_MAXCLKHZ 4.E6
#endif
#define TCD1304_MINCLKHZ 0.8E6

// ========================================
// used in seconds to/from counter value
#define COUNTSCEIL(a) (a<1.?(int)ceil((a)*F_BUS_ACTUAL):(int)a)
#define COUNTSFLOOR(a) (a<1.?(int)floor((a)*F_BUS_ACTUAL):(int)a)
#define SECONDS(a) ((float)(a)/F_BUS_ACTUAL)

#define ROUNDUP(a,b) (ceil((float)a/b)*b)
#define ROUNDTO(a,b) ((a/b)*b)
#define ROUNDTOMOD(a,b,c) (((a/b)*b)%c)

// ----------------------------------------
// cpu cycle counter values to seconds
// usage
#ifndef CYCCNT2SECS
#define CYCCNT2SECS(a) (((double)(a))/F_CPU)
#endif

#ifndef SECS2CYCCNT
#define SECS2CYCCNT(a) ((uint64_t)(((double)(a))*F_CPU))
#endif

#define TDIFFSECS(a,b) (((double)(a-b))/F_CPU)

// Clock and force configurations
#define PWM_CTRL2_CLOCK_MASTER FLEXPWM_SMCTRL2_FRCEN
#define PWM_CTRL2_CLOCK_SLAVE (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | (FLEXPWM_SMCTRL2_FRCEN |
FLEXPWM_SMCTRL2_FORCE_SEL(0X3)))
#define PWM_CTRL2_CLOCK_SYNC (FLEXPWM_SMCTRL2_CLK_SEL(0x2) | FLEXPWM_SMCTRL2_FRCEN)

// Flexpwm compare interrupts
#define CMPF_MASKA_ON (1«2)
#define CMPF_MASKA_OFF (1«3)
#define CMPF_MASKA_ON_OFF (CMPF_MASKA_ON|CMPF_MASKA_OFF)

#define CMPF_MASKB_ON (1«4)
#define CMPF_MASKB_OFF (1«5)
#define CMPF_MASKB_ON_OFF (CMPF_MASKB_ON|CMPF_MASKB_OFF)

//#define CLK_DEFAULT 48
#define CLK_DEFAULT 64
//#define CLK_DEFAULT 96
//#define CLK_DEFAULT 128

#define CLK_SUBMODULE 0
#define CLK_MASK 0x1
#define CLK_CHANNEL 1
#define CLK_PIN 4
```

```
#define CLK_MUXVAL 1
#define CLK_IRQ IRQ_FLEXPWM2_0

#define ICG_SUBMODULE 1
#define ICG_MASK 0x2
#define ICG_CHANNEL 1
#define ICG_PIN 5
#define ICG_MUXVAL 1
#define ICG_IRQ IRQ_FLEXPWM2_1

#define SH_SUBMODULE 2
#define SH_MASK 0x4
#define SH_CHANNEL 1
#define SH_PIN 6
#define SH_MUXVAL 2
#define SH_IRQ IRQ_FLEXPWM2_2

#define CNVST_SUBMODULE 3
#define CNVST_MASK 0x8
#define CNVST_CHANNEL 1
#define CNVST_IRQ IRQ_FLEXPWM2_3

/* Note that we do not configure the flexPWM to use this pin (CNVST_PIN),
   It is not on the mux list for this module,
   We operate this pin using digitalwrite, from the isr.
*/
#define CNVST_PIN 10
#define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)

#define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)

#define CLK_MONITOR_PIN 3

#define SYNC_PIN 0
#define BUSY_PIN 1
#define TRIGGER_PIN 2
//#define SPARE_PIN 3

#define SYNC_PIN_DEFAULT LOW
#define BUSY_PIN_DEFAULT HIGH

// Fast nofrills BUSY pin set, clear, flip state
/*
#define SETBUSYPIN (CORE_PIN1_PORTSET = CORE_PIN1_BITMASK)
#define CLEARBUSYPIN (CORE_PIN1_PORTCLEAR = CORE_PIN1_BITMASK)
#define TOGGLEBUSYPIN (CORE_PIN1_PORTTOGGLE = CORE_PIN1_BITMASK)
*/

// Fast nofrills SYNC pin set/clear
/*
#define SETSYNCPIN (CORE_PIN0_PORTSET = CORE_PIN0_BITMASK)
#define CLEARSYNCPIN (CORE_PIN0_PORTCLEAR = CORE_PIN0_BITMASK)
#define TOGGLESYNCPIN (CORE_PIN0_PORTTOGGLE = CORE_PIN0_BITMASK)
*/
// Sensor data readout
#define NREADOUT 3694
#define DATASTART 16
#define DATASTOP 3680

// Perhaps redundant but more mnemonic
#define NPIXELS NREADOUT

// Size for sending the data
#define SENDSTART 0
#define SENDLENGTHWORDS NREADOUT
#define SENDLENGTHBYTES (2*NREADOUT)
#define SENDLENGTHBYTES32 (4*NREADOUT)

// And first part of that is dark
#define DARKSTART (DATASTART-SENDSTART)
#define NDARK 13

#ifdef  ALLINONEBOARD
#define NBITS 12
#define VFS (3.3/5)
#else
#define NBITS 16
#define VFS (4.096/5)
#endif

#define VPERBIT (VFS/(1«NBITS))
```

```

#define SHUTTERMIN 5

#define SETCNVST (CORE_PIN10_PORTSET = CORE_PIN10_BITMASK)
#define CLEARCNVST (CORE_PIN10_PORTCLEAR = CORE_PIN10_BITMASK)

#define USBSPEED 480.0E6
#define USBTRANSFERSECS ((float)NREADOUT*(16/USBSPEED))

#define COUNTER_MAX_SECS ((float)(65535 * 128) / F_BUS_ACTUAL)

#ifdef DEBUG
int debugprintf(const char* format, ...)
{
  char buffer[128] = {0};
  int n;
  va_list argptr;
  va_start(argptr, format);
  n = vsprintf (buffer, format, argptr );
  va_end(argptr);
  if (n > 0) {
    Serial.println(buffer);
  }
  else if (n<0) {
    Serial.print("Error: debugprintf vsprintf");
    Serial.println(format);
  }
  return n;
}
#define DEBUGPRINTF(...) debuprintf( __VA_ARGS__ )
#else
#define DEBUGPRINTF(...)
#endif


// ---------------------------------------------------------------------------
// NOTE this is not designed to be thread safe.
// (Consider making it a singleton)

/*******************************************************************
 * TCD1304 Device implemented in FlexPWM2, pins 3,4,5,6 and 10
 */
typedef enum {
  NOTCONFIGURED,
  PULSE,
  PULSE_LOOP,
  PIT,
  FLEXPWM
} TCD1304_Mode_t;

class TCD1304Device
{
public:

  // struct to simplify our namespace for the flexpwm submodules
  struct SubModule {
    const char *name;
    const uint8_t submod;
    const uint8_t mask;
    const uint8_t pinA;
    const uint8_t muxvalA;
    const uint8_t pinB;
    const uint8_t muxvalB;
    IRQ_NUMBER_t irq;
    IMXRT_FLEXPWM_t *flexpwm;
    //uint16_t period_counts = 0;
    uint32_t period_counts = 0;
    uint16_t onA_counts = 0;
    uint16_t offA_counts = 0;
    uint16_t onB_counts = 0;
    uint16_t offB_counts = 0;
    uint16_t ctrl2_mask = 0;
    uint16_t intena_mask = 0;
    uint16_t divider = 1;
    uint8_t prescale = 0;
    uint8_t filler = 0;
    float period_secs = 0;
    float offset = 0; // offset in nanoseconds for start from time = 0, for cnvst

    bool invertA = false;
    bool invertB = false;
```

```

    bool newvals = false;

    void (*isr)() = nullptr;

    uint8_t interrupt_priority = 128;

    SubModule(const char *name_,
              uint8_t submod_, uint8_t mask_,
              uint8_t pinA_, uint8_t muxvalA_,
              uint8_t pinB_, uint8_t muxvalB_,
              IRQ_NUMBER_t irq_,
              IMXRT_FLEXPWM_t *flexpwm_) :
      name(name_),
      submod(submod_),mask(mask_),
      pinA(pinA_), muxvalA(muxvalA_),
      pinB(pinB_), muxvalB(muxvalB_),
      irq(irq_), flexpwm(flexpwm_) {};
  };

  inline static uint64_t sh_cyccnt64_start = 0;
  inline static uint64_t sh_cyccnt64_now = 0;
  inline static uint64_t sh_cyccnt64_prev = 0;
  inline static uint64_t sh_cyccnt64_exposure = 0;

  inline static uint64_t pit_cyccnt64_start = 0;
  inline static uint64_t pit_cyccnt64_now = 0;
  inline static uint64_t pit_cyccnt64_prev = 0;

  // For the trigger timer
  inline static uint64_t trigger_cyccnt64_start = 0;
  inline static uint64_t trigger_cyccnt64_now = 0;
  inline static uint64_t trigger_cyccnt64_prev = 0;

  // Refrenced by timer and trigger setups and by the header routine
  inline static TCD1304_Mode_t mode = NOTCONFIGURED;
  inline static bool trigger_mode = false;
  inline static bool trigger_attached = false;

  // bookkeeping counters
  inline static unsigned int sh_counter    = 0;
  inline static unsigned int icg_counter   = 0;
  inline static unsigned int cnvst_counter = 0;
  inline static unsigned int sh_counts_per_icg = 0;

  // bookkeeping for the charge clearing pulses on SH
  inline static unsigned int sh_clearing_counts = SH_CLEARING_DEFAULT;
  inline static unsigned int sh_clearing_counter = 0;
  inline static unsigned int sh_short_period_counts = 0;

  // read counters and callback
  inline static unsigned int read_counter = 0;
  inline static unsigned int read_counts  = 0;
  inline static uint16_t *read_buffer  = 0;
  inline static uint16_t *read_pointer = 0;
  inline static void (*read_callback)() = 0;

  // frame counters and frameset callback
  inline static unsigned int frame_counter = 0;
  inline static unsigned int frame_counts = 0;
  inline static void (*frames_completed_callback)() = 0;

  // framesets counters and framesets completed callback
  inline static unsigned int frameset_counter = 0;
  inline static unsigned int frameset_counts = 0;
  inline static void (*framesets_completed_callback)() = 0;

  // old code?
  inline static float pulse_interframe_min_secs = 0.;

  // interrupt (trigger) support
  inline static unsigned int trigger_counter = 0;
  inline static unsigned int trigger_counts  = 1;
  inline static void (*trigger_callback)() = 0;

  inline static uint8_t trigger_pin = TRIGGER_PIN;
  inline static uint8_t trigger_edge_mode = RISING;
  inline static uint8_t trigger_pin_mode = INPUT;

  // State of the flexpwm interface
  inline static bool busytoggled = false;
```

```
  // Sync pin management
  inline static uint8_t sync_pin = SYNC_PIN;
  inline static bool synctoggled = false;
  inline static bool sync_enabled = true;

  // And now.... the submodules
  inline static IMXRT_FLEXPWM_t * const flexpwm = &IMXRT_FLEXPWM2;
  inline static SubModule clk   = {"clk",CLK_SUBMODULE, CLK_MASK, CLK_PIN,CLK_MUXVAL, 0xFF,0, CLK_IRQ, &IMXRT_FLEXPWM2};
  inline static SubModule sh    = {"sh", SH_SUBMODULE, SH_MASK,  SH_PIN,SH_MUXVAL, 0xFF,0, SH_IRQ, &IMXRT_FLEXPWM2};
  inline static SubModule icg   = {"icg", ICG_SUBMODULE, ICG_MASK, ICG_PIN,ICG_MUXVAL, 0xFF,0, ICG_IRQ, &IMXRT_FLEXPWM2};
  inline static SubModule cnvst = {"cnvst", CNVST_SUBMODULE, CNVST_MASK, 0xFF,0, 0xFF,0, CNVST_IRQ, &IMXRT_FLEXPWM2}; // no pin

  // error bookkeeping
  inline static bool error_flag = false;
  inline static bool oops_flag = false;

  // completion flags
  inline static bool frames_complete = false;
  inline static bool framesets_complete = false;

  char oopsbuffer[128] = { 0 };

  //---------------------------------------------
  typedef struct Frame_Header_struct {
    uint16_t *buffer;
    unsigned int nbuffer;

    // from the dummy outputs, first 16 elements
    unsigned int rawoffset;
    float offset;
    bool offsetdone;

    float frame_elapsed_secs;
    float frame_exposure_secs;

    float pit_elapsed_secs;
    float pit_difference_secs;

    float trigger_elapsed_secs;
    float trigger_difference_secs;

    unsigned int frame_counter;
    unsigned int frameset_counter;
    unsigned int trigger_counter;

    TCD1304_Mode_t mode;
    bool trigger_mode;

    bool error_flag;
    bool oops_flag;

    bool frames_completed ;
    bool framesets_completed ;

    bool ready_for_send;
  } Frame_Header;


  //---------------------------------------------
  TCD1304Device(unsigned int period=CLK_DEFAULT)
  {
    stop_all();

    pinMode(trigger_pin, trigger_pin_mode);

    pinMode(BUSY_PIN, OUTPUT);
    digitalWrite(BUSY_PIN, BUSY_PIN_DEFAULT);

    pinMode(SYNC_PIN, OUTPUT);
    digitalWrite(SYNC_PIN, SYNC_PIN_DEFAULT);

    pinMode(CNVST_PIN, OUTPUT);
    digitalWrite(CNVST_PIN, LOW);

#ifdef PINPULLS
    pinMode(CLK_PIN, INPUT_PULLDOWN);
```

```
    pinMode(SH_PIN, INPUT_PULLDOWN);
    pinMode(ICG_PIN, INPUT_PULLUP);
#endif

    set_pulse_parameters(0.4E-6);
  }

  static void stop_all()
  {
    flexpwm->MCTRL = 0;
    flexpwm_running = false;

    stop_pit();

    flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(0xF);

    disable_irqs();

    clear_sync_busy_pins();

    clear_mode();  // if the irqs are detached there is not a valid mode anymore

  }

  static void disable_irqs()
  {
    // Disable all of the flexpwm interrupts
    NVIC_DISABLE_IRQ(CLK_IRQ);
    NVIC_DISABLE_IRQ(ICG_IRQ);
    NVIC_DISABLE_IRQ(SH_IRQ);
    NVIC_DISABLE_IRQ(CNVST_IRQ);
  }

  static void clear_error_flags()
  {
    oops_flag = false;
    error_flag = false;
  }

  static void clear_mode()
  {
    mode = NOTCONFIGURED;
    trigger_mode = false;
  }

  static void clear_sync_busy_pins()
  {
    if (busytoggled) {
      busytoggled = false;
      digitalToggleFast(BUSY_PIN);
    }

    // reset sync pin
    if (synctoggled) {
      digitalToggleFast(SYNC_PIN);
      synctoggled = false;
    }
  }

  static void toggle_busypin()
  {
    busytoggled = !busytoggled;
    digitalToggleFast(BUSY_PIN);
  }

  static void toggle_syncpin()
  {
    synctoggled = !synctoggled;
    digitalToggleFast(SYNC_PIN);
  }

  static void clear_busypin()
  {
    if (busytoggled) {
      busytoggled = false;
      digitalToggleFast(BUSY_PIN);
    }
  }

  static void clear_syncpin()
  {
```

```
    if (synctoggled) {
      synctoggled = false;
      digitalToggleFast(SYNC_PIN);
    }
  }

  /* ==========================================================================
     For ring buffering, we'll use this in the read callback to point to the next buffer
  */
  static void update_read_buffer(uint16_t *buffer)
  {
    read_buffer   = buffer;
    //read_counts   = nbuffer;

    read_pointer  = read_buffer;
    read_counter  = read_counts;  // will be set to 0 in the icg isr
  }

  static void fill_frame_header(Frame_Header *p)
  {
#ifdef TCD13404_OFFSET_ENABLED
    unsigned int utmp = 0;
    int n;
#endif

    p->buffer = read_buffer;
    p->nbuffer = read_counts;

    p->rawoffset = adc_raw_offset;
    p->offset = adc_offset;
    p->offsetdone = adc_offset_done;

    p->frame_counter = frame_counter;
    p->frameset_counter = frameset_counter;

    p->trigger_counter = trigger_counter;

    p->frames_completed = frames_complete;

    p->framesets_completed = framesets_complete;

    p->frame_elapsed_secs = sh_elapsed_secs();
    p->frame_exposure_secs = sh_exposure_secs();

    p->pit_elapsed_secs = pit_elapsed_secs();
    p->pit_difference_secs = pit_difference_secs();

    p->trigger_elapsed_secs = trigger_elapsed_secs();
    p->trigger_difference_secs = trigger_difference_secs();

    p->mode = mode;
    p->trigger_mode = trigger_mode;

    p->error_flag = error_flag;
    p->oops_flag = oops_flag;

    p->ready_for_send = true;
  }

  /* ==========================================================================
     High level API
   */

  // Pulse duration
#define CNVST_SECS 630.E-9

  // Overhead to start the pulse (isr + digitalwrite)
  #define CNVST_START_OVERHEAD 300.0E-9
  //#define CNVST_START_OVERHEAD 950.0E-9

  // Number of clock cycles to place the CNVST
  //#define CNVST_CLKS_FROM_ICG 3.75
#define CNVST_CLKS_FROM_ICG 3.75

  // CNVST offset before the next rising edge clock pulse (see above)
  //#define SAR_OFFSET 740E-9 // finish the sar just before the pixel transition
#define SAR_OFFSET 100.E-9 // CNVST at last feasible moment

  inline static float pulse_clk_secs = 0.;

  inline static float pulse_sh_offset_secs = 0.;
```

```
inline static float pulse_sh_secs = 0.;

inline static float pulse_icg_offset_secs = 0.;
inline static float pulse_icg_secs = 0.;

inline static float pulse_cnvst_offset_secs = 0.;
inline static float pulse_cnvst_secs = 0.;

inline static float pulse_loop_period_secs = 0.; // sh and icg shared period

// helper functions
double get_next_phase_time(double t, double p, double phase_fraction) {
  double target_offset = phase_fraction * p;
  double current_pos = fmod(t, p);

  if (current_pos < 0) current_pos += p;

  double wait_time = fmod(target_offset - current_pos, p);
  if (wait_time < 0) wait_time += p;

  return t + wait_time;
}

uint16_t get_next_phase_tick(uint16_t t, uint32_t p, double phase_fraction) {
  uint32_t target_offset = (uint32_t)round(phase_fraction * (double)p);
  uint32_t current_pos = t % p;

  int32_t wait_ticks = (int32_t)target_offset - (int32_t)current_pos;

  if (wait_ticks < 0) {
      wait_ticks += p;
  }

  return (uint16_t)((t + wait_ticks) % p);
}

// get next multiple of a greater than b
uint32_t get_next_multiple(uint32_t a, uint32_t b) {
  uint32_t remainder = b % a;
  return b + (remainder ? a - remainder:0);
}

void print_pulse_parameters()
{
  Serial.printf("timings clk %.4g sh duration %.4g offset %.4g icg duration %.4g offset %.4g cnvst duration %.4g offset %.4g period %4g\n",
                  pulse_clk_secs,
                  pulse_sh_secs,
                  pulse_sh_offset_secs,
                  pulse_icg_secs,
                  pulse_icg_offset_secs,
                  pulse_cnvst_secs,
                  pulse_cnvst_offset_secs,
                  pulse_loop_period_secs);
}

bool set_pulse_parameters(float clock_period_secs=0.4E-6, float period_secs=10.E-6)
{
  if (clock_period_secs>100.E3) {
    // interpret as frequency
    clock_period_secs = 1./clock_period_secs;
  }
  else if (clock_period_secs>1.) {
    // interpret as counts
    clock_period_secs = clock_period_secs/F_BUS_ACTUAL;
  }

  if (clock_period_secs<0.4E-6) {
    Serial.println("Error: clock speeds is too fast, limit 2.5MHz or 0.4E-6secs");
    return false;
  }
  if (clock_period_secs>1.25E-6) {
    Serial.println("Error: clock speeds is too slow, limit 800KHz or 1.25E-6secs");
    return false;
  }

  pulse_clk_secs          = clock_period_secs;

  pulse_sh_offset_secs    = pulse_clk_secs;
  pulse_sh_secs           = 1.E-6;

```

```
  pulse_icg_offset_secs   = pulse_clk_secs/2;

  pulse_icg_secs          = pulse_sh_offset_secs + pulse_sh_secs + 1.E-6;
  pulse_icg_secs          = get_next_phase_time(pulse_icg_secs, clock_period_secs, 0.75);
  pulse_icg_secs          -= pulse_icg_offset_secs;

  pulse_cnvst_offset_secs = (CNVST_CLKS_FROM_ICG * pulse_clk_secs) - SAR_OFFSET - CNVST_START_OVERHEAD;

  pulse_loop_period_secs  = period_secs;

  return true;
}

// ==========================================================

inline static float read_expected_time = 0.;

static void start_read()
{
  switch(mode) {
  case PIT:
    pit_start();
    break;
  case PULSE:
    pulse_start();
    break;
  case PULSE_LOOP:
    pulse_loop_start();
    break;
  default:
    Serial.println("Error: start_read mode not configured.");
    error_flag = true;
  }
}

bool wait_read(float timeout=0., float timestep=0.01, bool interruptible=false, bool verbose=false)
{
  bool retv = false;

  if (!timeout) timeout = read_expected_time - pit_elapsed_secs() + 0.010;
  if (timeout < 0.1) timeout = 0.1;

  switch(mode) {
  case PIT:
    retv = pit_wait(timeout,timestep, interruptible, verbose);
    break;
  case PULSE_LOOP:
    retv = pulse_loop_wait(timeout,timestep, interruptible, verbose);
    break;
  case PULSE:
    retv = flexpwm_wait(timeout,timestep, interruptible, verbose);
    break;
  default:
    Serial.println("Error: wait_read not configured");
    retv = false;
    break;
  }

  return retv;
}

// ----------------------------------------------------------

bool read(uint nframes, float exposure, uint16_t *bufferp,
          void (*frame_callbackf)(),
          void (*frameset_callbackf)(),
          void (*completion_callbackf)(),
          void (*setup_callbackf)(),
          bool start=true)
{

  if (!setup_pulse(pulse_clk_secs,
                   pulse_sh_secs, pulse_sh_offset_secs,
                   pulse_icg_secs, pulse_icg_offset_secs,
                   bufferp, NREADOUT,
                   frame_callbackf)) {
    Serial.println("Error: failed to setup pulse");
    return false;
  }

  frames_complete = false;
```

```
    framesets_complete = false;

    frame_counts = nframes+1;
    frames_completed_callback = frameset_callbackf;
    framesets_completed_callback = completion_callbackf;

    read_expected_time = exposure * nframes;

    if (exposure > 0.) {
      if (!setup_pit(exposure,pit_delay_secs,0)) {
        return false;
      }
    }

    if (setup_callbackf) {
      setup_callbackf();
    }

    if (start) {
      start_read();
    }

    return !error_flag;
  }

  // ------------------------------------------------------------

  bool read(uint nframes, float exposure, float frame_interval, uint16_t *bufferp,
            void (*frame_callbackf)(),
            void (*frameset_callbackf)(),
            void (*completion_callbackf)(),
            void (*setup_callbackf)(),
            bool start=true)
  {

    if (!setup_pulse_loop(pulse_clk_secs, pulse_sh_secs, pulse_sh_offset_secs,
                          pulse_icg_secs, pulse_icg_offset_secs, pulse_loop_period_secs,
                          exposure, frame_interval, nframes,
                          bufferp, NREADOUT, frame_callbackf)) {
      Serial.println("Error: failed to setup pulse loop");
      return false;
    }

    read_expected_time = frame_interval * nframes;
    frames_completed_callback = frameset_callbackf;
    framesets_completed_callback = completion_callbackf;

    if (setup_callbackf) {
      setup_callbackf();
    }

    if (start) {
      pulse_loop_start();
    }

    return !error_flag;
  }

  // ------------------------------------------------------------------------------------
  bool triggered_read(uint ntriggers, uint nframes, float exposure, uint16_t *bufferp,
                      void (*frame_callbackf)(),
                      void (*frameset_callbackf)(),
                      void (*completion_callbackf)(),
                      void (*setup_callbackf)(),
                      bool start=true)
  {
    if (!read(nframes, exposure, bufferp,frame_callbackf, frameset_callbackf, completion_callbackf,
  nullptr, false)) {
      Serial.println("Error: triggerd_read setup read");
      return false;
    }

    if (!setup_triggers(ntriggers)) {
      Serial.println("Error: triggerd_read setup triggers");
      return false;
    }

    if (setup_callbackf) setup_callbackf();

    if (start) {
      return start_triggers();
```

```
    }

    return true;
  }

  bool triggered_read(uint ntriggers, uint nframes, float exposure, float interval, uint16_t *bufferp,
                      void (*frame_callbackf)(),
                      void (*frameset_callbackf)(),
                      void (*completion_callbackf)(),
                      void (*setup_callbackf)(),
                      bool start=true)
  {
    if (!read(nframes, exposure, interval, bufferp,
              frame_callbackf, frameset_callbackf, completion_callbackf, nullptr, false)) {
      Serial.println("Error: triggerd_read setup read");
      return false;
    }

    if (!setup_triggers(ntriggers)) {
      Serial.println("Error: triggerd_read setup trigger");
      return false;
    }

    if (setup_callbackf) setup_callbackf();

    if (start) {
      return start_triggers();
    }

    return true;
  }

  bool wait_triggered_read(float timeout=1., float timestep=0.01, bool interruptible=true, bool verbose=false)
  {
    if (wait_trigger(timeout,timestep,interruptible,verbose)) {
      return wait_read(timeout,timestep,interruptible,verbose);
    }

    return false;
  }


  /* ==========================================================================
     64 bit elapsed time clock based on cpu cycles
     note that this is in the tcd1304 namespace
  */
  static uint64_t cycles64()
  {
    static uint32_t oldCycles = ARM_DWT_CYCCNT;
    static uint32_t highDWORD = 0;
    uint32_t newCycles = ARM_DWT_CYCCNT;
    if (newCycles < oldCycles)
    {
        ++highDWORD;
    }
    oldCycles = newCycles;
    return (((uint64_t)highDWORD << 32) | newCycles);
  }

  /* ----------------------------------------------------
     Elapsed time from first frame (sh) to most recent (cf actual timer interval)
  */
  static double sh_elapsed_secs()
  {
    return (double)(sh_cyccnt64_now - sh_cyccnt64_start)/F_CPU;
  }

  static double sh_difference_secs()
  {
    return (double)(sh_cyccnt64_now - sh_cyccnt64_prev)/F_CPU;
  }

  static double sh_exposure_secs()
  {
    return (double)sh_cyccnt64_exposure/F_CPU;
  }
  /* ----------------------------------------------------
     Elapsed time from timer start to most recent timer interrupt
  */
  static double pit_elapsed_secs()
```

```
{
  return (double)(pit_cyccnt64_now - pit_cyccnt64_start)/F_CPU;
}

static double pit_difference_secs()
{
  return (double)(pit_cyccnt64_now - pit_cyccnt64_prev)/F_CPU;
}

/* ----------------------------------------------------
   Elapsed time from timer start to most recent timer interrupt
*/
static double trigger_elapsed_secs()
{
  return (double)(trigger_cyccnt64_now - trigger_cyccnt64_start)/F_CPU;
}

static double trigger_difference_secs()
{
  return (double)(trigger_cyccnt64_now - trigger_cyccnt64_prev)/F_CPU;
}

/* ============================================================
   Print error message
*/
static void print_errormsg(const char *name, const char *errmsg)
{
  Serial.print("Error: ");
  Serial.print(name);
  Serial.print(" ");
  Serial.println(errmsg);
}

static void print_errormsg(SubModule *p, const char *errmsg)
{
  print_errormsg(p->name,errmsg);
}

static void print_counters()
{
  Serial.print(" sh "); Serial.print(sh_counter);
  Serial.print(" icg "); Serial.print(icg_counter);
  Serial.print(" cnvst "); Serial.print(cnvst_counter);
  Serial.print(" read "); Serial.print(read_counter);
  Serial.print(" "); Serial.print(read_counts);
  Serial.print(" frame "); Serial.print(frame_counter);
  Serial.print(" "); Serial.print(frame_counts);
  Serial.print(" frameset "); Serial.print(frameset_counter);
  Serial.print(" "); Serial.print(frameset_counts);
  if (mode==PIT) {
    Serial.print(" pit "); Serial.print(pit_counter);
    Serial.print(" "); Serial.print(pit_counts);
  }
  if (trigger_mode) {
    Serial.print(" triggers "); Serial.print(trigger_counter);
    Serial.print(" "); Serial.print(trigger_counts);
  }
  Serial.println("");
}

void print_setup_diagnostic(SubModule *p)
{
  Serial.printf("setup %s prescale %d, divider %d (%.6gs) period %lu (%.6gs) onA %u (%.6gs) offA %u (%.6gs)\n",
                p->name,
                p->prescale,p->divider,SECONDS(p->divider),
                p->period_counts, SECONDS(p->period_counts*p->divider),
                p->onA_counts, SECONDS(p->onA_counts*p->divider),
                p->offA_counts, SECONDS(p->offA_counts*p->divider));
}

/* ==========================================================================
   Print, check, setup submodule configuration, load into hardware
*/
void print_setup()
{
  print_submodule(&clk);
  print_submodule(&icg);
  print_submodule(&sh);
  print_submodule(&cnvst);

```

```
  Serial.print("configured mode:");
  if (trigger_mode) {
    Serial.print(" triggered");
  }

  switch(mode) {
  case PULSE:
    Serial.println(" PULSE");
    break;
  case PULSE_LOOP:
    Serial.println(" PULSE_LOOP");
    break;
  case PIT:
    Serial.printf(" PIT %f %f\n", pit_delay_secs, pit_interval_secs);
    break;
  default:
    Serial.println(" NONE");
  }
}

void print_submodule(SubModule *p)
{
  char cbuffer[128] = {0};

  float frequency = p->period_counts ? F_BUS_ACTUAL/(p->divider*p->period_counts) : 0;
  float period_secs = (float) (p->period_counts*p->divider)/F_BUS_ACTUAL;

  float A_on_secs = (float)(p->onA_counts*p->divider)/F_BUS_ACTUAL;
  float A_off_secs = (float)(p->offA_counts*p->divider)/F_BUS_ACTUAL;
  float A_duration_secs = (float)((p->offA_counts-p->onA_counts)*p->divider)/F_BUS_ACTUAL;

  float B_on_secs = (float)(p->onB_counts*p->divider)/F_BUS_ACTUAL;
  float B_off_secs = (float)(p->offB_counts*p->divider)/F_BUS_ACTUAL;
  float B_duration_secs = (float)((p->offB_counts-p->onB_counts)*p->divider)/F_BUS_ACTUAL;

  snprintf( cbuffer, sizeof(cbuffer),
            "FLEXPWM: %s submod %d mask 0x%x pinA %d muxA %d pinB %d muxB %d irq %d flexpwm %p",
            p->name, p->submod, p->mask, p->pinA, p->muxvalA, p->pinB, p->muxvalB, p->irq, p->flexpwm);
  Serial.println(cbuffer);

  snprintf( cbuffer, sizeof(cbuffer),
        "flexpwm: %s period %lu presc %d div %d offset %.1f => %.6g secs %.6g Hz ctrl2_mask x%02x",
        p->name, p->period_counts, p->prescale, p->divider, p->offset, period_secs, frequency,
p->ctrl2_mask);
  Serial.println(cbuffer);

  snprintf( cbuffer, sizeof(cbuffer),
        "flexpwm: %s A pin %d mux 0x%x on %u off %u %s => %.6g to %.6g, %.6g secs",
        p->name, p->pinA, p->muxvalA, p->onA_counts, p->offA_counts,
        p->invertA?"inverting":"noninverting",
        A_on_secs, A_off_secs, A_duration_secs);
  Serial.println(cbuffer);

  snprintf( cbuffer, sizeof(cbuffer),
        "flexpwm: %s B pin %d mux 0x%x on %u off %u %s => %.6g to %.6g, %.6g secs",
        p->name, p->pinB, p->muxvalB, p->onB_counts, p->offB_counts,
        p->invertB?"inverting":"noninverting",
        B_on_secs, B_off_secs, B_duration_secs);
  Serial.println(cbuffer);
}

// -------------------------------------------------------
bool check_submodule(SubModule *p)
{
  bool retv = true;
  if (p->period_counts > 65536) {
    print_errormsg(p->name, "period counts > 2^16");
  }
  if (p->divider<1) {
    print_errormsg(p->name, "divider < 1");
    retv = false;
  }
  if (p->divider>128) {
    print_errormsg(p->name, "divider > 128");
    retv = false;
  }

  return retv;
}

// -------------------------------------------------------
```

```
bool print_and_check_submodule(SubModule *p)
{
  print_submodule(p);
  return check_submodule(p);
}

// ----------------------------------------------------
void load_submodule(SubModule *p)
{
  IMXRT_FLEXPWM_t *q = p->flexpwm;
  unsigned int submod = p->submod;
  uint16_t mask = p->mask;

  Serial.print("loading sub_module ");
  Serial.println(p->name);

  // stop this channel - harmless if it was done globally already
  q->MCTRL |= FLEXPWM_MCTRL_CLDOK(mask);
  q->MCTRL &= ~FLEXPWM_MCTRL_RUN(mask);

  q->SM[submod].CTRL = FLEXPWM_SMCTRL_FULL | FLEXPWM_SMCTRL_PRSC(p->prescale);

  q->SM[submod].INIT = 0;
  q->SM[submod].CNT  = 0;
  q->SM[submod].VAL0 = 0;
  q->SM[submod].VAL1 = (uint16_t) (p->period_counts - 1);
  q->SM[submod].VAL2 = p->onA_counts;
  q->SM[submod].VAL3 = (p->offA_counts > 0) ? p->offA_counts: 0;
  q->SM[submod].VAL4 = p->onB_counts;
  q->SM[submod].VAL5 = (p->offB_counts > 0) ? p->offB_counts : 0;

  // Convenience, save the clock period in seconds
  p->period_secs = (float) p->period_counts * ((float)(1«p->prescale)/ F_BUS_ACTUAL);

  // Do we have a Pin for the A channel?
  if (p->pinA != 0xFF && (p->offA_counts|p->onA_counts)) {

    q->OUTEN |= FLEXPWM_OUTEN_PWMA_EN(mask);

    *(portConfigRegister(p->pinA)) =  p->muxvalA;

    if (p->invertA) {
      q->SM[submod].OCTRL |= 1«10;  // Is inverted
    }
    else {
      q->SM[submod].OCTRL &= ~(1«10);  // Is not inverted
    }
  }
  else {
    q->OUTEN &= ~FLEXPWM_OUTEN_PWMA_EN(mask);
  }

  // Do we have a Pin for the B channel?
  if (p->pinB != 0xFF && (p->offB_counts|p->onB_counts)) {

    q->OUTEN |= FLEXPWM_OUTEN_PWMB_EN(mask);

    *(portConfigRegister(p->pinB)) = p->muxvalB;

    if (p->invertB) {
      q->SM[submod].OCTRL |= 1«9;  // Is inverted
    }
    else {
      q->SM[submod].OCTRL &= ~(1«9);  // Is not inverted
    }
  }
  else {
    q->OUTEN &= ~FLEXPWM_OUTEN_PWMB_EN(mask);
  }

  // Setup shared clocks and forced starts
  q->SM[submod].CTRL2 = p->ctrl2_mask;

  p->newvals = false;
}

// ----------------------------------------------------------------------------
bool setup_submodule( SubModule *p, uint8_t prescale, uint32_t period_counts,
                      uint16_t onA_counts, uint16_t offA_counts, bool invertA,
                      uint16_t onB_counts, uint16_t offB_counts, bool invertB,
                      uint16_t ctrl2_mask)
```

```
{
  if (p) {
    p->divider = (1«p->prescale);
    p->period_counts = period_counts;

    p->onA_counts = onA_counts;
    p->offA_counts = offA_counts;
    p->invertA = invertA;

    p->onB_counts = onB_counts;
    p->offB_counts = offB_counts;
    p->invertB = invertB;

    p->ctrl2_mask = ctrl2_mask;

    if (print_and_check_submodule(p)) {
      // Here we actually load it into the registers
      load_submodule(p);
      return true;
    }
  }
  return false;
}

/* ========================================================================
   Attach ISR and enable
   see above, CMPF_MASKA_ON, CMPF_MASKA_OFF, etc
*/
void attach_isr( SubModule *p, uint16_t cmpf_mask, void (*isrf)(), uint8_t priority=0, bool enable=true)
{
  IMXRT_FLEXPWM_t *q = p->flexpwm;
  unsigned int submod = p->submod;
  uint16_t status;

  // disable the irq for this submodule
  NVIC_DISABLE_IRQ(p->irq);

  // clear all of this module's interrupt status bits
  status = q->SM[submod].STS;
  q->SM[submod].STS = status;

  // enable the specified bits only
  q->SM[submod].INTEN = enable ? cmpf_mask : 0;
  p->intena_mask = cmpf_mask;

  // bookkeeping
  p->isr = isrf;

  // override the configured priority
  if (priority > 0) {
    p->interrupt_priority = priority;
  }

  // register the isr to this irq
  attachInterruptVector(p->irq, isrf);

  // set priority
  if (p->interrupt_priority > 0) {
    NVIC_SET_PRIORITY(p->irq, p->interrupt_priority);
  }

  // and now, enable interrupts on this irq
  NVIC_ENABLE_IRQ(p->irq);
}


/* ============================================================
   Setup callbacks for frames (a single frameset) completed.
*/
void clear_frames_completed_callback()
{
  frames_completed_callback = 0;
}

void load_frames_completed_callback(void (*callback)(), unsigned int nframes=0)
{
  // allow for rearm callback. same nframes
  if (nframes) {
    frame_counts = nframes;
  }
  frames_completed_callback = callback;
```

```
}

/* ============================================================
   Setup callbacks for frameset completed.
*/
void clear_framesets_completed_callback()
{
  framesets_completed_callback = 0;
}

void load_framesets_completed_callback(void (*callback)(), unsigned int nsets=0)
{
  // allow for rearm callback. same nframes
  if (nsets) {
    frameset_counts = nsets;
  }
  framesets_completed_callback = callback;
}

/* ==========================================================================================
   Generic wait routine
*/
bool generic_wait(bool *flag, float timeout_=1., float timestep_=0.01,
                  bool interruptible=false, bool verbose=false)
{
  uint32_t timeout = (uint) (timeout_ * 1000);
  uint32_t increment = (uint) (timestep_ * 1000);
  uint32_t elapsed = 0;

  if (verbose) {
    Serial.printf("wait %d msec %d msec\n", timeout, increment);
  }

  while((*flag) && !error_flag && elapsed < timeout) {
    delay(increment);
    elapsed += increment;
    if (interruptible && Serial.available( )) {
      Serial.println("Abort: Wait interrupted by serial input available");
      return false;
    }
  }

  if (error_flag) {
    Serial.println("Error: wait - error_flag is set.");
    return false;
  }

  if (elapsed >= timeout) {
    Serial.printf("Error: wait timeout elapsed %d timeout %d msecs\n", elapsed, timeout);
    return false;
  }

  if ((*flag)) {
    Serial.printf("Error: wait oops!  Return with timer still running, elapsed %d timeout %d msecs\n", elapsed, timeout);
    return false;
  }

  if (verbose) {
    Serial.printf("Success: wait running %d err0r %d elapsed %d timeout %d msecs\n", *flag, error_flag, elapsed, timeout);
  }

  return true;
}

/* ==================================================================
   CNVST idle start
*/
static void cnvst_start()
{
  flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(CNVST_MASK);
  flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN(CNVST_MASK); // set run clk, sh, icg
}

/* ==================================================================
   FlexPWM stop and wait
*/

inline static bool flexpwm_running = false;

```

```
static void flexpwm_stop(uint8_t mask=0xF)
{
  flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(mask);
  flexpwm_running = false;
}

static void flexpwm_start(uint8_t mask=0xF)
{
  oops_flag = false;
  error_flag = false;
  flexpwm_running = true;

  // Set load ok bits - clk, sh, icg
  flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(mask);

  // set run bits - clk, sh, icg
  flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN(mask);
}

static void flexpwm_resetcounters(uint8_t mask=0xFF)
{
  flexpwm_running = false;

  // stop
  flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(mask); // clear run clk, sh, icg

  // clear the load bits
  flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(mask);

  // clear the counters  - clk, sh, icg
  if (mask&0x1) flexpwm->SM[0].CNT = 0;
  if (mask&0x2) flexpwm->SM[1].CNT = 0;
  if (mask&0x4) flexpwm->SM[2].CNT = 0;
  if (mask&0x8) flexpwm->SM[3].CNT = 0;

  // now we load and init  - clk, sh, icg
  flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(mask);

  // and force initialization - through clk - sh and icg are slaved
  //flexpwm->SM[CLK_SUBMODULE].CTRL2 |= FLEXPWM_SMCTRL2_FORCE;  // we probably dont need this
}

bool flexpwm_wait(float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
{
  if (verbose) {
    Serial.print("waiting for flexpwm_running");
  }
  return generic_wait(&flexpwm_running,timeout_,timestep_,interruptible,verbose);
}


inline static bool pulse_armed = false;

static void pulse_sh_isr()
{
  uint16_t status;

  bool do_sync_toggle_here = false;

  // this is a trailing edge interrupt := exposure timer
  uint64_t cyccnt64_now = cycles64();

  // clear the interrupt
  status = flexpwm->SM[SH_SUBMODULE].STS;
  flexpwm->SM[SH_SUBMODULE].STS = status;

  // ======================================
  // no clearing pulses
  if (!sh_clearing_counts) {
    // no more sh interrupts
    flexpwm->SM[SH_SUBMODULE].INTEN = 0;

    // never on again until next start
    flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(SH_MASK);
    flexpwm->SM[SH_SUBMODULE].VAL2 = 0xFFFF;
    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(SH_MASK);

    do_sync_toggle_here = true;
  }

  // first pulse
```

```
    else if (!sh_clearing_counter) {
      /* shorten the period, this will happen
     after completing the present period. */
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(SH_MASK);
      flexpwm->SM[SH_SUBMODULE].VAL1 = sh_short_period_counts;
      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(SH_MASK);

      // this is the end of the previous exposure
      do_sync_toggle_here = true;

      // need to count
      sh_clearing_counter++;
    }

    // last pulse?
    else if (++sh_clearing_counter == sh_clearing_counts) {
      // no more sh interrupts
      flexpwm->SM[SH_SUBMODULE].INTEN = 0;

      // never on again until next start
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(SH_MASK);
      flexpwm->SM[SH_SUBMODULE].VAL2 = 0xFFFF;
      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(SH_MASK);

      sh_clearing_counter = 0;

      // this is the start of the next exposure
      do_sync_toggle_here = true;
    }

    // for elapsed time and exposure
    sh_cyccnt64_prev = sh_cyccnt64_now;
    sh_cyccnt64_now  = cyccnt64_now;

    // sync is toggled on start and end of exposure
    if (sync_enabled && do_sync_toggle_here) {
      digitalToggleFast(SYNC_PIN);
      synctoggled = !synctoggled;
    }

    // ====================================
    // bookkeeping
    sh_counter++;

    // diagnostics
    if (!(status&CMPF_MASKA_OFF)) {
      oops_flag = true;
      Serial.printf("OOPS! pulse sh without off bit set 0x%04x", status);
    }
  }

  static void pulse_icg_isr()
  {
    uint16_t status;

    status = flexpwm->SM[ICG_SUBMODULE].STS;
    flexpwm->SM[ICG_SUBMODULE].STS = status;

    //Serial.print("icg_isr "); print_counters();
    if (status & CMPF_MASKA_ON) {

      //Serial.println("pulse icg maska_on");

      // prepare to restart the CNVST clock
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(CNVST_MASK);   // clear ldok
      flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(CNVST_MASK);    // stop

      flexpwm->SM[CNVST_SUBMODULE].STS = flexpwm->SM[CNVST_SUBMODULE].STS;
      NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + CNVST_SUBMODULE);

      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(CNVST_MASK);    // set ldok
      flexpwm->SM[CNVST_SUBMODULE].CNT = 0;                // zero the submodule counter
      flexpwm->SM[CNVST_SUBMODULE].CTRL2 |= FLEXPWM_SMCTRL2_FORCE;  // force init
      read_counter = 0;                                    // zero the readout counter
    }

    else if (status & CMPF_MASKA_OFF) {

      //Serial.println("pulse icg maska_off");

      // Start the cnvst clock
```

```
      flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN(CNVST_MASK);

      // no more icg interrupts
      flexpwm->SM[ICG_SUBMODULE].INTEN = 0;

      // never on again - slaved, run bit is not effective
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(ICG_MASK);
      flexpwm->SM[ICG_SUBMODULE].VAL2 = 0xFFFF;
      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(ICG_MASK);

      // here is our exposure, diff to most recent previous sh gate
      sh_cyccnt64_exposure = sh_cyccnt64_now - sh_cyccnt64_prev;

      // bookkeeping
      icg_counter++;
    }

    // diagnostics
    else {
      oops_flag = true;
      Serial.println("OOPS! pulse icg without on or off bit set");
    }
  }

  static void pulse_cnvst_isr()
  {
    uint16_t status;

    status = flexpwm->SM[CNVST_SUBMODULE].STS;
    flexpwm->SM[CNVST_SUBMODULE].STS = status;

    if ((status&CMPF_MASKA_OFF)) {

#ifndef ALLINONEBOARD
        // Assert the convert pin
        SETCNVST;
        delayNanoseconds( 670 );  // 710 nanoseconds minus spi setup time
        CLEARCNVST;               // need 30 nanoseconds after this
#endif

      if (read_counter < read_counts) {

#ifdef ALLINONEBOARD
        adc->adc0->startReadFast(ANALOGPIN);
        while ( adc->adc0->isConverting() );
        *read_pointer = adc->adc0->readSingle();
#else
        *read_pointer = SPI.transfer16(0xFFFF);
        *read_pointer ^= (0x1<<15);
#endif

        // bookkeeping for the read
        read_pointer++;
        read_counter++;

        // The read is done
        if (read_counter == read_counts) {

          // wait for start of next clk period
          if (flexpwm->MCTRL & FLEXPWM_MCTRL_RUN(CLK_MASK)) {
            while (!(flexpwm->SM[CLK_SUBMODULE].STS & FLEXPWM_SMSTS_RF));
            flexpwm->SM[CLK_SUBMODULE].STS = FLEXPWM_SMSTS_RF;
          }

          // Stop clk, sh, icg
          // ** Indicate stop only after callback **
          flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((CLK_MASK|SH_MASK|ICG_MASK));

          // now stop everything
          flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN((CLK_MASK|SH_MASK|ICG_MASK));

          flexpwm_running = false;

#ifdef  CNVST_IDLE_DISABLE
          flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(CNVST_MASK);
#endif

          // ----------------------------------------
          // User supplied function, per read complete
          // First exposure is frame 1, frame 0 is the pre-exposure

```

```
          frames_complete = (frame_counter+1 >= frame_counts);
          framesets_complete = frames_complete && ((frameset_counter+1) >= frameset_counts);

          if (read_callback) {
            read_callback();
          }

          // ============================================
          // Frame bookkeeping, re-arm etc
          frame_counter++;

          // More frames?
          if (frame_counter < frame_counts) {
            pulse_reset_and_arm();
          }

          // Nope, we're done with this frameset
          else if (frame_counter == frame_counts) {

        if (frames_completed_callback) {
          frames_completed_callback();
        }

            // frameset completed, update frameset counter
            frameset_counter++;

            // More framesets?
            if (frameset_counter < frameset_counts) {

              sh_cyccnt64_start   = 0;
              sh_cyccnt64_prev    = 0;
              sh_cyccnt64_now     = 0;

              sh_counter    = 0;
              icg_counter   = 0;
              cnvst_counter = 0;

              frame_counter = 0;

              pulse_reset_and_arm();
            }

            // Nope, we're done with all of the framesets
            else if (frameset_counter == frameset_counts) {

              if (framesets_completed_callback) {
                framesets_completed_callback();
              }

              // and get ready for the next frame set
              sh_cyccnt64_start   = 0;
              sh_cyccnt64_prev    = 0;
              sh_cyccnt64_now     = 0;

              sh_counter    = 0;
              icg_counter   = 0;
              cnvst_counter = 0;

              frame_counter = 0;
              frameset_counter = 0;

              pulse_reset_and_arm();
            }

            // oops, shouldn't get here
            else {
              oops_flag = true;
              Serial.printf("line %d, read_counter %d/%d frameset %d/%d\n", __LINE__, read_counter, read_counts, frameset_counter, frameset_counts);
              Serial.println("Oops!  cnvst interrupt after frameset_counts complete");
              stop_all();
            }

          }

          // oops, shouldn't get here
          else {
            oops_flag = true;
            Serial.printf("line %d, read_counter %d/%d frameset %d/%d\n", __LINE__, read_counter, read_counts, frameset_counter, frameset_counts);
            Serial.println("Oops!  cnvst interrupt after frame_counts complete");
```

```
            stop_all();
          }

        }

      }

    }

    // oops, shouldn't get here
    else {
      oops_flag = true;
      //Serial.printf("OOPS! pulse cnvst without off bit set 0x%x\n", status);
    }

    // diagnostics
    cnvst_counter++;

  }

  static void pulse_start()
  {
    if (!pulse_armed) {
      error_flag = true;
      Serial.println("\nError: pulse not armed");
      return;
    }
    pulse_armed = false;

    flexpwm_running = true;
    flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN((CLK_MASK|SH_MASK|ICG_MASK)); // set run clk, sh, icg
  }

  static void pulse_reset_and_arm()
  {
    uint16_t status;

    // Reset the clk
    flexpwm->SM[CLK_SUBMODULE].CNT = 0;

    // Restore and reset the SH submodule
    status = flexpwm->SM[SH_SUBMODULE].STS;
    flexpwm->SM[SH_SUBMODULE].STS = status;

    flexpwm->SM[SH_SUBMODULE].INTEN = CMPF_MASKA_OFF;
    flexpwm->SM[SH_SUBMODULE].VAL1 = (uint16_t) (sh.period_counts - 1);
    flexpwm->SM[SH_SUBMODULE].VAL2 = sh.onA_counts;
    flexpwm->SM[SH_SUBMODULE].VAL3 = sh.offA_counts;
    flexpwm->SM[SH_SUBMODULE].CNT = 0;

    sh_clearing_counter = 0;

    // Restore and reset the ICG submodule
    status = flexpwm->SM[ICG_SUBMODULE].STS;
    flexpwm->SM[ICG_SUBMODULE].STS = status;

    flexpwm->SM[ICG_SUBMODULE].INTEN = CMPF_MASKA_ON_OFF;

    flexpwm->SM[ICG_SUBMODULE].VAL2 = icg.onA_counts;
    flexpwm->SM[ICG_SUBMODULE].VAL3 = icg.offA_counts;
    flexpwm->SM[ICG_SUBMODULE].CNT = 0;

    // CNVST is never disabled, but do Reset the buffer pointer
    read_pointer = read_buffer;
    read_counter = read_counts; // will be set to 0 in the icg isr

    // and renable
    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((CLK_MASK|SH_MASK|ICG_MASK));
    flexpwm->SM[CLK_SUBMODULE].CTRL2 |= FLEXPWM_SMCTRL2_FORCE;

    pulse_armed = true;
  }

  bool setup_pulse(float clk_secs, float sh_secs, float sh_offset_secs, float icg_secs, float
icg_offset_secs,
                   uint16_t *buffer, unsigned int nbuffer, void (*callbackf)())
  {
    char cbuffer[128] = {0};

    // stops all, clears ldok, disables irqs and clears mode
    stop_all();
```

```
    // clear the error flags
    clear_error_flags();

    // and we show the settings
    sprintf(cbuffer,
            "#tcd1304 setup pulse, clk %.5gs sh %.5gs offset %.5gs icg offset %.5gs",
        clk_secs, sh_secs, sh_offset_secs, icg_offset_secs);
    Serial.println(cbuffer);

    // clear elapsed times
    sh_cyccnt64_start   = 0;
    sh_cyccnt64_prev    = 0;
    sh_cyccnt64_now     = 0;

    pit_cyccnt64_start   = 0;
    pit_cyccnt64_prev    = 0;
    pit_cyccnt64_now     = 0;

    trigger_cyccnt64_start   = 0;
    trigger_cyccnt64_prev    = 0;
    trigger_cyccnt64_now     = 0;

    // clear bookkeeping counters
    sh_counter    = 0;
    icg_counter   = 0;
    cnvst_counter = 0;

    sh_clearing_counter = 0;

    // default to 1 frame and 1 frameset
    frame_counter = 0;
    frame_counts = 1;

    frameset_counter = 0;
    frameset_counts = 1;

    // ----------------------------------------------------------------
    // Setup the master clock

    if (clk_secs < 0.25E-6) {
      Serial.println("Error: requested clk < 0.25E-6 (4MHz)");
      error_flag = true;
      return false;
    }

    if (clk_secs > 1.25E-6) {
      Serial.println("Error: requested clk > 1.25E-6 (800kHZ)");
      error_flag = true;
      return false;
    }

    clk.prescale = 0;
    clk.divider  = 1;

    clk.period_counts = ceil(clk_secs*F_BUS_ACTUAL);    // rouund up

    clk.offset        = 0.;

    clk.offA_counts   = 0;
    clk.onA_counts    = (clk.period_counts/2);
    clk.invertA       = false;

    clk.onB_counts    = 0;
    clk.offB_counts   = 0;
    clk.invertB       = false;

    clk.intena_mask    = 0;

    clk.ctrl2_mask    = PWM_CTRL2_CLOCK_MASTER;

    // ----------------------------------------------------------------
    // Setup the CNVST clock, 4 times the clock period,
    cnvst.prescale = clk.prescale;
    cnvst.divider = (1«cnvst.prescale);

    cnvst.period_counts = 4*clk.period_counts;

    cnvst.offset        = 0;

    cnvst.onA_counts    = ceil(pulse_cnvst_offset_secs*F_BUS_ACTUAL/cnvst.divider);    // delay to approx
```

```
the fourth clock
    cnvst.offA_counts   = cnvst.onA_counts + ceil(pulse_cnvst_secs*F_BUS_ACTUAL/cnvst.divider);
    cnvst.invertA       = false;

    cnvst.onB_counts    = 0;
    cnvst.offB_counts   = 0;
    cnvst.invertB       = false;

    //cnvst.ctrl2_mask    = PWM_CTRL2_CLOCK_SYNC;
    cnvst.ctrl2_mask    = PWM_CTRL2_CLOCK_MASTER; // needs to be master, because run bit is disabled when
running from clk's clock.

    // ----------------------------------------------------------------
    // setup the sh, sh_offset-icg_offset is the icg-sh timing, runs once, isr has to set on,off to 0xffff
    sh.prescale = clk.prescale;
    sh.divider  = (1«sh.prescale);

    sh.period_counts  = 0x8000;   // long period, we are going to pulse once and stop

    sh.offset         = 0.;

    sh.onA_counts     = ceil(sh_offset_secs*F_BUS_ACTUAL/sh.divider);
    sh.offA_counts    = sh.onA_counts + ceil(sh_secs*F_BUS_ACTUAL/sh.divider);
    sh.invertA        = false;

    sh.onB_counts     = 0;
    sh.offB_counts    = 0;
    sh.invertB        = false;

    sh.ctrl2_mask     = PWM_CTRL2_CLOCK_SLAVE;
    //sh.ctrl2_mask     = FLEXPWM_SMCTRL2_CLK_SEL(0x2);

    // ----------------------------------------------------------------
    // setup icg,  icg interrupt has to stop sh and icg by moving val2  to > val1
    icg.prescale = clk.prescale;
    icg.divider  = (1«icg.prescale);

    icg.period_counts = 0x8000;   // long period, we are going to pulse once and stop

    icg.offset        = 0.;

    icg.onA_counts    = ceil(icg_offset_secs*F_BUS_ACTUAL/icg.divider);
    icg.offA_counts   = icg.onA_counts + ceil(icg_secs*F_BUS_ACTUAL/icg.divider);
    icg.invertA       = true;

    icg.onB_counts    = 0;
    icg.offB_counts   = 0;
    icg.invertB       = false;

    icg.ctrl2_mask     = PWM_CTRL2_CLOCK_SLAVE;
    //icg.ctrl2_mask    = FLEXPWM_SMCTRL2_CLK_SEL(0x2);

    // this is used by the timing graphics software
    cnvst.offset = (((float)icg.offA_counts / F_BUS_ACTUAL) + CNVST_START_OVERHEAD)*1.E9;

    // ----------------------------------------------------------------
    // optional: sh gate clearing cycles
    if (sh_clearing_counts) {
      // set sh period to after icg but before the first cnvst isr
      sh.period_counts = icg.offA_counts + clk.period_counts;   //
    }

    // for short clearing cycles, the period is twice the sh pulse width
    sh_short_period_counts = 2*(sh.offA_counts - sh.onA_counts) + 1;

    // if sh offset is too large, the period is the full offset + width
    if (sh_short_period_counts <= sh.offA_counts) {
      sh_short_period_counts = sh.offA_counts + 1;
    }

    // ----------------------------------------------------------------
    Serial.println("");
    if (!print_and_check_submodule(&clk)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    if (!print_and_check_submodule(&icg)) {
      error_flag = true;
      return false;
```

```
    }

    Serial.println("");
    if (!print_and_check_submodule(&sh)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    if (!print_and_check_submodule(&cnvst)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    Serial.print("flexpwm: clearing_pulses ");
    Serial.println(sh_clearing_counts);

    // ----------------------------------------------------------------

    load_submodule(&clk);
    load_submodule(&icg);
    load_submodule(&sh);
    load_submodule(&cnvst);

    // ---------------------------------------------
    // Setup the readout buffer
    read_buffer   = buffer;
    read_counts   = nbuffer;

    read_pointer  = read_buffer;
    read_counter  = read_counts; // will be set to 0 in the icg isr

    read_callback = callbackf;

    // ---------------------------------------------
    attach_isr( &sh, CMPF_MASKA_OFF, pulse_sh_isr, 128);
    attach_isr( &icg, CMPF_MASKA_ON_OFF, pulse_icg_isr, 64);
    attach_isr( &cnvst, CMPF_MASKA_OFF, pulse_cnvst_isr, 64);

    // ---------------------------------------------
    // stuff that other routines need to know
    mode = PULSE;
    trigger_mode = false;

    // timer needs this
    pulse_interframe_min_secs  = (icg.offA_counts/F_BUS_ACTUAL)*icg.divider;    // offset before read starts
    pulse_interframe_min_secs += ((float)cnvst.period_counts/F_BUS_ACTUAL)*cnvst.divider * read_counts; // add read time
    pulse_interframe_min_secs += USBTRANSFERSECS;  // add usb transfer time

    // round-up minimum interframe time to 1 msec
    pulse_interframe_min_secs = ceil((pulse_interframe_min_secs+1.E-3)/1.E-3)*1.E-3;

    Serial.print("TCD1304 setup_pulse success, min frame interval ");
    Serial.print(pulse_interframe_min_secs,6);
    Serial.println(" secs");

    // now we set load ok and force init  - clk, sh, icg
    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((CLK_MASK|SH_MASK|ICG_MASK));
    flexpwm->SM[CLK_SUBMODULE].CTRL2 = FLEXPWM_SMCTRL2_FORCE;  // force out while ldok, initializes the counters

    pulse_armed  = true;

    return true;
  }

  // ADC read to measure the zero

  inline static bool adc_blank_running = false;

  inline static uint adc_blank_counter = 0;

  inline static uint adc_raw_offset = 0;
  inline static double adc_offset = 0.;

  inline static bool adc_offset_done = false;

  static void adc_blank_isr()
```

```
  {
    uint16_t status;

    uint16_t u16;

    status = flexpwm->SM[CNVST_SUBMODULE].STS;
    flexpwm->SM[CNVST_SUBMODULE].STS = status;

    if ((status&CMPF_MASKA_OFF)) {

#ifdef ALLINONEBOARD
      adc->adc0->startReadFast(ANALOGPIN);
      while ( adc->adc0->isConverting() );
      u16 = adc->adc0->readSingle();
#else
      SETCNVST;
      delayNanoseconds( 670 );  // 710 nanoseconds minus spi setup time
      CLEARCNVST;               // need 30 nanoseconds after this

      u16 = SPI.transfer16(0xFFFF);
      u16 ^= (0x1<<15);
#endif

      adc_blank_counter++;

      if (adc_blank_counter > NREADOUT) {

        adc_raw_offset += u16;

        if (adc_blank_counter >= 2*NREADOUT) {

          // stop the counter
          flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(CNVST_MASK);
          flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(CNVST_MASK);

          // generate the results
          adc_blank_running = false;
        }

      }

    }

  }

  bool read_adc_offset()
  {
    stop_all();

    adc_blank_running = false;

    adc_raw_offset = 0;
    adc_offset = 0;

    adc_blank_counter = 0;

    // load the cnvst submodule

    cnvst.prescale = 0;
    cnvst.divider  = 1;

    cnvst.period_counts = 4*ceil(pulse_clk_secs*F_BUS_ACTUAL) ;

    cnvst.offset        = 0;

    cnvst.onA_counts    = ceil(pulse_cnvst_offset_secs*F_BUS_ACTUAL/cnvst.divider);    // delay to approx the fourth clock
    cnvst.offA_counts   = cnvst.onA_counts + ceil(pulse_cnvst_secs*F_BUS_ACTUAL/cnvst.divider);
    cnvst.invertA       = false;

    cnvst.onB_counts    = 0;
    cnvst.offB_counts   = 0;
    cnvst.invertB       = false;

    cnvst.ctrl2_mask    = PWM_CTRL2_CLOCK_MASTER; // needs to be master, because run bit is disabled when running from clk's clock.

    // check it
    Serial.println("");
    if (!print_and_check_submodule(&cnvst)) {
      error_flag = true;
```

```
      return false;
    }

    // load it
    load_submodule(&cnvst);

    attach_isr( &cnvst, CMPF_MASKA_OFF, adc_blank_isr, 64);

    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(CNVST_MASK);

    flexpwm->SM[CNVST_SUBMODULE].CTRL2 = FLEXPWM_SMCTRL2_FORCE;  // force out while ldok, initializes the counters

    flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN(CNVST_MASK);

    adc_blank_running = true;

    while(adc_blank_running) {
      yield();
    }
    Serial.printf("adc_blank_measurement sum %u counter %d\n",adc_raw_offset,adc_blank_counter);

    adc_offset = (adc_raw_offset * VPERBIT)/NREADOUT;
    adc_raw_offset /= NREADOUT;
    adc_offset_done = true;

    Serial.printf("adc_blank_measurement mean %u ADU %.6g volts counter %d\n",adc_raw_offset,adc_offset,adc_blank_counter);

    return true;
  }

  // Pulse loop, exposure times and frame interval all managed by statemachine within isrs and the flexpwm

  typedef enum {
    IDLE,
    START_EXPOSURE,
    WAIT_EXPOSURE,
    WAIT_FRAME,
    READOUT,
  } PULSE_LOOP_STATE;

  inline static bool pulse_loop_armed = false;
  inline static bool pulse_loop_running = false;

  inline static uint64_t cyccnt64_prepare_readout = 0;
  inline static uint64_t cyccnt64_prepare_exposure = 0;

  inline static PULSE_LOOP_STATE pulse_loop_state = IDLE;

  inline static float pulse_loop_frame_min_secs  = 0.;

#ifdef DEBUG
  inline static uint64_t cyccnt_at_wait_frame = 0;
  inline static uint64_t cyccnt_at_start_exposure = 0;
  inline static uint64_t cyccnt_at_wait_exposure = 0;

  static void print_sh_debugger()
  {
    double tdiff = 0.;

    Serial.print("debuggery wait_frame "); Serial.print(cyccnt_at_wait_frame);
    Serial.print(" start_exposure "); Serial.print(cyccnt_at_start_exposure);
    Serial.print(" wait_exposure "); Serial.print(cyccnt_at_wait_exposure);

    tdiff = (double)(cyccnt_at_start_exposure - cyccnt_at_wait_frame)/F_CPU;
    Serial.print(" start_exp - wait_frame "); Serial.print(tdiff,9);

    tdiff = (double)(cyccnt_at_wait_exposure - cyccnt_at_wait_frame)/F_CPU;
    Serial.print(" wait_exp - wait_frame "); Serial.print(tdiff,9);

    tdiff = (double)cyccnt64_prepare_exposure/F_CPU;
    Serial.print(" prepare exposure "); Serial.print(tdiff,9);

    tdiff = (double)cyccnt64_prepare_readout/F_CPU;
    Serial.print(" prepare readout "); Serial.print(tdiff,9);

    Serial.println("");
  }
#endif

```

```
static void pulse_loop_sh_isr()
{
  uint16_t status;

  // this is a trailing edge interrupt := exposure timer
  uint64_t cyccnt64_now = cycles64();

  // clear the interrupt
  status = flexpwm->SM[SH_SUBMODULE].STS;
  flexpwm->SM[SH_SUBMODULE].STS = status;

  sh_counter++; // set this to zero at start

  switch(pulse_loop_state) {

  case WAIT_FRAME:

    if ( (cyccnt64_now - sh_cyccnt64_prev) >= cyccnt64_prepare_exposure ) {

      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((SH_MASK | ICG_MASK));
      flexpwm->SM[SH_SUBMODULE].VAL2 = sh.onA_counts;  // next cycle we have SH
      flexpwm->SM[ICG_SUBMODULE].VAL2 = 0xFFFF;        // but not ICG on
      flexpwm->SM[ICG_SUBMODULE].VAL3 = 0xFFFF;        // and not ICG off (interrupt)
      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((SH_MASK | ICG_MASK));

      pulse_loop_state = START_EXPOSURE;

      // for elapsed time and exposure
      sh_cyccnt64_prev = sh_cyccnt64_now;
      sh_cyccnt64_now  = cyccnt64_now;

#ifdef DEBUG
      cyccnt_at_wait_frame = cyccnt64_now;
#endif
    }

    break;

  case START_EXPOSURE:

#ifdef DEBUG
    Serial.print("start exp ");
    Serial.println(cyccnt64_now);
#endif

    flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((SH_MASK | ICG_MASK));
    flexpwm->SM[SH_SUBMODULE].VAL2 = 0xFFFF;   // no more SH until exposure over
    flexpwm->SM[ICG_SUBMODULE].VAL2 = 0xFFFF;  // no more ICG on until exposure over
    flexpwm->SM[ICG_SUBMODULE].VAL3 = 0xFFFF;  // no more ICG off until exposure over
    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((SH_MASK | ICG_MASK));

    pulse_loop_state = WAIT_EXPOSURE;

    // for elapsed time and exposure
    sh_cyccnt64_prev = sh_cyccnt64_now;
    sh_cyccnt64_now  = cyccnt64_now;

#ifdef DEBUG
    cyccnt_at_start_exposure = cyccnt64_now;
#endif

    break;

  case WAIT_EXPOSURE:

    /* Note that ICG has turn of SH and ICG and readout has to
       turn them back on and set state to wait_frame
     */

    if ((cyccnt64_now - sh_cyccnt64_prev) > cyccnt64_prepare_readout ) {

#ifdef DEBUG
      Serial.print("wait exp ");
      Serial.println(cyccnt64_now);
#endif

      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((SH_MASK | ICG_MASK));

      flexpwm->SM[SH_SUBMODULE].VAL2 = sh.onA_counts;    // next cycle we pulse SH,

      flexpwm->SM[ICG_SUBMODULE].VAL2 = icg.onA_counts;  // and ICG togther
```

```
        flexpwm->SM[ICG_SUBMODULE].VAL3 = icg.offA_counts;

        flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((SH_MASK | ICG_MASK));

        pulse_loop_state = IDLE;

        // for elapsed time and exposure
        sh_cyccnt64_prev = sh_cyccnt64_now;
        sh_cyccnt64_now  = cyccnt64_now;

#ifdef DEBUG
        cyccnt_at_wait_exposure = cyccnt64_now;
#endif

      }
      break;


    default:

      break;

    }

    // diagnostics
    if (!(status&CMPF_MASKA_OFF)) {
      oops_flag = true;
      Serial.printf("OOPS! pulse loop sh without off bit set 0x%04x", status);
    }

    sh_counter++;
  }

  static void pulse_loop_icg_isr()
  {
    uint16_t status;

    status = flexpwm->SM[ICG_SUBMODULE].STS;
    flexpwm->SM[ICG_SUBMODULE].STS = status;

    //Serial.print("icg_isr "); print_counters();
    if (status & CMPF_MASKA_ON) {

      //Serial.println("pulse icg maska_on");

      // prepare to restart the CNVST clock
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(CNVST_MASK);   // clear ldok
      flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN(CNVST_MASK);    // stop

      flexpwm->SM[CNVST_SUBMODULE].STS = flexpwm->SM[CNVST_SUBMODULE].STS;
      NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + CNVST_SUBMODULE);

      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(CNVST_MASK);    // set ldok
      flexpwm->SM[CNVST_SUBMODULE].CNT = 0;                // zero the submodule counter
      flexpwm->SM[CNVST_SUBMODULE].CTRL2 |= FLEXPWM_SMCTRL2_FORCE;  // force init
      read_counter = 0;                                    // zero the readout counter
    }

    else if (status & CMPF_MASKA_OFF) {

      //Serial.println("pulse icg maska_off");

      // Start the cnvst clock
      flexpwm->MCTRL |= FLEXPWM_MCTRL_RUN(CNVST_MASK);

      /* Turnoff icg and sh completely because of the interrupts,until we are done with the readout,
         or we could disalbe the onA and disable interrupts, lets keep it as simple as possible.
         Note also that they are left in the off state  at the end of the cycle
      */
      flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((SH_MASK|ICG_MASK));
      flexpwm->SM[ICG_SUBMODULE].VAL2 = 0xFFFF;
      flexpwm->SM[ICG_SUBMODULE].VAL3 = 0xFFFF;
      flexpwm->SM[SH_SUBMODULE].VAL2 = 0xFFFF;
      flexpwm->SM[SH_SUBMODULE].VAL3 = 0xFFFF;
      flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((SH_MASK|ICG_MASK));

      // here is our exposure, diff to most recent previous sh gate
      sh_cyccnt64_exposure = sh_cyccnt64_now - sh_cyccnt64_prev;

      // kluge to fix the reporting "bug"
      sh_cyccnt64_exposure += pulse_loop_period_secs * F_CPU;
```

```
    }

    // bookkeeping
    icg_counter++;
  }

  static void pulse_loop_cnvst_isr()
  {
    uint16_t status;

    status = flexpwm->SM[CNVST_SUBMODULE].STS;
    flexpwm->SM[CNVST_SUBMODULE].STS = status;

    if ((status&CMPF_MASKA_OFF)) {

#ifndef ALLINONEBOARD
        // Assert the convert pin
        SETCNVST;
        delayNanoseconds( 670 );  // 710 nanoseconds minus spi setup time
        CLEARCNVST;               // need 30 nanoseconds after this
#endif

      if (read_counter < read_counts) {

#ifdef ALLINONEBOARD
        adc->adc0->startReadFast(ANALOGPIN);
        while ( adc->adc0->isConverting() );
        *read_pointer = adc->adc0->readSingle();
#else
        *read_pointer = SPI.transfer16(0xFFFF);
        *read_pointer ^= (0x1<<15);
#endif

        // bookkeeping for the read
        read_pointer++;
        read_counter++;

        // The read is done
        if (read_counter == read_counts) {

          // ----------------------------------------
          // User supplied function, per read complete, we need this 0 indexed
          // First exposure is frame 1, frame 0 is the pre-exposure

          frames_complete = (frame_counter+1 == frame_counts);    // both of these are reset in the fill header funnction
          framesets_complete = frames_complete && (frameset_counter+1 == frameset_counts);

          if (read_callback) {
            read_callback();
          }

#ifdef DEBUG
          print_sh_debugger();
#endif
          frame_counter++;
          if (frame_counter < frame_counts) {

            // wait for start of next sh period
            if (flexpwm->MCTRL & FLEXPWM_MCTRL_RUN(SH_MASK)) {
              while (!(flexpwm->SM[SH_SUBMODULE].STS & FLEXPWM_SMSTS_RF));
              flexpwm->SM[SH_SUBMODULE].STS = FLEXPWM_SMSTS_RF;
            }

            // Setup the next state
            pulse_loop_state  = WAIT_FRAME;

            // Re-enable SH (not icg)
            flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK(SH_MASK);
            flexpwm->SM[SH_SUBMODULE].VAL2 = sh.onA_counts;
            flexpwm->SM[SH_SUBMODULE].VAL3 = sh.offA_counts;
            flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK(SH_MASK);

          }

          else if (frame_counter==frame_counts) {

            if (frames_completed_callback) {
          frames_completed_callback();
            }

```

```
            frameset_counter++;
            if (frameset_counter==frameset_counts) {

              if (framesets_completed_callback) {
                framesets_completed_callback();
              }
            }

            pulse_loop_arm();
          }

        }

      }

    }

    // diagnostics
    cnvst_counter++;

  }

  // ----------------------------------------------------------------

  bool pulse_loop_wait( float timeout_=1., float timestep_=0.01,
                        bool interruptible=false, bool verbose=false)
  {
    if (verbose) {
      Serial.println("waiting for pulse_loop_running");
    }
    return generic_wait(&pulse_loop_running, timeout_, timestep_, interruptible, verbose);
  }

  static void pulse_loop_stop()
  {
    flexpwm_stop();

    pulse_loop_armed    = false;
    pulse_loop_running  = false;
  }

  static void pulse_loop_start()
  {
    uint64_t cyccnt64_now = cycles64();

    if (pulse_loop_running) {
      //error_flag = true;
      Serial.println("Error: pulse start with flexpwm already running");
      error_flag = true;
      //return;
    }

    if (!pulse_loop_armed) {
      error_flag = true;
      Serial.println("Error: pulse loop not armed");
      return;
    }

    pulse_loop_armed    = false;  // means that we need to arm again before we can start again.
    pulse_loop_running  = true;

    pulse_loop_state = START_EXPOSURE;

    sh_cyccnt64_prev = sh_cyccnt64_now;
    sh_cyccnt64_now  = cyccnt64_now;

    flexpwm_running = true;
    flexpwm->MCTRL  |= FLEXPWM_MCTRL_RUN((CLK_MASK|SH_MASK|ICG_MASK|CNVST_MASK)); // set run clk, sh, icg
  }

  static void pulse_loop_arm()
  {
    if (mode != PULSE_LOOP) {
      Serial.println("Error: pulse_loop_arm called but not in pulse_loop mode");
      error_flag = true;
      return;
    }

    // clear ldok, everything
    flexpwm->MCTRL |= FLEXPWM_MCTRL_CLDOK((CNVST_MASK|CLK_MASK|SH_MASK|ICG_MASK));
    flexpwm_running = false;
```

```
  // stop everything
  flexpwm->MCTRL &= ~FLEXPWM_MCTRL_RUN((CNVST_MASK|CLK_MASK|SH_MASK|ICG_MASK));

  // clear all of the interrupts and the pending interrupts
  flexpwm->SM[CLK_SUBMODULE].STS = flexpwm->SM[CLK_SUBMODULE].STS;
  NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + CLK_SUBMODULE);

  flexpwm->SM[SH_SUBMODULE].STS = flexpwm->SM[SH_SUBMODULE].STS;
  NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + SH_SUBMODULE);

  flexpwm->SM[ICG_SUBMODULE].STS = flexpwm->SM[ICG_SUBMODULE].STS;
  NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + ICG_SUBMODULE);

  flexpwm->SM[CNVST_SUBMODULE].STS = flexpwm->SM[CNVST_SUBMODULE].STS;
  NVIC_CLEAR_PENDING(IRQ_FLEXPWM1_0 + CNVST_SUBMODULE);

  // Setup for SH.  CLK and CNVST are always configured
  flexpwm->SM[SH_SUBMODULE].VAL2 = sh.onA_counts;
  flexpwm->SM[SH_SUBMODULE].VAL3 = sh.offA_counts;

  // ICG is disabled, will be enabled in SH isr for the end of exposure
  flexpwm->SM[ICG_SUBMODULE].VAL2 = 0xFFFF;
  flexpwm->SM[ICG_SUBMODULE].VAL3 = 0xFFFF;

  /* ---------------------------------------------------------------------------
     Zero the counters - keep in mind we are now stopped and this is not the start.
     We leave everything stopped so that we can have an almost immediate start if we
     are triggered. The alternative would have clk and cnvst running, but then we
     would have jitter equal to one clk period while we resync. Therefore, full stop.
     ---------------------------------------------------------------------------
  */
  flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((CNVST_MASK|CLK_MASK|SH_MASK|ICG_MASK));    // set ldok

  flexpwm->SM[CLK_SUBMODULE].CNT = 0;           // zero the submodule counter
  flexpwm->SM[SH_SUBMODULE].CNT = 0;            // zero the submodule counter
  flexpwm->SM[ICG_SUBMODULE].CNT = 0;           // zero the submodule counter
  flexpwm->SM[CNVST_SUBMODULE].CNT = 0;         // zero the submodule counter

  // Force init, SH and ICG are slaved to CLK, CNVST only shares the clock
  flexpwm->SM[CLK_SUBMODULE].CTRL2   |= FLEXPWM_SMCTRL2_FORCE;  // force init
  //flexpwm->SM[SH_SUBMODULE].CTRL2    |= FLEXPWM_SMCTRL2_FORCE;  // force init
  //flexpwm->SM[ICG_SUBMODULE].CTRL2   |= FLEXPWM_SMCTRL2_FORCE;  // force init
  flexpwm->SM[CNVST_SUBMODULE].CTRL2 |= FLEXPWM_SMCTRL2_FORCE;  // force init

  // Reset pointed and disable readout until enabled by ICG isr
  read_pointer = read_buffer;
  read_counter = read_counts;

  // starting a new frameset if we are here
  frame_counter = 0;
  frames_complete = false;

  // starting a new set of framesets?
  if (frameset_counter >= frameset_counts) {
    frameset_counter = 0;
    framesets_complete = false;
  }

  // and this is where we start
  pulse_loop_armed = true;
  pulse_loop_running = false;
}


bool setup_pulse_loop(float clk_secs, float sh_secs, float sh_offset_secs,
                      float icg_secs, float icg_offset_secs, float period_secs,
                      float exposure_secs, float frame_secs, uint nframes,
                      uint16_t *buffer, unsigned int nbuffer, void (*callbackf)())
{
  char cbuffer[128] = {0};

  uint64_t u64;
  float rtmp;

  // stops all, clears ldok, disables irqs and clears mode
  stop_all();

  // clear the error flags
  clear_error_flags();

```

```
    sprintf(cbuffer,
            "#tcd1304 setup pulse, clk %.5gs sh %.5gs offset %.5gs icg offset %.5gs",
        clk_secs, sh_secs, sh_offset_secs, icg_offset_secs);
    Serial.println(cbuffer);

    // clear elapsed times
    sh_cyccnt64_start   = 0;
    sh_cyccnt64_prev    = 0;
    sh_cyccnt64_now     = 0;

    pit_cyccnt64_start   = 0;
    pit_cyccnt64_prev    = 0;
    pit_cyccnt64_now     = 0;

    trigger_cyccnt64_start   = 0;
    trigger_cyccnt64_prev    = 0;
    trigger_cyccnt64_now     = 0;

    // clear bookkeeping counters
    sh_counter    = 0;
    icg_counter   = 0;
    cnvst_counter = 0;

    // default to 1 frame and 1 frameset
    frame_counter = 0;
    frame_counts  = nframes ? nframes+1 : 10;

    frameset_counter = 0;
    frameset_counts = 1;

    // ----------------------------------------------------------------
    // Setup the master clock

    if (clk_secs < 0.25E-6) {
      Serial.println("Error: requested clk < 0.25E-6 (4MHz)");
      error_flag = true;
      return false;
    }

    if (clk_secs > 1.25E-6) {
      Serial.println("Error: requested clk > 1.25E-6 (800kHZ)");
      error_flag = true;
      return false;
    }

    clk.prescale = 0;
    clk.divider  = 1;

    clk.period_counts = ceil(clk_secs*F_BUS_ACTUAL);    // rouund up

    clk.offset        = 0.;

    clk.offA_counts   = 0;
    clk.onA_counts    = (clk.period_counts/2);
    clk.invertA       = false;

    clk.onB_counts    = 0;
    clk.offB_counts   = 0;
    clk.invertB       = false;

    clk.intena_mask    = 0;

    clk.ctrl2_mask    = PWM_CTRL2_CLOCK_MASTER;

    // ----------------------------------------------------------------
    // Setup the CNVST clock, 4 times the clock period,
    cnvst.prescale = clk.prescale;
    cnvst.divider = (1«cnvst.prescale);

    cnvst.period_counts = 4*clk.period_counts;

    cnvst.offset        = 0;

    cnvst.onA_counts    = ceil(pulse_cnvst_offset_secs*F_BUS_ACTUAL/cnvst.divider);    // delay to approx
the fourth clock
    cnvst.offA_counts   = cnvst.onA_counts + ceil(pulse_cnvst_secs*F_BUS_ACTUAL/cnvst.divider);
    cnvst.invertA       = false;

    cnvst.onB_counts    = 0;
    cnvst.offB_counts   = 0;
    cnvst.invertB       = false;
```

```

    //cnvst.ctrl2_mask    = PWM_CTRL2_CLOCK_SYNC;
    cnvst.ctrl2_mask    = PWM_CTRL2_CLOCK_MASTER; // needs to be master, because run bit is disabled when running from clk's clock.

    // ----------------------------------------------------------------
    // setup the sh, sh_offset-icg_offset is the icg-sh timing, runs once, isr has to set on,off to 0xffff
    sh.prescale = clk.prescale;
    sh.divider  = (1«sh.prescale);

    sh.period_counts  = get_next_multiple(clk.period_counts, COUNTSCEIL(period_secs));

    sh.offset         = 0.;

    sh.onA_counts     = ceil(sh_offset_secs*F_BUS_ACTUAL/sh.divider);
    sh.offA_counts    = sh.onA_counts + ceil(sh_secs*F_BUS_ACTUAL/sh.divider);
    sh.invertA        = false;

    sh.onB_counts     = 0;
    sh.offB_counts    = 0;
    sh.invertB        = false;

    sh.ctrl2_mask     = PWM_CTRL2_CLOCK_SLAVE;
    //sh.ctrl2_mask     = FLEXPWM_SMCTRL2_CLK_SEL(0x2);

    // ----------------------------------------------------------------
    // setup icg,  icg interrupt has to stop sh and icg by moving val2  to > val1
    icg.prescale = clk.prescale;
    icg.divider  = (1«icg.prescale);

    icg.period_counts = sh.period_counts;

    icg.offset        = 0.;

    icg.onA_counts    = ceil(icg_offset_secs*F_BUS_ACTUAL/icg.divider);
    icg.offA_counts   = icg.onA_counts + ceil(icg_secs*F_BUS_ACTUAL/icg.divider);
    icg.invertA       = true;

    icg.onB_counts    = 0;
    icg.offB_counts   = 0;
    icg.invertB       = false;

    icg.ctrl2_mask     = PWM_CTRL2_CLOCK_SLAVE;
    //icg.ctrl2_mask    = FLEXPWM_SMCTRL2_CLK_SEL(0x2);

    // this is used by the timing graphics software
    cnvst.offset = (((float)icg.offA_counts / F_BUS_ACTUAL) + CNVST_START_OVERHEAD)*1.E9;

    // ---------------------------------------------------------------------
    // CPU CYCCNT triggers to prepare the next state (1 1/2 SH short of the  exposure,frame)
    rtmp = SECONDS(sh.period_counts*sh.divider);
    uint64_t sh_period_u64 = (uint64_t) (rtmp*F_CPU);

#define DEBUG
#ifdef DEBUG
    Serial.printf("sh period %6g cycnt64 %lu\n", rtmp, sh_period_u64);
#endif

    u64 = (3*sh_period_u64)/2;
    Serial.printf("3/2  => %lu\n", u64);

    cyccnt64_prepare_exposure  = (uint64_t) (frame_secs*F_CPU);

#ifdef DEBUG
    Serial.printf("frame_secs %6g prepare => cycnt64 %lu", frame_secs, cyccnt64_prepare_exposure);
#endif

    cyccnt64_prepare_exposure  -= u64;

#ifdef DEBUG
    Serial.printf(" => cycnt64 %lu\n", cyccnt64_prepare_exposure);
#endif

    cyccnt64_prepare_readout   = (uint64_t) (exposure_secs*F_CPU);

#ifdef DEBUG
    Serial.printf("exposure_secs %6g prepare => cycnt64 %lu", exposure_secs, cyccnt64_prepare_readout);
#endif

    cyccnt64_prepare_readout  -= sh_period_u64/2;
    //cyccnt64_prepare_readout  -= u64;
```

```

#ifdef DEBUG
    Serial.printf(" => cycnt64 %lu\n", cyccnt64_prepare_readout);
#endif
#undef DEBUG

    // ----------------------------------------------------------------
    // needs this to check frame interval is long enough. reu
    pulse_loop_frame_min_secs  = (icg.offA_counts/F_BUS_ACTUAL)*icg.divider;    // offset before read starts
    pulse_loop_frame_min_secs += ((float)cnvst.period_counts/F_BUS_ACTUAL)*cnvst.divider * read_counts; // add read time
    pulse_loop_frame_min_secs += USBTRANSFERSECS;  // add usb transfer time

    if (frame_secs < pulse_loop_frame_min_secs) {
      Serial.printf("Error: frame is too short for readout time %.5g\n", pulse_loop_frame_min_secs);
      return false;
    }

    if (exposure_secs < SECONDS(sh.period_counts*sh.divider)) {
      Serial.println("Error: exposure is too short for specified pulse period");
      return false;
    }

    // ----------------------------------------------------------------
    Serial.println("");
    if (!print_and_check_submodule(&clk)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    if (!print_and_check_submodule(&icg)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    if (!print_and_check_submodule(&sh)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    if (!print_and_check_submodule(&cnvst)) {
      error_flag = true;
      return false;
    }

    Serial.println("");
    Serial.print("flexpwm: clearing_pulses ");
    Serial.println(sh_clearing_counts);

    // ----------------------------------------------------------------

    // these set cldok (clear ldok) and clear the run bit for each submodule
    load_submodule(&clk);
    load_submodule(&icg);
    load_submodule(&sh);
    load_submodule(&cnvst);

    // ---------------------------------------------
    // Setup the readout buffer
    read_buffer   = buffer;
    read_counts   = nbuffer;

    read_pointer  = read_buffer;
    read_counter  = read_counts; // will be set to 0 in the icg isr

    read_callback = callbackf;

    // ---------------------------------------------
    attach_isr( &sh, CMPF_MASKA_OFF, pulse_loop_sh_isr, 128);
    attach_isr( &icg, CMPF_MASKA_ON_OFF, pulse_loop_icg_isr, 64);
    attach_isr( &cnvst, CMPF_MASKA_OFF, pulse_loop_cnvst_isr, 64);

    // ---------------------------------------------
    // stuff that other routines need to know
    mode = PULSE_LOOP;
    trigger_mode = false;

```

```
    Serial.print("TCD1304 setup_pulse_loop success, min frame interval ");
    Serial.print(pulse_interframe_min_secs,6);
    Serial.println(" secs");

    // now we set load ok and force init  - clk, sh, icg
    /*  left over coode from setup_pulse()
    flexpwm->MCTRL |= FLEXPWM_MCTRL_LDOK((CLK_MASK|SH_MASK|ICG_MASK));
    flexpwm->SM[CLK_SUBMODULE].CTRL2 = FLEXPWM_SMCTRL2_FORCE;  // force out while ldok, initializes the counters
    */
    // ---------------------------------------------
    // and now we arm it, it is on a hair trigger after this
    pulse_loop_arm();

    return true;
  }

  /* ============================================================
     Periodic interrupt timer (PIT)
   */

#define PIT_CHANNEL 0  // the teensy libraries will see that it is enabled and take something else
#define PIT_FREQ 24000000.0f
#define PITSECS(a) ((a)/PIT_FREQ)
#define PITCOUNTS(a) (uint32_t)((a)*PIT_FREQ))

  inline static float pit_delay_secs = 1.E-6;
  inline static float pit_interval_secs = 0.1;

  inline static uint32_t pit_delay_counts = 0;
  inline static uint32_t pit_interval_counts = 0;
  inline static uint8_t pit_state = 0;

  inline static unsigned int pit_counts = 2;
  inline static unsigned int pit_counter = 0;

  inline static bool pit_running = false;
  inline static bool pit_begun = false;

  inline static void (*pit_callback)() = 0;


  void begin_pit() {

    // Enable PIT Clock
    CCM_CCGR1 |= CCM_CCGR1_PIT(CCM_CCGR_ON);
    PIT_MCR = 0; // Turn on PIT module

    attachInterruptVector(IRQ_PIT, pit_isr);
    NVIC_SET_PRIORITY(IRQ_PIT, 64); // High priority (lower number = higher priority)
    NVIC_ENABLE_IRQ(IRQ_PIT);

    pit_begun = true;

    Serial.println("begin_pit done");
  }

  static void pit_isr() {

    uint64_t cyccnt64_now = cycles64();

    pit_cyccnt64_prev = pit_cyccnt64_now;
    pit_cyccnt64_now  = cyccnt64_now;

    //digitalToggleFast(9);   // 8nsecs

    // Check if our specific channel triggered the interrupt
    if (IMXRT_PIT_CHANNELS[PIT_CHANNEL].TFLG & 1) {   // Test flag
      IMXRT_PIT_CHANNELS[PIT_CHANNEL].TFLG = 1;       // Clear flag - both together 300 nsecs

      //digitalToggleFast(9);

      if (pit_state == 0) {
        // --- FIRST PULSE (End of Delay) ---
        pulse_start();        // 80 nsecs

        //digitalToggleFast(9);

        // Force an immediate reload of the recurring interval value  - this takes 480 nsecs
        IMXRT_PIT_CHANNELS[PIT_CHANNEL].TCTRL = 0;
        IMXRT_PIT_CHANNELS[PIT_CHANNEL].LDVAL = pit_interval_counts;
```

```
        IMXRT_PIT_CHANNELS[PIT_CHANNEL].TCTRL = 3;                      // altogether 480nsecs

        //digitalToggleFast(9);

        pit_state = 1;
      } else {
        // --- SUBSEQUENT PULSES (Recurring Interval) ---
        pulse_start();       // 80 nsecs

        //digitalToggleFast(9);
      }
    }


  pit_counter++;

  if (pit_counter >= pit_counts) {
    IMXRT_PIT_CHANNELS[PIT_CHANNEL].TCTRL = 0;
    IMXRT_PIT_CHANNELS[PIT_CHANNEL].TFLG = 1;
    pit_running = false;
  }
}


bool pit_wait(float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
{
  if (verbose) {
    Serial.print("waiting for pit_running "); Serial.print(timeout_);
    Serial.print(" "); Serial.println(timestep_);
  }
  return generic_wait(&pit_running, timeout_, timestep_, interruptible, verbose);
}

static void pit_start()
{
  // PIT runs on 24MHz or 150MHz depending on root clock.
  // Standard Teensy 4.x is 24MHz for PIT by default.

  if (pit_running) {
    error_flag = true;
    Serial.println("Error: pit_start() but already running");
    return;
  }

  pit_cyccnt64_start = cycles64();
  pit_cyccnt64_prev  = pit_cyccnt64_start;
  pit_cyccnt64_now   = pit_cyccnt64_start;

  if (pulse_armed  && frame_counts && pit_counts) {

    oops_flag = false;
    error_flag = false;

    pit_running = true;  // do it before the start, in case isr changes it

    frame_counter = 0;

    pit_counter = 0;

    pit_state = 0;

    IMXRT_PIT_CHANNELS[PIT_CHANNEL].LDVAL = pit_delay_counts;
    IMXRT_PIT_CHANNELS[PIT_CHANNEL].TCTRL = 3;    // TEN=1 (Enable), TIE=1 (Interrupt Enable)
  }

  else {

    error_flag = true;

    Serial.print("Error: pit_start() - but not setup ");
    Serial.print(" pulse armed " ); Serial.print(pulse_armed);
    Serial.print(" frame_counts "); Serial.println(frame_counts);

    stop_pit();
  }

  //Serial.println("pit_start done");
}

static void stop_pit() {
  Serial.println("stop_pit");
```

```
    IMXRT_PIT_CHANNELS[PIT_CHANNEL].TCTRL = 0;
    IMXRT_PIT_CHANNELS[PIT_CHANNEL].TFLG = 1;
    pit_running = false;
  }


  bool setup_pit(float exposure_secs, float delay_secs=1.E-6, unsigned int ncounts=0)
  {
    // we cannot touch the run mode, only clear the trigger mode
    trigger_mode = false;

    // clear error and oops flags
    clear_error_flags();

    // reset elapsed time counters
    pit_cyccnt64_start   = 0;
    pit_cyccnt64_prev    = 0;
    pit_cyccnt64_now     = 0;

    // Check, timer is for single pulse flexpwm only
    if (mode!=PULSE) {
      Serial.println("Error: setup_pit, need to call setup_pulse() first");
      error_flag = true;
      return false;
    }

    // Check, minimum exposure is the readout and transfer time
    if (exposure_secs < pulse_interframe_min_secs) {
      Serial.printf( "Error: setup_pit exposure %.6gs is too short for this pulse configuration, need at least %.6g",
              exposure_secs,pulse_interframe_min_secs);
      error_flag = true;
      return false;
    }

    if (delay_secs < 0.95E-6) {
      Serial.printf( "Error: setup_pit delay %.6gs is too short, need at least 1usec", delay_secs);
      error_flag = true;
      return false;
    }

    /* ======================================================
       Setup timer callback, counts and period
    */
    pit_callback  = pulse_start;
    pit_counts    = ncounts ? ncounts+1 : frame_counts;
    frame_counts  = pit_counts;

    // Exposure
    pit_interval_secs = exposure_secs;
    pit_delay_secs = delay_secs;

    pit_delay_counts = (uint32_t)(pit_delay_secs * PIT_FREQ);
    pit_interval_counts = (uint32_t)(pit_interval_secs * PIT_FREQ);

    // 0 is legal, but everything expires in N+1 tick.
    if (pit_delay_counts > 0) pit_delay_counts--;
    if (pit_interval_counts > 0) pit_interval_counts--;

    pit_state = 0;

    Serial.printf("setup_pit pit_counts %d frame_counts %d delay %.6g (%d) interval %.6g (%d)\n",
                  pit_counts, frame_counts, PITSECS(pit_delay_counts+1), pit_delay_counts,
                  PITSECS(pit_interval_counts+1), pit_interval_counts);

    // ==============================================

    if (!pit_begun) {
      begin_pit();
      pit_begun = true;
    }

    // ==============================================
    // pit mode is enabled
    mode = PIT;

    Serial.println("setup_pit done");
    return true;
  }

  /* ============================================================
```

```
     Setup triggers
  */
  //#define DEBUG_TRIGGERS

  inline static bool trigger_busy = false;

  static void trigger_isr()
  {
    uint64_t cyccnt_now = cycles64();

#ifdef DEBUG_TRIGGERS
    Serial.print("interrupt "); Serial.println(trigger_counter);
    Serial.print(" t="); Serial.println( (float)(cyccnt_now-interrup_cyccnt64_start)/F_CPU, 6);
#endif

    if (trigger_busy) {
      oops_flag = true;
      Serial.println("Oops! trigger interrupt while busy");
      return;
    }
    trigger_busy = true;

    // need to do this before sending data to the host
    trigger_cyccnt64_prev = trigger_cyccnt64_now;
    trigger_cyccnt64_now  = cyccnt_now;

    // the callback should have the right counter
    trigger_counter++;

    // this might result in a data send
    if (trigger_callback) {
      trigger_callback();
    }

    // are we stopping?
    if (trigger_counter >= trigger_counts) {
      detachInterrupt(digitalPinToInterrupt(trigger_pin));
      trigger_attached = false;

#ifdef DEBUG_TRIGGERS
      Serial.println("interrupt stop");
#endif
    }

    trigger_busy = false;
  }

  static void stop_triggers()
  {
    if (trigger_attached) {
      detachInterrupt(digitalPinToInterrupt(trigger_pin));
      trigger_attached = false;
    }
  }

  bool start_triggers( )
  {
    oops_flag = false;
    error_flag = false;

    if (!trigger_mode) {
      Serial.println("start triggers but not trigger mode");
      error_flag = true;
      return false;
    }
    if (trigger_attached) {
      Serial.println("start triggers but trigger already attached");
      error_flag = true;
      return false;
    }

    Serial.printf("start_triggers %d\n",trigger_pin);

    trigger_counter = 0;

    trigger_cyccnt64_start = cycles64();
    trigger_cyccnt64_now = trigger_cyccnt64_start;
    trigger_cyccnt64_prev = trigger_cyccnt64_start;

    // attach the trigger interrupt
    trigger_attached = true;
```

```
  attachInterrupt(digitalPinToInterrupt(trigger_pin), trigger_isr, trigger_edge_mode);

  return true;
}

bool wait_trigger(float timeout_=1., float timestep_=0.01, bool interruptible=false, bool verbose=false)
{
  if (verbose) {
    Serial.println("waiting for trigger attached");
  }

  return generic_wait(&trigger_attached,timeout_,timestep_,interruptible,verbose);
}


bool setup_triggers(unsigned int ncounts=0)
{
  // we cannot clear the operating mode, only the trigger mode.
  trigger_mode = false;

  // but we must clear the error flags at this point
  clear_error_flags();

  switch(mode) {
  case PULSE:
    Serial.println("trigger callback = pulse_start()");
    trigger_callback = pulse_start;
    trigger_counts   = ncounts ? ncounts : frame_counts;
    if (trigger_counts) {
      frame_counts = trigger_counts;
      trigger_mode = true;
    }
    else {
      Serial.println("Error: setup_interrupt pulse, but no frame_counts");
      error_flag = true;
    }
    break;
  case PIT:
    Serial.println("trigger callback = pit_start()");
    trigger_callback = pit_start;
    trigger_counts   = ncounts ? ncounts : 1;
    trigger_mode = true;
    break;
  case PULSE_LOOP:
    Serial.println("trigger callback = pulse_loop_start()");
    trigger_callback = pulse_loop_start;
    trigger_counts   = ncounts ? ncounts : 1;
    trigger_mode = true;
    break;
  default:
    Serial.println("Error: setup_interrupt, but not configured");
    error_flag = true;
    break;
  }

  return trigger_mode;
}


  /*
  IMXRT_FLEXPWM_t *q = p->flexpwm;
  unsigned int submod = p->submod;
  uint16_t mask = p->mask;
  */

void set_clock_master(uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)
{
  q->SM[submod].CTRL2 &= 0xFFF0;
  q->SM[submod].CTRL2 |= PWM_CTRL2_CLOCK_MASTER;
}

void set_clock_master(SubModule *p)
{
  set_clock_master(p->submod, p->flexpwm);
}

// ------------------------------------------------------------
void set_clock_slave(uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm)
{
  q->SM[submod].CTRL2 &= 0xFFF0;
  q->SM[submod].CTRL2 |= PWM_CTRL2_CLOCK_SLAVE;
```

```
}

void set_clock_slave(SubModule *p)
{
  set_clock_slave(p->submod, p->flexpwm);
}

// ------------------------------------------------------------
void set_clock_sync(uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm){
  q->SM[submod].CTRL2 &= 0xFFF0;
  q->SM[submod].CTRL2 |= PWM_CTRL2_CLOCK_SYNC;
}

void set_clock_sync(SubModule *p)
{
  set_clock_sync(p->submod, p->flexpwm);
}

// ------------------------------------------------------------
void clear_ldok(uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm) {
  q->MCTRL |= FLEXPWM_MCTRL_CLDOK(mask);
}

void set_ldok(uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm) {
  q->MCTRL |= FLEXPWM_MCTRL_LDOK(mask);
}

// ------------------------------------------------------------
void clear_run(uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm) {
  q->MCTRL &= FLEXPWM_MCTRL_RUN(~mask);
}
void set_run(uint8_t mask, IMXRT_FLEXPWM_t *q=flexpwm) {
  q->MCTRL |= FLEXPWM_MCTRL_RUN(mask);
}

// ------------------------------------------------------------
void force(uint8_t submod, IMXRT_FLEXPWM_t *q=flexpwm) {
  if(submod<4) {
    q->SM[submod].CTRL2 = FLEXPWM_SMCTRL2_FORCE;
  }
}

// ------------------------------------------------------------
void set_prescale(uint8_t submod, uint8_t prescale=0, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].CTRL = FLEXPWM_SMCTRL_FULL | FLEXPWM_SMCTRL_PRSC(prescale);
  }
}

void set_init(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].INIT = value;
  }
}

void set_cnt(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].CNT = value;
  }
}

void set_val0(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].VAL0 = value;
  }
}

void set_val1(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].VAL1 = value;
  }
}

void set_val2(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
    q->SM[submod].VAL2 = value;
  }
}

void set_val3(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
  if (submod<4) {
```

```
      q->SM[submod].VAL3 = value;
    }
  }

  void set_val4(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
    if (submod<4) {
      q->SM[submod].VAL4 = value;
    }
  }

  void set_val5(uint8_t submod,uint16_t value, IMXRT_FLEXPWM_t *q=flexpwm) {
    if (submod<4) {
      q->SM[submod].VAL5 = value;
    }
  }
  // ------------------------------------------------------------
  uint16_t set_outen( uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm) {
    q->OUTEN = mask16;
    return q->OUTEN;
  }

  uint16_t set_outen_on( uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm) {
    q->OUTEN |= mask16;
    return q->OUTEN;
  }

  uint16_t set_outen_off( uint16_t mask16, IMXRT_FLEXPWM_t *q=flexpwm) {
    q->OUTEN &= ~mask16;
    return q->OUTEN;
  }

  uint16_t set_outenA_on( uint8_t mask8, IMXRT_FLEXPWM_t *q=flexpwm) {
    q->OUTEN |= FLEXPWM_OUTEN_PWMA_EN(mask8);
    return q->OUTEN;
  }



  // ============================================================

  void printbits16_(uint16_t u16, int bits)
  {
    while(bits> 0) {
      bits--;
      Serial.print((u16»bits)&1);
    }
  }

  void register_dump(IMXRT_FLEXPWM_t * const  q=flexpwm, uint8_t mask=0xFF)
  {
    uint16_t u16;

    Serial.print("flexpwm ");
    Serial.println((unsigned int)q, HEX);
    u16 = q->MCTRL;
    Serial.print("MCTRL "); printbits16_(u16,16); // Serial.print(u16,BIN);
    Serial.print(" IPOL "); printbits16_((u16»12),4); // Serial.print((u16»12)&0XF,BIN);
    Serial.print(" RUN "); printbits16_((u16»8),4); // Serial.print((u16»8)&0XF,BIN);
    Serial.print(" LDOK "); printbits16_(u16,4); // Serial.print(u16&0XF,BIN);
    Serial.println("");

    u16 = q->MCTRL2;
    Serial.print("MCTRL2 MONPLL "); Serial.print(u16&0XF,BIN);
    Serial.println("");

    u16 = q->OUTEN;
    Serial.print("OUTEN "); printbits16_(u16,16); // Serial.print(u16,BIN);
    Serial.print(" PWMA "); printbits16_((u16»8),4); // Serial.print((u16>8)&0xF,BIN);
    Serial.print(" PWMB "); printbits16_((u16»4),4); // Serial.print((u16>4)&0xF,BIN);
    Serial.print(" PWMX "); printbits16_(u16,4); // Serial.print(u16&0xF,BIN);
    Serial.println("");

    for (uint8_t submodule = 0; submodule<4; submodule++) {

      if (mask & (1«submodule)) {

        Serial.println("#-------------------------------------");
        Serial.print("Submodule "); Serial.print(submodule);
        Serial.println("");

        u16 = q->SM[submodule].CNT;
```

```
        Serial.print("CNT "); Serial.println(u16);

        u16 = q->SM[submodule].INIT;
        Serial.print("INIT "); Serial.println(u16);

        u16 = q->SM[submodule].CTRL2;
        Serial.print("CTRL2 "); printbits16_(u16,16); // Serial.print(u16,BIN);
        Serial.print(" DBGEN "); Serial.print((u16»15));
        Serial.print(" WAITEN "); Serial.print(((u16»14)&1));
        Serial.print(" INDEP "); Serial.print(((u16»13)&1));
        Serial.print(" PWM23_INIT "); Serial.print(((u16»12)&1));
        Serial.print(" PWM45_INIT "); Serial.print(((u16»11)&1));
        Serial.print(" PWMX_INIT "); Serial.print(((u16»10)&1));
        Serial.print(" INIT_SEL "); Serial.print(((u16»8)&3));
        Serial.print(" FRCEN "); Serial.print(((u16»7)&1));
        Serial.print(" FORCE "); Serial.print(((u16»6)&1));
        Serial.print(" FORCE_SEL "); Serial.print(((u16»3)&7));
        Serial.print(" RELOAD_SEL "); Serial.print(((u16»2)&1));
        Serial.print(" CLK_SEL "); Serial.print((u16&3));
        Serial.println("");

        u16 = q->SM[submodule].CTRL;
        Serial.print("CTRL "); printbits16_(u16,16); // Serial.print(u16,BIN);
        Serial.print(" LDFQ "); Serial.print(((u16»12)&0xF));
        Serial.print(" HALF "); Serial.print(((u16»11)&1));
        Serial.print(" FULL "); Serial.print(((u16»10)&1));
        Serial.print(" DT "); Serial.print(((u16»9)&0x1));Serial.print(((u16»8)&0x1));
        Serial.print(" COMPMODE "); Serial.print(((u16»10)&1));
        Serial.print(" PRSC "); Serial.print(((u16»4)&0x7));
        Serial.print(" SPLIT "); Serial.print(((u16»3)&1));
        Serial.print(" LDMOD "); Serial.print(((u16»2)&1));
        Serial.print(" DBLX "); Serial.print(((u16»1)&1));
        Serial.print(" DBLEN "); Serial.print((u16&1));
        Serial.println("");

        u16 = q->SM[submodule].OCTRL;
        Serial.print("OCTRL "); printbits16_(u16,16); // Serial.print(u16,BIN);
        Serial.print(" PWMA_IN "); Serial.print(((u16»15)&1));
        Serial.print(" PWMB_IN "); Serial.print(((u16»14)&1));
        Serial.print(" PWMX_IN "); Serial.print(((u16»13)&1));
        Serial.print(" POLA "); Serial.print((u16»10)&1);
        Serial.print(" POLB "); Serial.print((u16»9)&1);
        Serial.print(" POLX "); Serial.print((u16»8)&1);
        Serial.print(" PWMA_FS "); Serial.print((u16»4)&3);
        Serial.print(" PWMB_FS "); Serial.print((u16»2)&3);
        Serial.print(" PWMX_FS "); Serial.print(u16&3);
        Serial.println("");

        u16 = q->SM[submodule].VAL0;
        Serial.print("VAL0 "); Serial.print(u16);

        u16 = q->SM[submodule].VAL1;
        Serial.print(" VAL1 "); Serial.print(u16);

        u16 = q->SM[submodule].VAL2;
        Serial.print(" VAL2 "); Serial.print(u16);

        u16 = q->SM[submodule].VAL3;
        Serial.print(" VAL3 "); Serial.print(u16);

        u16 = q->SM[submodule].VAL4;
        Serial.print(" VAL4 "); Serial.print(u16);

        u16 = q->SM[submodule].VAL5;
        Serial.print(" VAL5 "); Serial.print(u16);
        Serial.println("");

        u16 = q->SM[submodule].STS;
        Serial.print("STS "); printbits16_(u16,16); // Serial.print(u16,BIN);
        Serial.print(" RUF "); Serial.print(((u16»14)&0x1));
        Serial.print(" REF "); Serial.print(((u16»13)&0x1));
        Serial.print(" RF "); Serial.print(((u16»12)&0x1));
        Serial.print(" CFA1 "); Serial.print(((u16»11)&0x1));
        Serial.print(" CFA0 "); Serial.print(((u16»10)&0x1));
        Serial.print(" CFB1 "); Serial.print(((u16»9)&0x1));
        Serial.print(" CFB0 "); Serial.print(((u16»8)&0x1));
        Serial.print(" CFX1 "); Serial.print(((u16»7)&0x1));
        Serial.print(" CFX0 "); Serial.print(((u16»6)&0x1));
        Serial.print(" CMPF "); printbits16_(u16,6); // Serial.print(u16&0x3F,BIN);
        Serial.println("");

```

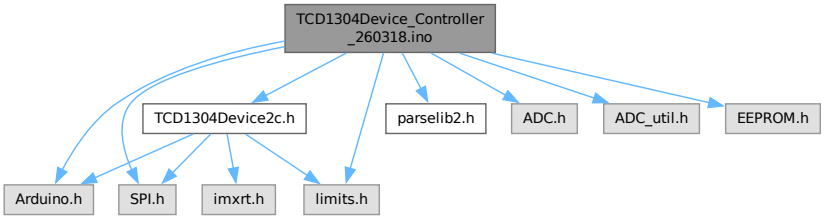
```
        u16 = q->SM[submodule].INTEN;
        Serial.print("INTEN "); printbits16_(u16,16); // Serial.print(u16,BIN);
        Serial.print(" REIE "); Serial.print(((u16»13)&0x1));
        Serial.print(" RIE "); Serial.print(((u16»12)&0x1));
        Serial.print(" CA1IE "); Serial.print(((u16»11)&0x1));
        Serial.print(" CA0IE "); Serial.print(((u16»10)&0x1));
        Serial.print(" CB1IE "); Serial.print(((u16»9)&0x1));
        Serial.print(" CB0IE "); Serial.print(((u16»8)&0x1));
        Serial.print(" CX1IE "); Serial.print(((u16»7)&0x1));
        Serial.print(" CX0IE "); Serial.print(((u16»6)&0x1));
        Serial.print(" CMPIE "); printbits16_(u16,6); // Serial.print(u16&0x3F,BIN);
        Serial.println("");
      }
    }
  }

  // ----------------------------------------
  static void setup_digital_pins()
  {
    pinMode(TRIGGER_PIN, INPUT);

    pinMode(BUSY_PIN, OUTPUT);
    digitalWriteFast(BUSY_PIN, BUSY_PIN_DEFAULT);

    pinMode(SYNC_PIN, OUTPUT);
    digitalWriteFast(SYNC_PIN, SYNC_PIN_DEFAULT);

    // Clock
    pinMode(CLK_PIN,  OUTPUT);
    digitalWriteFast(CLK_PIN, LOW);

    pinMode(CLK_MONITOR_PIN, INPUT);

    // SH gate
    pinMode(SH_PIN, OUTPUT);
    digitalWriteFast(SH_PIN, LOW);

    // Integratinon Clear Gate
    pinMode(ICG_PIN, OUTPUT);
    digitalWriteFast(ICG_PIN, HIGH);

    // CNVST to ADC
    pinMode(CNVST_PIN, OUTPUT);
    digitalWriteFast(CNVST_PIN, LOW);

    // clear the mode
    clear_mode();
  }

  static void close()
  {
    // stop both flexpwms, disable interrrupts and reset clear and busy pins
    stop_all();

    // Reactivation as digital i/o pins
    setup_digital_pins();

  }

};

#endif
```

5.7 TCD1304Device_Controller_260318.ino File Reference

```
#include "Arduino.h"
#include "TCD1304Device2c.h"
#include "parselib2.h"
#include <SPI.h>
#include <limits.h>
#include <ADC.h>
#include <ADC_util.h>
```

```
#include <EEPROM.h>
```

Include dependency graph for TCD1304Device_Controller_260318.ino:



Classes

- struct usb_string_descriptor_struct_manufacturer
- struct usb_string_descriptor_struct_product
- struct usb_string_descriptor_struct_serial_number

Macros

- #define DRMCNELSONLAB
- #define CONTROLLER_OCPIN 23
- #define CONTROLLER_CSPIN 10
- #define CONTROLLER_CSPIN2 9
- #define CYCLES_PER_USEC (F_CPU / 1000000)
- #define thisMANUFACTURER_NAME {'D','R','M','C','N','E','L','S','O','N','L','A','B' }
- #define thisMANUFACTURER_NAME_LEN 13
- #define thisPRODUCT_NAME {'T','C','D','1','3','0','4','S','P','I'}
- #define thisPRODUCT_NAME_LEN 10
- #define thisPRODUCT_SERIAL_NUMBER { 'S','N','0','0','0','0','0','0','0','0','0','1' }
- #define thisPRODUCT_SERIAL_NUMBER_LEN 12
- #define RCVLEN 256
- #define SNDLEN 256
- #define IMR_INDEX 5
- #define ISR_INDEX 6
- #define NADC_CHANNELS 4
- #define ADC_RESOLUTION (3.3/4096.)
- #define EEPROM_SIZE 1080
- #define EEPROM_ID_ADDR 0
- #define EEPROM_ID_LEN 64
- #define EEPROM_COEFF_ADDR 64
- #define EEPROM_NCOEFFS 4
- #define EEPROM_COEFF_LEN (EEPROM_NCOEFFS ∗ sizeof(float))
- #define EEPROM_UNITS_ADDR (EEPROM_COEFF_ADDR + EEPROM_COEFF_LEN)
- #define EEPROM_NUNITS 8
- #define EEPROM_UNITS_LEN EEPROM_NUNITS
- #define NBUFFERS 32
- #define BINARY 0
- #define ASCII 1

Functions

- SPISettings spi_settings (30000000, MSBFIRST, SPI_MODE0)
- float tempmonGetTemp (void)
- void blink ()
- void pulsePin (unsigned int pin, unsigned int usecs)
- void ocpinRead ()
- void ocpinISR ()
- void ocpinAttach ()
- void ocpinDetach ()
- void resumePinInterrupts (uint8_t pin)
- void disablePinInterrupts (uint8_t pin)
- uint64_t cycles64 ()
- int fastAnalogRead (uint8_t pin)
- void sendADCs (unsigned int averages)
- void sendChipTemperature (unsigned int averages)
- void eeread (unsigned int address, int nbytes, char ∗p)
- void eewrite (unsigned int address, int nbytes, char ∗p)
- void eereadUntil (unsigned int address, int nbytes, char ∗p)
- void eewriteUntil (unsigned int address, int nbytes, char ∗p)
- void eeErase (unsigned int address, int nbytes)
- void eraseIdentifier ()
- void readIdentifier (char ∗p)
- void storeIdentifier (char ∗p)
- void printIdentifier ()
- void eraseUnits ()
- void readUnits (char ∗p)
- void storeUnits (char ∗p)
- void printUnits ()
- void eraseCoefficients ()
- void readCoefficients (float ∗vals)
- void storeCoefficients (float ∗vals)
- void printCoefficients ()
- void clock_isr ()
- void clock_stop ()
- void clock_start ()
- bool clock_setup (float secs, unsigned int ncounts=0)
- uint8_t calculateCRC (uint16_t ∗bufferp)
- uint32_t sumData (uint16_t ∗bufferp)
- void sendBuffer_Formatted (uint16_t ∗bp)
- void sendBuffer_Binary (uint16_t ∗bp)
- void sendDataCRC (uint16_t ∗bp)
- void sendDataSum (uint16_t ∗bp)
- void sendData (uint16_t ∗bp)
- void sendDataReady ()
- void sendTestData (uint16_t ∗bp)
- void send_header ()
- bool send_frame (TCD1304Device ::Frame_Header ∗p)
- void send_frames ()
- void frame_callback ()
- bool strPin (char ∗pc, uint8_t ∗pin, char ∗∗next)

- void printTriggerSetup ()
- void strWaitParameters (char ∗pc, float ∗timeout, float ∗timestep, bool ∗interruptible, bool ∗verbose, char ∗∗next=0)
- bool strTrigger (char ∗pc, char ∗∗next)
- bool strTimings (char ∗s, char ∗∗next=0)
- bool strStartNoStart (char ∗pc, bool ∗start, char ∗∗next=0)
- bool strSubModule (char ∗s, char ∗∗next=0)
- void help (const char ∗key)
- void setup ()
- void loop ()

Variables

- ADC ∗ adc = new ADC( )
- TCD1304Device tcd1304device = TCD1304Device( )
- usb_string_descriptor_struct_manufacturer usb_string_manufacturer_name
- usb_string_descriptor_struct_product usb_string_product_name
- usb_string_descriptor_struct_serial_number usb_string_serial_number
- char rcvbuffer [RCVLEN]
- uint16_t nrcvbuf = 0
- char sndbuffer [SNDLEN]
- const char versionstr [ ] = "TCD1304Device vers 0.6 " __DATE__
- const char authorstr [ ] = "Patents Pending and (c) 2020, 2023, 2025, 2026 by Mitchell C. Nelson, Ph.D. "
- const char srcfilestr [ ] = __FILE__
- const char sensorstr [ ] = "TCD1304"
- const int syncPin = SYNC_PIN
- const int busyPin = BUSY_PIN
- const int triggerPin = TRIGGER_PIN
- const int fMPin = CLK_PIN
- const int ICGPin = ICG_PIN
- const int SHPin = SH_PIN
- const int CNVSTPin = CNVST_PIN
- const int SDIPin = 11
- const int SDOPin = 12
- const int CLKPin = 13
- const int analogPin = A0
- bool ocpinstate = false
- bool ocpinattached = false
- elapsedMicros elapsed_usecs
- elapsedMicros diagnostic_usecs
- bool diagnostics = false
- uint16_t adc_data [NADC_CHANNELS] = { 0 }
- unsigned int adc_averages = 0
- unsigned int chipTemp_averages = 0
- IntervalTimer clock_timer
- void(∗ clock_callback)() = nullptr
- unsigned int clock_counts = 0
- unsigned int clock_counter = 0
- float clock_usecs = 0
- bool clock_mode = false

- unsigned char crctable [256]
- TCD1304Device::Frame_Header frame_header_ring [NBUFFERS] = {0}
- TCD1304Device::Frame_Header ∗ framep = &frame_header_ring[0]
- uint16_t buffer_ring [NBUFFERS][NREADOUT] = { 0 }
- uint16_t ∗ bufferp = buffer_ring[0]
- unsigned int frame_index = 0
- unsigned int buffer_index = 0
- unsigned int frame_send_index = 0
- unsigned int dataformat = BINARY
- unsigned int counter = 0
- bool data_async = true
- bool crc_enable = false
- bool sum_enable = false

5.7.1 Macro Definition Documentation

5.7.1.1 ADC_RESOLUTION

```
#define ADC_RESOLUTION (3.3/4096.)
```

5.7.1.2 ASCII

```
#define ASCII 1
```

5.7.1.3 BINARY

```
#define BINARY 0
```

5.7.1.4 CONTROLLER_CSPIN

```
#define CONTROLLER_CSPIN 10
```

5.7.1.5 CONTROLLER_CSPIN2

```
#define CONTROLLER_CSPIN2 9
```

5.7.1.6 CONTROLLER_OCPIN

```
#define CONTROLLER_OCPIN 23
```

5.7.1.7 CYCLES_PER_USEC

```
#define CYCLES_PER_USEC (F_CPU / 1000000)
```

5.7.1.8 DRMCNELSONLAB

```
#define DRMCNELSONLAB
```

5.7.1.9 EEPROM_COEFF_ADDR

```
#define EEPROM_COEFF_ADDR 64
```

5.7.1.10 EEPROM_COEFF_LEN

```
#define EEPROM_COEFF_LEN (EEPROM_NCOEFFS * sizeof(float))
```

5.7.1.11 EEPROM_ID_ADDR

```
#define EEPROM_ID_ADDR 0
```

5.7.1.12 EEPROM_ID_LEN

```
#define EEPROM_ID_LEN 64
```

5.7.1.13 EEPROM_NCOEFFS

```
#define EEPROM_NCOEFFS 4
```

5.7.1.14 EEPROM_NUNITS

```
#define EEPROM_NUNITS 8
```

5.7.1.15 EEPROM_SIZE

```
#define EEPROM_SIZE 1080
```

5.7.1.16 EEPROM_UNITS_ADDR

```
#define EEPROM_UNITS_ADDR (EEPROM_COEFF_ADDR + EEPROM_COEFF_LEN)
```

5.7.1.17 EEPROM_UNITS_LEN

```
#define EEPROM_UNITS_LEN EEPROM_NUNITS
```

5.7.1.18 IMR_INDEX

```
#define IMR_INDEX 5
```

5.7.1.19 ISR_INDEX

```
#define ISR_INDEX 6
```

5.7.1.20 NADC_CHANNELS

```
#define NADC_CHANNELS 4
```

5.7.1.21 NBUFFERS

```
#define NBUFFERS 32
```

5.7.1.22 RCVLEN

```
#define RCVLEN 256
```

5.7.1.23 SNDLEN

```
#define SNDLEN 256
```

5.7.1.24 thisMANUFACTURER_NAME

```
#define thisMANUFACTURER_NAME {'D','R','M','C','N','E','L','S','O','N','L','A','B' }
```

5.7.1.25 thisMANUFACTURER_NAME_LEN

```
#define thisMANUFACTURER_NAME_LEN 13
```

5.7.1.26 thisPRODUCT_NAME

```
#define thisPRODUCT_NAME {'T','C','D','1','3','0','4','S','P','I'}
```

5.7.1.27 thisPRODUCT_NAME_LEN

```
#define thisPRODUCT_NAME_LEN 10
```

5.7.1.28 thisPRODUCT_SERIAL_NUMBER

```
#define thisPRODUCT_SERIAL_NUMBER { 'S','N','0','0','0','0','0','0','0','0','0','1' }
```

5.7.1.29 thisPRODUCT_SERIAL_NUMBER_LEN

```
#define thisPRODUCT_SERIAL_NUMBER_LEN 12
```

5.7.2 Function Documentation

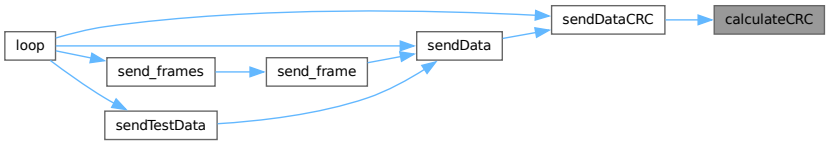
5.7.2.1 blink()

```
void blink ()
```

5.7.2.2 calculateCRC()

```
uint8_t calculateCRC (
            uint16_t * bufferp)  [inline]
```

Here is the caller graph for this function:



5.7.2.3 clock_isr()

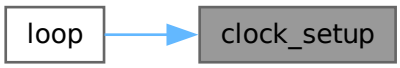
```
void clock_isr ()
```

Here is the caller graph for this function:

5.7.2.4 clock_setup()

```
bool clock_setup (
            float secs,
            unsigned int ncounts = 0)
```

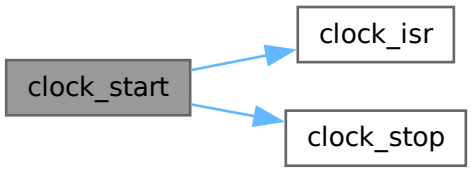
Here is the caller graph for this function:



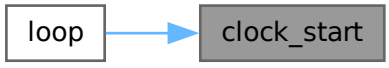
5.7.2.5 clock_start()

```
void clock_start ()
```

Here is the call graph for this function:



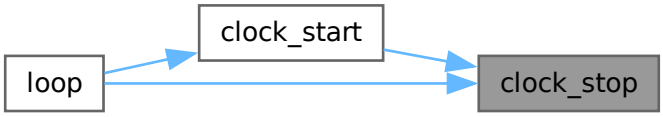
Here is the caller graph for this function:



5.7.2.6 clock_stop()

```
void clock_stop ()
```

Here is the caller graph for this function:



5.7.2.7 cycles64()

```
uint64_t cycles64 ()
```

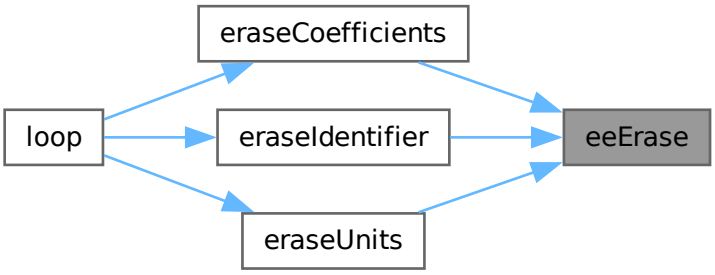
5.7.2.8 disablePinInterrupts()

```
void disablePinInterrupts (
            uint8_t pin)
```

5.7.2.9 eeErase()

```
void eeErase (
            unsigned int address,
            int nbytes)
```

Here is the caller graph for this function:



5.7.2.10 eeread()

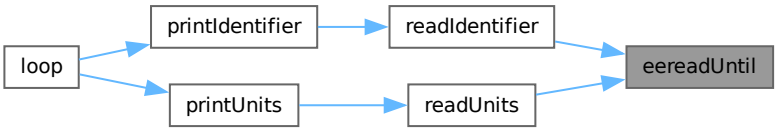
```
void eeread (
            unsigned int address,
            int nbytes,
            char * p)
```

Here is the caller graph for this function:

5.7.2.11 eereadUntil()

```
void eereadUntil (
            unsigned int address,
            int nbytes,
            char * p)
```

Here is the caller graph for this function:



5.7.2.12 eewrite()

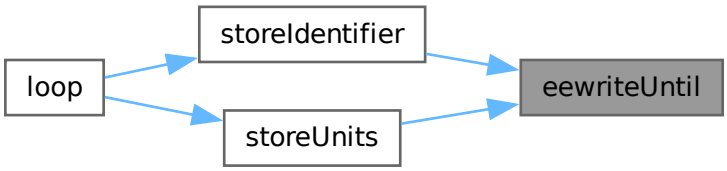
```
void eewrite (
            unsigned int address,
            int nbytes,
            char * p)
```

Here is the caller graph for this function:

5.7.2.13 eewriteUntil()

```
void eewriteUntil (
            unsigned int address,
            int nbytes,
            char * p)
```

Here is the caller graph for this function:



5.7.2.14 eraseCoefficients()

```
void eraseCoefficients ()
```

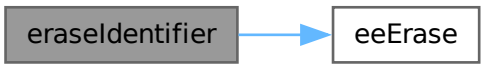
Here is the call graph for this function:

Here is the caller graph for this function:

5.7.2.15 eraseIdentifier()

```
void eraseIdentifier ()
```

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.2.16 eraseUnits()

```
void eraseUnits ()
```

Here is the call graph for this function:

Here is the caller graph for this function:

5.7.2.17 fastAnalogRead()

```
int fastAnalogRead (
            uint8_t pin)  [inline]
```

Here is the caller graph for this function:

5.7.2.18 frame_callback()

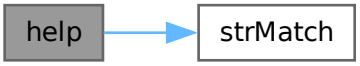
```
void frame_callback ()
```

Here is the caller graph for this function:

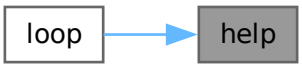
5.7.2.19 help()

```
void help (
            const char * key)
```

Here is the call graph for this function:



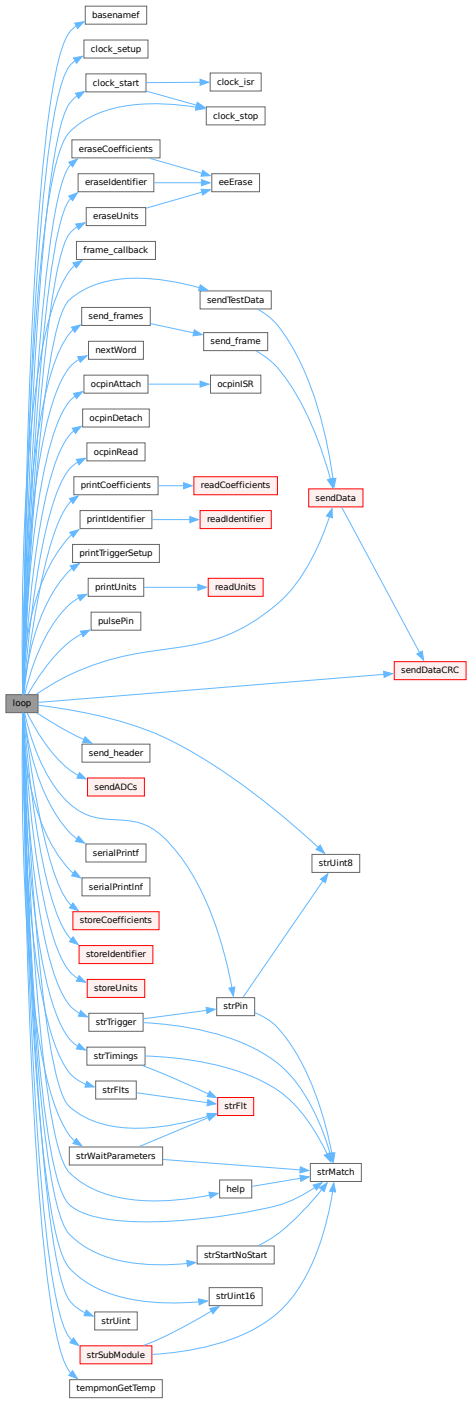
Here is the caller graph for this function:



5.7.2.20 loop()

```
void loop ()
```

Here is the call graph for this function:

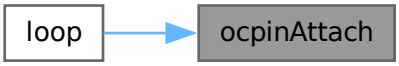


5.7.2.21 ocpinAttach()

```
void ocpinAttach ()
```

Here is the call graph for this function:

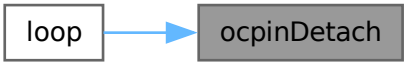
Here is the caller graph for this function:



5.7.2.22 ocpinDetach()

```
void ocpinDetach ()
```

Here is the caller graph for this function:



5.7.2.23 ocpinISR()

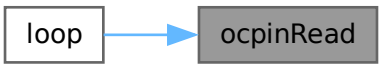
```
void ocpinISR ()
```

Here is the caller graph for this function:

5.7.2.24 ocpinRead()

```
void ocpinRead ()
```

Here is the caller graph for this function:



5.7.2.25 printCoefficients()

```
void printCoefficients ()
```

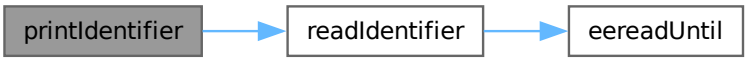
Here is the call graph for this function:

Here is the caller graph for this function:

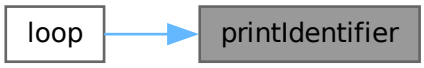
5.7.2.26 printIdentifier()

```
void printIdentifier ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.27 printTriggerSetup()

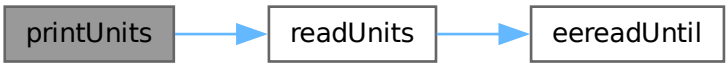
```
void printTriggerSetup ()
```

Here is the caller graph for this function:

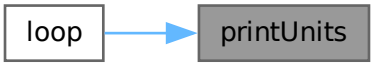
5.7.2.28 printUnits()

```
void printUnits ()
```

Here is the call graph for this function:



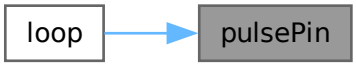
Here is the caller graph for this function:



5.7.2.29 pulsePin()

```
void pulsePin (
            unsigned int pin,
            unsigned int usecs)
```

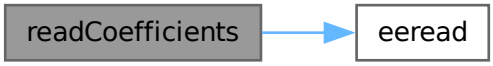
Here is the caller graph for this function:



5.7.2.30 readCoefficients()

```
void readCoefficients (
            float * vals)
```

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.2.31 readIdentifier()

```
void readIdentifier (
            char * p)
```

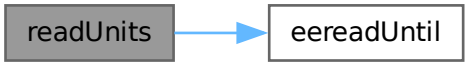
Here is the call graph for this function:

Here is the caller graph for this function:

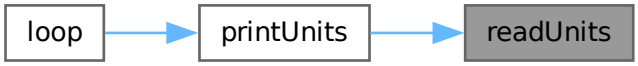
5.7.2.32 readUnits()

```
void readUnits (
            char * p)
```

Here is the call graph for this function:



Here is the caller graph for this function:



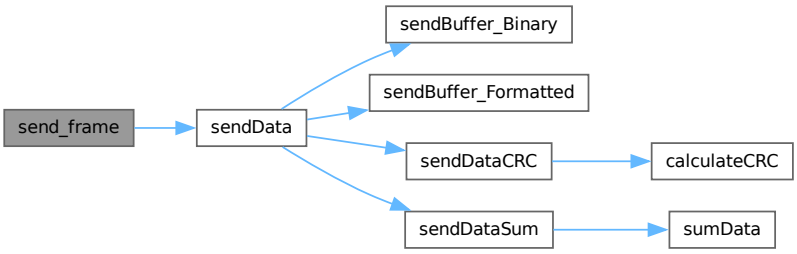
5.7.2.33 resumePinInterrupts()

```
void resumePinInterrupts (
            uint8_t pin)
```

5.7.2.34 send_frame()

```
bool send_frame (
            TCD1304Device ::Frame_Header * p)
```

Here is the call graph for this function:

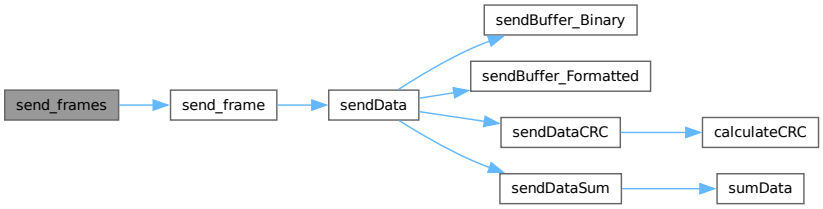


Here is the caller graph for this function:

5.7.2.35 send_frames()

```
void send_frames ()
```

Here is the call graph for this function:



Here is the caller graph for this function:

5.7.2.36 send_header()

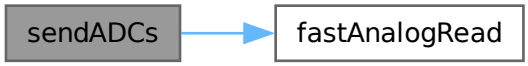
```
void send_header ()
```

Here is the caller graph for this function:

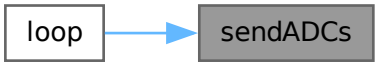
5.7.2.37 sendADCs()

```
void sendADCs (
            unsigned int averages)
```

Here is the call graph for this function:



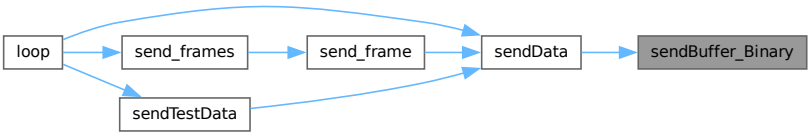
Here is the caller graph for this function:



5.7.2.38 sendBuffer_Binary()

```
void sendBuffer_Binary (
            uint16_t * bp)
```

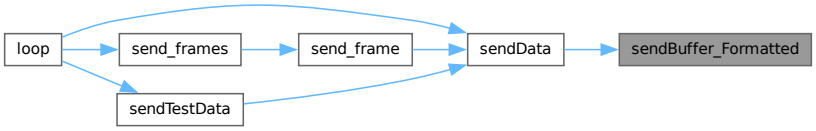
Here is the caller graph for this function:



5.7.2.39 sendBuffer_Formatted()

```
void sendBuffer_Formatted (
            uint16_t * bp)
```

Here is the caller graph for this function:



5.7.2.40 sendChipTemperature()

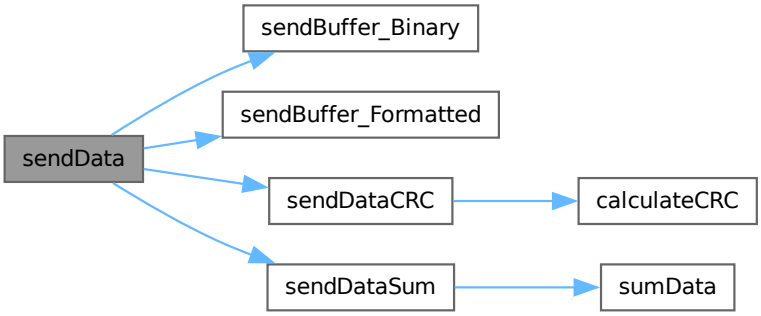
```
void sendChipTemperature (
            unsigned int averages)
```

Here is the call graph for this function:

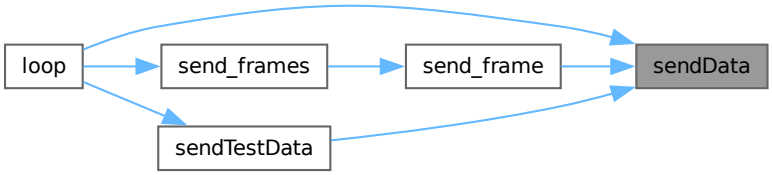
5.7.2.41 sendData()

```
void sendData (
            uint16_t * bp)
```

Here is the call graph for this function:



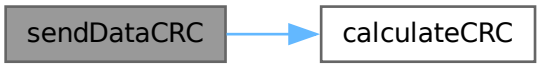
Here is the caller graph for this function:



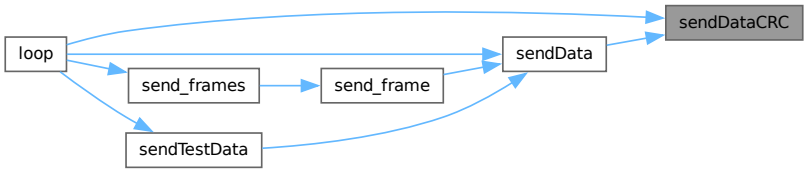
5.7.2.42 sendDataCRC()

```
void sendDataCRC (
            uint16_t * bp)  [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.7.2.43 sendDataReady()

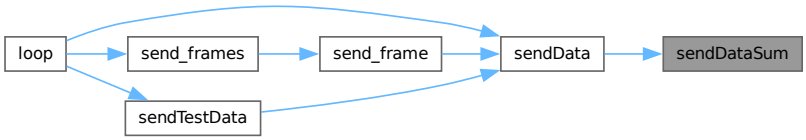
```
void sendDataReady ()
```

5.7.2.44 sendDataSum()

```
void sendDataSum (
            uint16_t * bp)  [inline]
```

Here is the call graph for this function:

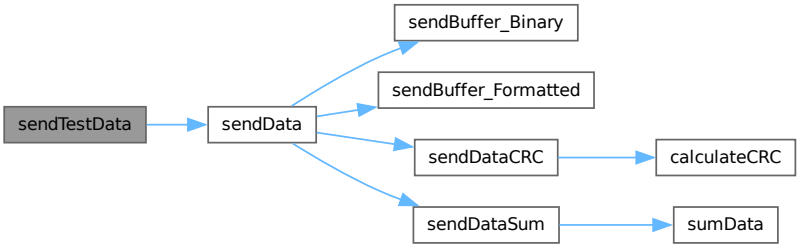
Here is the caller graph for this function:



5.7.2.45 sendTestData()

```
void sendTestData (
            uint16_t * bp)
```

Here is the call graph for this function:

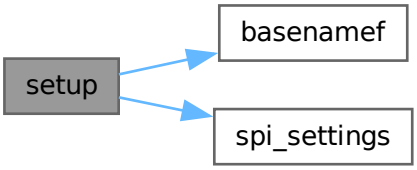


Here is the caller graph for this function:

5.7.2.46 setup()

```
void setup ()
```

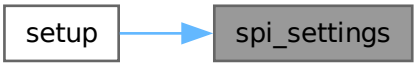
Here is the call graph for this function:



5.7.2.47 spi_settings()

```
SPISettings spi_settings (
            30000000 ,
            MSBFIRST ,
            SPI_MODE0 )
```

Here is the caller graph for this function:



5.7.2.48 storeCoefficients()

```
void storeCoefficients (
            float * vals)
```

Here is the call graph for this function:

Here is the caller graph for this function:

5.7.2.49 storeIdentifier()

```
void storeIdentifier (
            char * p)
```

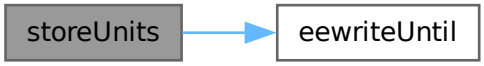
Here is the call graph for this function:

Here is the caller graph for this function:

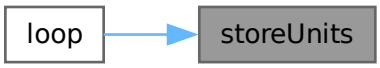
5.7.2.50 storeUnits()

```
void storeUnits (
            char * p)
```

Here is the call graph for this function:



Here is the caller graph for this function:

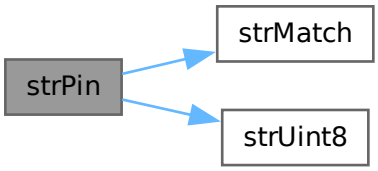


5.7.2.51 strPin()

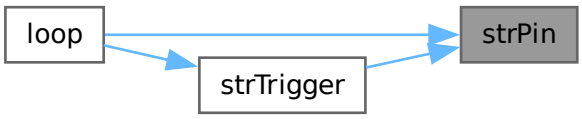
```
bool strPin (
            char * pc,
```

```
            uint8_t * pin,
            char ** next)
```

Here is the call graph for this function:



Here is the caller graph for this function:

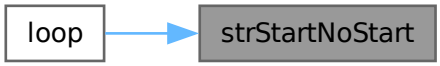


5.7.2.52 strStartNoStart()

```
bool strStartNoStart (
            char * pc,
            bool * start,
            char ** next = 0)
```

Here is the call graph for this function:

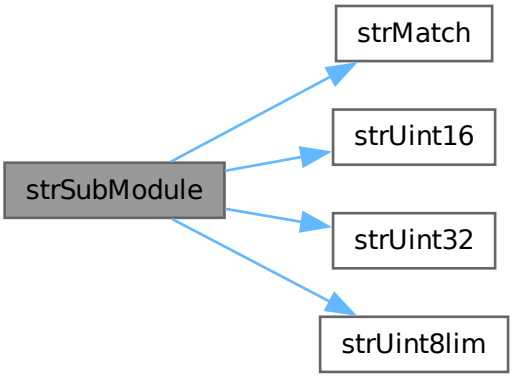
Here is the caller graph for this function:



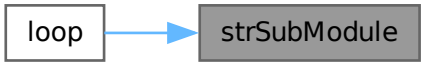
5.7.2.53 strSubModule()

```
bool strSubModule (
            char * s,
            char ** next = 0)
```

Here is the call graph for this function:



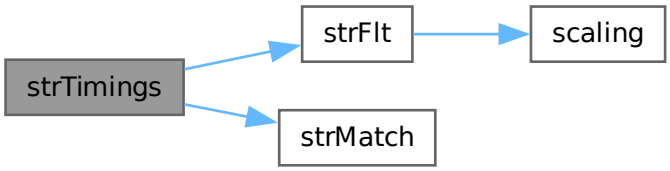
Here is the caller graph for this function:



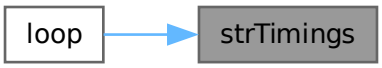
5.7.2.54 strTimings()

```
bool strTimings (
            char * s,
            char ** next = 0)
```

Here is the call graph for this function:



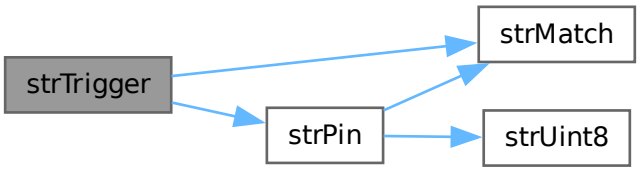
Here is the caller graph for this function:



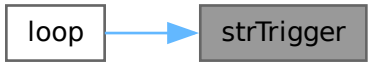
5.7.2.55 strTrigger()

```
bool strTrigger (
            char * pc,
            char ** next)
```

Here is the call graph for this function:



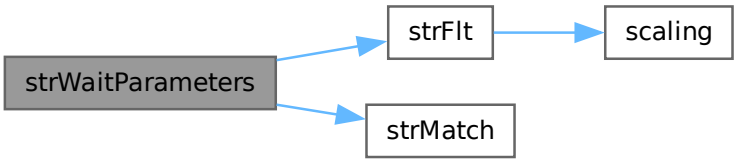
Here is the caller graph for this function:



5.7.2.56 strWaitParameters()

```
void strWaitParameters (
            char * pc,
            float * timeout,
            float * timestep,
            bool * interruptible,
            bool * verbose,
            char ** next = 0)
```

Here is the call graph for this function:

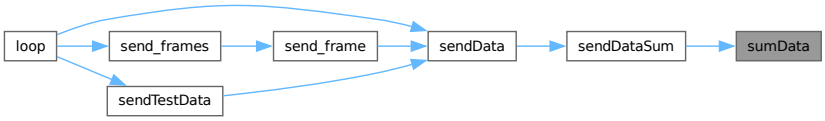


Here is the caller graph for this function:

5.7.2.57 sumData()

```
uint32_t sumData (
            uint16_t * bufferp)  [inline]
```

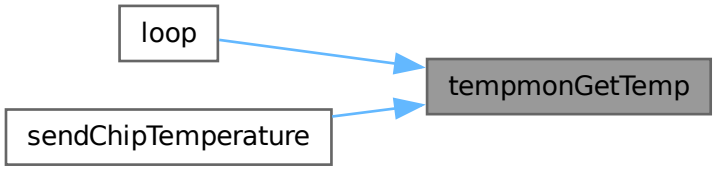
Here is the caller graph for this function:



5.7.2.58 tempmonGetTemp()

```
float tempmonGetTemp (
            void )  [extern]
```

Here is the caller graph for this function:



5.7.3 Variable Documentation

5.7.3.1 adc

```
ADC* adc = new ADC( )
```

5.7.3.2 adc_averages

```
unsigned int adc_averages = 0
```

5.7.3.3 adc_data

```
uint16_t adc_data[NADC_CHANNELS] = { 0 }
```

5.7.3.4 analogPin

```
const int analogPin = A0
```

5.7.3.5 authorstr

```
const char authorstr[] = "Patents Pending and (c) 2020, 2023, 2025, 2026 by Mitchell C. Nelson,
Ph.D. "
```

5.7.3.6 buffer_index

```
unsigned int buffer_index = 0
```

5.7.3.7 buffer_ring

```
uint16_t buffer_ring[NBUFFERS][NREADOUT] = { 0 }
```

5.7.3.8 bufferp

```
uint16_t* bufferp = buffer_ring[0]
```

5.7.3.9 busyPin

```
const int busyPin = BUSY_PIN
```

5.7.3.10 chipTemp_averages

```
unsigned int chipTemp_averages = 0
```

5.7.3.11 CLKPin

```
const int CLKPin = 13
```

5.7.3.12 clock_callback

```
void(* clock_callback) () () = nullptr
```

5.7.3.13 clock_counter

```
unsigned int clock_counter = 0
```

5.7.3.14 clock_counts

```
unsigned int clock_counts = 0
```

5.7.3.15 clock_mode

```
bool clock_mode = false
```

5.7.3.16 clock_timer

```
IntervalTimer clock_timer
```

5.7.3.17 clock_usecs

```
float clock_usecs = 0
```

5.7.3.18 CNVSTPin

```
const int CNVSTPin = CNVST_PIN
```

5.7.3.19 counter

```
unsigned int counter = 0
```

5.7.3.20 crc_enable

```
bool crc_enable = false
```

5.7.3.21 crctable

```
unsigned char crctable[256]
```

Initial value:

```
= {
  0x00,0x31,0x62,0x53,0xc4,0xf5,0xa6,0x97,0xb9,0x88,0xdb,0xea,0x7d,0x4c,0x1f,0x2e,
  0x43,0x72,0x21,0x10,0x87,0xb6,0xe5,0xd4,0xfa,0xcb,0x98,0xa9,0x3e,0x0f,0x5c,0x6d,
  0x86,0xb7,0xe4,0xd5,0x42,0x73,0x20,0x11,0x3f,0x0e,0x5d,0x6c,0xfb,0xca,0x99,0xa8,
  0xc5,0xf4,0xa7,0x96,0x01,0x30,0x63,0x52,0x7c,0x4d,0x1e,0x2f,0xb8,0x89,0xda,0xeb,
  0x3d,0x0c,0x5f,0x6e,0xf9,0xc8,0x9b,0xaa,0x84,0xb5,0xe6,0xd7,0x40,0x71,0x22,0x13,
  0x7e,0x4f,0x1c,0x2d,0xba,0x8b,0xd8,0xe9,0xc7,0xf6,0xa5,0x94,0x03,0x32,0x61,0x50,
  0xbb,0x8a,0xd9,0xe8,0x7f,0x4e,0x1d,0x2c,0x02,0x33,0x60,0x51,0xc6,0xf7,0xa4,0x95,
  0xf8,0xc9,0x9a,0xab,0x3c,0x0d,0x5e,0x6f,0x41,0x70,0x23,0x12,0x85,0xb4,0xe7,0xd6,
  0x7a,0x4b,0x18,0x29,0xbe,0x8f,0xdc,0xed,0xc3,0xf2,0xa1,0x90,0x07,0x36,0x65,0x54,
  0x39,0x08,0x5b,0x6a,0xfd,0xcc,0x9f,0xae,0x80,0xb1,0xe2,0xd3,0x44,0x75,0x26,0x17,
  0xfc,0xcd,0x9e,0xaf,0x38,0x09,0x5a,0x6b,0x45,0x74,0x27,0x16,0x81,0xb0,0xe3,0xd2,
  0xbf,0x8e,0xdd,0xec,0x7b,0x4a,0x19,0x28,0x06,0x37,0x64,0x55,0xc2,0xf3,0xa0,0x91,
  0x47,0x76,0x25,0x14,0x83,0xb2,0xe1,0xd0,0xfe,0xcf,0x9c,0xad,0x3a,0x0b,0x58,0x69,
  0x04,0x35,0x66,0x57,0xc0,0xf1,0xa2,0x93,0xbd,0x8c,0xdf,0xee,0x79,0x48,0x1b,0x2a,
  0xc1,0xf0,0xa3,0x92,0x05,0x34,0x67,0x56,0x78,0x49,0x1a,0x2b,0xbc,0x8d,0xde,0xef,
  0x82,0xb3,0xe0,0xd1,0x46,0x77,0x24,0x15,0x3b,0x0a,0x59,0x68,0xff,0xce,0x9d,0xac
}
```

5.7.3.22 data_async

```
bool data_async = true
```

5.7.3.23 dataformat

```
unsigned int dataformat = BINARY
```

5.7.3.24 diagnostic_usecs

```
elapsedMicros diagnostic_usecs
```

5.7.3.25 diagnostics

```
bool diagnostics = false
```

5.7.3.26 elapsed_usecs

```
elapsedMicros elapsed_usecs
```

5.7.3.27 fMPin

```
const int fMPin = CLK_PIN
```

5.7.3.28 frame_header_ring

```
TCD1304Device::Frame_Header frame_header_ring[NBUFFERS] = {0}
```

5.7.3.29 frame_index

```
unsigned int frame_index = 0
```

5.7.3.30 frame_send_index

```
unsigned int frame_send_index = 0
```

5.7.3.31 framep

```
TCD1304Device::Frame_Header* framep = &frame_header_ring[0]
```

5.7.3.32 ICGPin

```
const int ICGPin = ICG_PIN
```

5.7.3.33 nrcvbuf

```
uint16_t nrcvbuf = 0
```

5.7.3.34 ocpinattached

```
bool ocpinattached = false
```

5.7.3.35 ocpinstate

```
bool ocpinstate = false
```

5.7.3.36 rcvbuffer

```
char rcvbuffer[RCVLEN]
```

5.7.3.37 SDIPin

```
const int SDIPin = 11
```

5.7.3.38 SDOPin

```
const int SDOPin = 12
```

5.7.3.39 sensorstr

```
const char sensorstr[] = "TCD1304"
```

5.7.3.40 SHPin

```
const int SHPin = SH_PIN
```

5.7.3.41 sndbuffer

```
char sndbuffer[SNDLEN]
```

5.7.3.42 srcfilestr

```
const char srcfilestr[] = __FILE__
```

5.7.3.43 sum_enable

```
bool sum_enable = false
```

5.7.3.44 syncPin

```
const int syncPin = SYNC_PIN
```

5.7.3.45 tcd1304device

```
TCD1304Device tcd1304device = TCD1304Device( )
```

5.7.3.46 triggerPin

```
const int triggerPin = TRIGGER_PIN
```

5.7.3.47 usb_string_manufacturer_name

```
usb_string_descriptor_struct_manufacturer usb_string_manufacturer_name
```

Initial value:

```
= {
    2 + thisMANUFACTURER_NAME_LEN * 2,
    3,
    thisMANUFACTURER_NAME
  }
```

5.7.3.48 usb_string_product_name

```
usb_string_descriptor_struct_product usb_string_product_name
```

Initial value:

```
= {
    2 + thisPRODUCT_NAME_LEN * 2,
    3,
    thisPRODUCT_NAME
  }
```

5.7.3.49 usb_string_serial_number

```
usb_string_descriptor_struct_serial_number usb_string_serial_number
```

Initial value:

```
=
    {
      2 + thisPRODUCT_SERIAL_NUMBER_LEN * 2,
      3,
      thisPRODUCT_SERIAL_NUMBER
    }
```

5.7.3.50 versionstr

```
const char versionstr[] = "TCD1304Device vers 0.6 " __DATE__
```

Index

adc
TCD1304Device_Controller_260318.ino, 188
adc_averages
TCD1304Device_Controller_260318.ino, 188
adc_blank_counter
TCD1304Device, 62
adc_blank_isr
TCD1304Device, 20
adc_blank_running
TCD1304Device, 62
adc_data
TCD1304Device_Controller_260318.ino, 188
adc_offset
TCD1304Device, 62
adc_offset_done
TCD1304Device, 62
adc_raw_offset
TCD1304Device, 62
ADC_RESOLUTION
TCD1304Device_Controller_260318.ino, 156
analogPin
TCD1304Device_Controller_260318.ino, 188
ASCII
TCD1304Device_Controller_260318.ino, 156
attach_isr
TCD1304Device, 20
authorstr
TCD1304Device_Controller_260318.ino, 188

basenamef
parselib2.cpp, 76
parselib2.h, 87
bDescriptorType
usb_string_descriptor_struct_manufacturer, 71
usb_string_descriptor_struct_product, 72
usb_string_descriptor_struct_serial_number, 73
begin_pit
TCD1304Device, 21
BINARY
TCD1304Device_Controller_260318.ino, 156
bLength
usb_string_descriptor_struct_manufacturer, 71
usb_string_descriptor_struct_product, 72
usb_string_descriptor_struct_serial_number, 73
blink
TCD1304Device_Controller_260318.ino, 159
buffer
TCD1304Device::Frame_Header_struct, 7
buffer_index
TCD1304Device_Controller_260318.ino, 188
buffer_ring
TCD1304Device_Controller_260318.ino, 188
bufferp
TCD1304Device_Controller_260318.ino, 188
BUSY_PIN
TCD1304Device2c.h, 99
BUSY_PIN_DEFAULT
TCD1304Device2c.h, 99
busyPin
TCD1304Device_Controller_260318.ino, 188
busytoggled
TCD1304Device, 62

calculateCRC
TCD1304Device_Controller_260318.ino, 159
check_submodule
TCD1304Device, 21
chipTemp_averages
TCD1304Device_Controller_260318.ino, 188
clear_busypin
TCD1304Device, 22
clear_error_flags
TCD1304Device, 22
clear_frames_completed_callback
TCD1304Device, 22
clear_framesets_completed_callback
TCD1304Device, 23
clear_ldok
TCD1304Device, 23
clear_mode
TCD1304Device, 23
clear_run
TCD1304Device, 23
clear_sync_busy_pins
TCD1304Device, 23
clear_syncpin
TCD1304Device, 24
CLEARCNVST
TCD1304Device2c.h, 99, 100
clk
TCD1304Device, 63
CLK_CHANNEL

TCD1304Device2c.h, 100
CLK_DEFAULT
TCD1304Device2c.h, 100
CLK_IRQ
TCD1304Device2c.h, 100
CLK_MASK
TCD1304Device2c.h, 100
CLK_MONITOR_PIN
TCD1304Device2c.h, 100
CLK_MUXVAL
TCD1304Device2c.h, 100
CLK_PIN
TCD1304Device2c.h, 100
CLK_SUBMODULE
TCD1304Device2c.h, 100
CLKPin
TCD1304Device_Controller_260318.ino, 189
clock_callback
TCD1304Device_Controller_260318.ino, 189
clock_counter
TCD1304Device_Controller_260318.ino, 189
clock_counts
TCD1304Device_Controller_260318.ino, 189
clock_isr
TCD1304Device_Controller_260318.ino, 159
clock_mode
TCD1304Device_Controller_260318.ino, 189
clock_setup
TCD1304Device_Controller_260318.ino, 159
clock_start
TCD1304Device_Controller_260318.ino, 160
clock_stop
TCD1304Device_Controller_260318.ino, 160
clock_timer
TCD1304Device_Controller_260318.ino, 189
clock_usecs
TCD1304Device_Controller_260318.ino, 189
close
TCD1304Device, 24
CMPF_MASKA_OFF
TCD1304Device2c.h, 100
CMPF_MASKA_ON
TCD1304Device2c.h, 101
CMPF_MASKA_ON_OFF
TCD1304Device2c.h, 101
CMPF_MASKB_OFF
TCD1304Device2c.h, 101
CMPF_MASKB_ON
TCD1304Device2c.h, 101
CMPF_MASKB_ON_OFF
TCD1304Device2c.h, 101
cnvst
TCD1304Device, 63
CNVST_CHANNEL
TCD1304Device2c.h, 101
CNVST_CLKS_FROM_ICG
TCD1304Device2c.h, 101
cnvst_counter
TCD1304Device, 63
CNVST_IRQ
TCD1304Device2c.h, 101
CNVST_MASK
TCD1304Device2c.h, 101
CNVST_PIN
TCD1304Device2c.h, 101
CNVST_SECS
TCD1304Device2c.h, 102
cnvst_start
TCD1304Device, 24
CNVST_START_OVERHEAD
TCD1304Device2c.h, 102
CNVST_SUBMODULE
TCD1304Device2c.h, 102
CNVSTPin
TCD1304Device_Controller_260318.ino, 189
CONTROLLER_CSPIN
TCD1304Device_Controller_260318.ino, 156
CONTROLLER_CSPIN2
TCD1304Device_Controller_260318.ino, 156
CONTROLLER_OCPIN
TCD1304Device_Controller_260318.ino, 156
counter
TCD1304Device_Controller_260318.ino, 189
COUNTER_MAX_SECS
TCD1304Device2c.h, 102
COUNTSCEIL
TCD1304Device2c.h, 102
COUNTSFLOOR
TCD1304Device2c.h, 102
countWords
parselib2.cpp, 76
parselib2.h, 87
crc_enable
TCD1304Device_Controller_260318.ino, 189
crctable
TCD1304Device_Controller_260318.ino, 190
ctrl2_mask
TCD1304Device::SubModule, 11
CYCCNT2SECS
TCD1304Device2c.h, 102
cyccnt64_prepare_exposure
TCD1304Device, 63
cyccnt64_prepare_readout
TCD1304Device, 63
cycles64
TCD1304Device, 24
TCD1304Device_Controller_260318.ino, 161
CYCLES_PER_USEC

TCD1304Device_Controller_260318.ino, 156

DARKSTART
TCD1304Device2c.h, 102
data_async
TCD1304Device_Controller_260318.ino, 190
dataformat
TCD1304Device_Controller_260318.ino, 190
DATASTART
TCD1304Device2c.h, 103
DATASTOP
TCD1304Device2c.h, 103
DEBUG
TCD1304Device2c.h, 103
DEBUGPRINTF
TCD1304Device2c.h, 103
diagnostic_usecs
TCD1304Device_Controller_260318.ino, 190
diagnostics
TCD1304Device_Controller_260318.ino, 190
disable_irqs
TCD1304Device, 25
disablePinInterrupts
TCD1304Device_Controller_260318.ino, 161
divider
TCD1304Device::SubModule, 11
DRMCNELSONLAB
TCD1304Device_Controller_260318.ino, 157

eeErase
TCD1304Device_Controller_260318.ino, 161
EEPROM_COEFF_ADDR
TCD1304Device_Controller_260318.ino, 157
EEPROM_COEFF_LEN
TCD1304Device_Controller_260318.ino, 157
EEPROM_ID_ADDR
TCD1304Device_Controller_260318.ino, 157
EEPROM_ID_LEN
TCD1304Device_Controller_260318.ino, 157
EEPROM_NCOEFFS
TCD1304Device_Controller_260318.ino, 157
EEPROM_NUNITS
TCD1304Device_Controller_260318.ino, 157
EEPROM_SIZE
TCD1304Device_Controller_260318.ino, 157
EEPROM_UNITS_ADDR
TCD1304Device_Controller_260318.ino, 157
EEPROM_UNITS_LEN
TCD1304Device_Controller_260318.ino, 157
eeread
TCD1304Device_Controller_260318.ino, 161
eereadUntil
TCD1304Device_Controller_260318.ino, 162
eewrite
TCD1304Device_Controller_260318.ino, 162
eewriteUntil
TCD1304Device_Controller_260318.ino, 163
elapsed_usecs
TCD1304Device_Controller_260318.ino, 190
eos
parselib2.cpp, 77
eow
parselib2.cpp, 77
eraseCoefficients
TCD1304Device_Controller_260318.ino, 163
eraseIdentifier
TCD1304Device_Controller_260318.ino, 164
eraseUnits
TCD1304Device_Controller_260318.ino, 164
error_flag
TCD1304Device, 63
TCD1304Device::Frame_Header_struct, 7

fastAnalogRead
TCD1304Device_Controller_260318.ino, 165
fill_frame_header
TCD1304Device, 25
filler
TCD1304Device::SubModule, 11
Firwmare for the TCD1304 using FlexPWM, 1
FLEXPWM
TCD1304Device2c.h, 109
flexpwm
TCD1304Device, 63
TCD1304Device::SubModule, 11
flexpwm_resetcounters
TCD1304Device, 26
flexpwm_running
TCD1304Device, 63
flexpwm_start
TCD1304Device, 26
flexpwm_stop
TCD1304Device, 26
flexpwm_wait
TCD1304Device, 27
fMPin
TCD1304Device_Controller_260318.ino, 190
force
TCD1304Device, 27
frame_callback
TCD1304Device_Controller_260318.ino, 165
frame_counter
TCD1304Device, 63
TCD1304Device::Frame_Header_struct, 8
frame_counts
TCD1304Device, 64
frame_elapsed_secs
TCD1304Device::Frame_Header_struct, 8
frame_exposure_secs

TCD1304Device::Frame_Header_struct, 8
Frame_Header
TCD1304Device, 19
frame_header_ring
TCD1304Device_Controller_260318.ino, 191
frame_index
TCD1304Device_Controller_260318.ino, 191
frame_send_index
TCD1304Device_Controller_260318.ino, 191
framep
TCD1304Device_Controller_260318.ino, 191
frames_complete
TCD1304Device, 64
frames_completed
TCD1304Device::Frame_Header_struct, 8
frames_completed_callback
TCD1304Device, 64
frameset_counter
TCD1304Device, 64
TCD1304Device::Frame_Header_struct, 8
frameset_counts
TCD1304Device, 64
framesets_complete
TCD1304Device, 64
framesets_completed
TCD1304Device::Frame_Header_struct, 8
framesets_completed_callback
TCD1304Device, 64

generic_wait
TCD1304Device, 27
get_next_multiple
TCD1304Device, 28
get_next_phase_tick
TCD1304Device, 28
get_next_phase_time
TCD1304Device, 28

help
TCD1304Device_Controller_260318.ino, 166

icg
TCD1304Device, 64
ICG_CHANNEL
TCD1304Device2c.h, 103
icg_counter
TCD1304Device, 64
ICG_IRQ
TCD1304Device2c.h, 103
ICG_MASK
TCD1304Device2c.h, 103
ICG_MUXVAL
TCD1304Device2c.h, 103
ICG_PIN
TCD1304Device2c.h, 103
ICG_SUBMODULE
TCD1304Device2c.h, 104
ICGPin
TCD1304Device_Controller_260318.ino, 191
IDLE
TCD1304Device, 19
IMR_INDEX
TCD1304Device_Controller_260318.ino, 158
intena_mask
TCD1304Device::SubModule, 11
interrupt_priority
TCD1304Device::SubModule, 11
invertA
TCD1304Device::SubModule, 11
invertB
TCD1304Device::SubModule, 12
irq
TCD1304Device::SubModule, 12
isr
TCD1304Device::SubModule, 12
ISR_INDEX
TCD1304Device_Controller_260318.ino, 158

load_frames_completed_callback
TCD1304Device, 29
load_framesets_completed_callback
TCD1304Device, 29
load_submodule
TCD1304Device, 29
loop
TCD1304Device_Controller_260318.ino, 166

mask
TCD1304Device::SubModule, 12
mode
TCD1304Device, 65
TCD1304Device::Frame_Header_struct, 8
muxvalA
TCD1304Device::SubModule, 12
muxvalB
TCD1304Device::SubModule, 12

NADC_CHANNELS
TCD1304Device_Controller_260318.ino, 158
name
TCD1304Device::SubModule, 12
NBITS
TCD1304Device2c.h, 104
nbuffer
TCD1304Device::Frame_Header_struct, 8
NBUFFERS
TCD1304Device_Controller_260318.ino, 158
NDARK
TCD1304Device2c.h, 104
newvals

TCD1304Device::SubModule, 12
nextWord
parselib2.cpp, 77
parselib2.h, 88
NOTCONFIGURED
TCD1304Device2c.h, 109
NPIXELS
TCD1304Device2c.h, 104
nprintbuffer
parselib2.cpp, 86
nrcvbuf
TCD1304Device_Controller_260318.ino, 191
NREADOUT
TCD1304Device2c.h, 104

ocpinAttach
TCD1304Device_Controller_260318.ino, 167
ocpinattached
TCD1304Device_Controller_260318.ino, 191
ocpinDetach
TCD1304Device_Controller_260318.ino, 168
ocpinISR
TCD1304Device_Controller_260318.ino, 168
ocpinRead
TCD1304Device_Controller_260318.ino, 169
ocpinstate
TCD1304Device_Controller_260318.ino, 191
offA_counts
TCD1304Device::SubModule, 12
offB_counts
TCD1304Device::SubModule, 12
offset
TCD1304Device::Frame_Header_struct, 8
TCD1304Device::SubModule, 13
offsetdone
TCD1304Device::Frame_Header_struct, 8
onA_counts
TCD1304Device::SubModule, 13
onB_counts
TCD1304Device::SubModule, 13
oops_flag
TCD1304Device, 65
TCD1304Device::Frame_Header_struct, 9
oopsbuffer
TCD1304Device, 65

parselib2.cpp, 75
basenamef, 76
countWords, 76
eos, 77
eow, 77
nextWord, 77
nprintbuffer, 86
printbuffer, 86
scaling, 78
serialPrintf, 78
serialPrintlnf, 79
strBool, 79
strFlt, 80
strFlts, 81
strMatch, 81
strUint, 82
strUint16, 83
strUint32, 83
strUint32s, 84
strUint8, 84
strUint8lim, 85
strUints, 85
wordLength, 86
parselib2.h, 86
basenamef, 87
countWords, 87
nextWord, 88
PARSELIB_H, 87
serialPrintf, 88
serialPrintlnf, 89
strBool, 89
strFlt, 89
strFlts, 90
strMatch, 91
strUint, 92
strUint16, 92
strUint32, 93
strUint32s, 93
strUint8, 94
strUint8lim, 94
strUints, 95
wordLength, 95
PARSELIB_H
parselib2.h, 87
period_counts
TCD1304Device::SubModule, 13
period_secs
TCD1304Device::SubModule, 13
pinA
TCD1304Device::SubModule, 13
pinB
TCD1304Device::SubModule, 13
PINPULLS
TCD1304Device2c.h, 104
PIT
TCD1304Device2c.h, 109
pit_begun
TCD1304Device, 65
pit_callback
TCD1304Device, 65
PIT_CHANNEL
TCD1304Device2c.h, 104
pit_counter

TCD1304Device, 65
pit_counts
TCD1304Device, 65
pit_cyccnt64_now
TCD1304Device, 65
pit_cyccnt64_prev
TCD1304Device, 65
pit_cyccnt64_start
TCD1304Device, 65
pit_delay_counts
TCD1304Device, 66
pit_delay_secs
TCD1304Device, 66
pit_difference_secs
TCD1304Device, 29
TCD1304Device::Frame_Header_struct, 9
pit_elapsed_secs
TCD1304Device, 30
TCD1304Device::Frame_Header_struct, 9
PIT_FREQ
TCD1304Device2c.h, 104
pit_interval_counts
TCD1304Device, 66
pit_interval_secs
TCD1304Device, 66
pit_isr
TCD1304Device, 30
pit_running
TCD1304Device, 66
pit_start
TCD1304Device, 31
pit_state
TCD1304Device, 66
pit_wait
TCD1304Device, 31
PITCOUNTS
TCD1304Device2c.h, 104
PITSECS
TCD1304Device2c.h, 105
prescale
TCD1304Device::SubModule, 13
print_and_check_submodule
TCD1304Device, 32
print_counters
TCD1304Device, 32
print_errormsg
TCD1304Device, 33
print_pulse_parameters
TCD1304Device, 33
print_setup
TCD1304Device, 33
print_setup_diagnostic
TCD1304Device, 34
print_submodule
TCD1304Device, 34
printbits16_
TCD1304Device, 34
printbuffer
parselib2.cpp, 86
printCoefficients
TCD1304Device_Controller_260318.ino, 169
printIdentifier
TCD1304Device_Controller_260318.ino, 170
printTriggerSetup
TCD1304Device_Controller_260318.ino, 170
printUnits
TCD1304Device_Controller_260318.ino, 171
PULSE
TCD1304Device2c.h, 109
pulse_armed
TCD1304Device, 66
pulse_clk_secs
TCD1304Device, 66
pulse_cnvst_isr
TCD1304Device, 35
pulse_cnvst_offset_secs
TCD1304Device, 66
pulse_cnvst_secs
TCD1304Device, 67
pulse_icg_isr
TCD1304Device, 35
pulse_icg_offset_secs
TCD1304Device, 67
pulse_icg_secs
TCD1304Device, 67
pulse_interframe_min_secs
TCD1304Device, 67
PULSE_LOOP
TCD1304Device2c.h, 109
pulse_loop_arm
TCD1304Device, 36
pulse_loop_armed
TCD1304Device, 67
pulse_loop_cnvst_isr
TCD1304Device, 36
pulse_loop_frame_min_secs
TCD1304Device, 67
pulse_loop_icg_isr
TCD1304Device, 37
pulse_loop_period_secs
TCD1304Device, 67
pulse_loop_running
TCD1304Device, 67
pulse_loop_sh_isr
TCD1304Device, 37
pulse_loop_start
TCD1304Device, 37
PULSE_LOOP_STATE

TCD1304Device, 19
pulse_loop_state
TCD1304Device, 67
pulse_loop_stop
TCD1304Device, 38
pulse_loop_wait
TCD1304Device, 38
pulse_reset_and_arm
TCD1304Device, 39
pulse_sh_isr
TCD1304Device, 39
pulse_sh_offset_secs
TCD1304Device, 67
pulse_sh_secs
TCD1304Device, 68
pulse_start
TCD1304Device, 40
pulsePin
TCD1304Device_Controller_260318.ino, 171
PWM_CTRL2_CLOCK_MASTER
TCD1304Device2c.h, 105
PWM_CTRL2_CLOCK_SLAVE
TCD1304Device2c.h, 105
PWM_CTRL2_CLOCK_SYNC
TCD1304Device2c.h, 105

rawoffset
TCD1304Device::Frame_Header_struct, 9
rcvbuffer
TCD1304Device_Controller_260318.ino, 191
RCVLEN
TCD1304Device_Controller_260318.ino, 158
read
TCD1304Device, 40, 41
read_adc_offset
TCD1304Device, 42
read_buffer
TCD1304Device, 68
read_callback
TCD1304Device, 68
read_counter
TCD1304Device, 68
read_counts
TCD1304Device, 68
read_expected_time
TCD1304Device, 68
read_pointer
TCD1304Device, 68
readCoefficients
TCD1304Device_Controller_260318.ino, 172
readIdentifier
TCD1304Device_Controller_260318.ino, 172
README.md, 96
READOUT
TCD1304Device, 19
readUnits
TCD1304Device_Controller_260318.ino, 173
ready_for_send
TCD1304Device::Frame_Header_struct, 9
register_dump
TCD1304Device, 43
resumePinInterrupts
TCD1304Device_Controller_260318.ino, 174
ROUNDTO
TCD1304Device2c.h, 105
ROUNDTOMOD
TCD1304Device2c.h, 105
ROUNDUP
TCD1304Device2c.h, 105

SAR_OFFSET
TCD1304Device2c.h, 106
scaling
parselib2.cpp, 78
SDIPin
TCD1304Device_Controller_260318.ino, 191
SDOPin
TCD1304Device_Controller_260318.ino, 192
SECONDS
TCD1304Device2c.h, 106
SECS2CYCCNT
TCD1304Device2c.h, 106
send_frame
TCD1304Device_Controller_260318.ino, 174
send_frames
TCD1304Device_Controller_260318.ino, 174
send_header
TCD1304Device_Controller_260318.ino, 175
sendADCs
TCD1304Device_Controller_260318.ino, 175
sendBuffer_Binary
TCD1304Device_Controller_260318.ino, 176
sendBuffer_Formatted
TCD1304Device_Controller_260318.ino, 176
sendChipTemperature
TCD1304Device_Controller_260318.ino, 177
sendData
TCD1304Device_Controller_260318.ino, 177
sendDataCRC
TCD1304Device_Controller_260318.ino, 178
sendDataReady
TCD1304Device_Controller_260318.ino, 178
sendDataSum
TCD1304Device_Controller_260318.ino, 178
SENDLENGTHBYTES
TCD1304Device2c.h, 106
SENDLENGTHBYTES32
TCD1304Device2c.h, 106

SENDLENGTHWORDS
TCD1304Device2c.h, 106
SENDSTART
TCD1304Device2c.h, 106
sendTestData
TCD1304Device_Controller_260318.ino, 179
sensorstr
TCD1304Device_Controller_260318.ino, 192
serialPrintf
parselib2.cpp, 78
parselib2.h, 88
serialPrintlnf
parselib2.cpp, 79
parselib2.h, 89
set_clock_master
TCD1304Device, 43, 44
set_clock_slave
TCD1304Device, 44
set_clock_sync
TCD1304Device, 45
set_cnt
TCD1304Device, 46
set_init
TCD1304Device, 46
set_ldok
TCD1304Device, 46
set_outen
TCD1304Device, 46
set_outen_off
TCD1304Device, 46
set_outen_on
TCD1304Device, 46
set_outenA_on
TCD1304Device, 47
set_prescale
TCD1304Device, 47
set_pulse_parameters
TCD1304Device, 47
set_run
TCD1304Device, 47
set_val0
TCD1304Device, 48
set_val1
TCD1304Device, 48
set_val2
TCD1304Device, 48
set_val3
TCD1304Device, 48
set_val4
TCD1304Device, 48
set_val5
TCD1304Device, 48
SETCNVST
TCD1304Device2c.h, 107
setup
TCD1304Device_Controller_260318.ino, 180
setup_digital_pins
TCD1304Device, 49
setup_pit
TCD1304Device, 49
setup_pulse
TCD1304Device, 50
setup_pulse_loop
TCD1304Device, 51
setup_submodule
TCD1304Device, 52
setup_triggers
TCD1304Device, 53
sh
TCD1304Device, 68
SH_CHANNEL
TCD1304Device2c.h, 107
sh_clearing_counter
TCD1304Device, 68
sh_clearing_counts
TCD1304Device, 69
SH_CLEARING_DEFAULT
TCD1304Device2c.h, 107
sh_counter
TCD1304Device, 69
sh_counts_per_icg
TCD1304Device, 69
sh_cyccnt64_exposure
TCD1304Device, 69
sh_cyccnt64_now
TCD1304Device, 69
sh_cyccnt64_prev
TCD1304Device, 69
sh_cyccnt64_start
TCD1304Device, 69
sh_difference_secs
TCD1304Device, 54
sh_elapsed_secs
TCD1304Device, 54
sh_exposure_secs
TCD1304Device, 54
SH_IRQ
TCD1304Device2c.h, 107
SH_MASK
TCD1304Device2c.h, 107
SH_MUXVAL
TCD1304Device2c.h, 107
SH_PIN
TCD1304Device2c.h, 107
sh_short_period_counts
TCD1304Device, 69
SH_SUBMODULE
TCD1304Device2c.h, 107

SHPin
TCD1304Device_Controller_260318.ino, 192
SHUTTERMIN
TCD1304Device2c.h, 107
sndbuffer
TCD1304Device_Controller_260318.ino, 192
SNDLEN
TCD1304Device_Controller_260318.ino, 158
spi_settings
TCD1304Device_Controller_260318.ino, 180
srcfilestr
TCD1304Device_Controller_260318.ino, 192
START_EXPOSURE
TCD1304Device, 19
start_read
TCD1304Device, 54
start_triggers
TCD1304Device, 55
stop_all
TCD1304Device, 56
stop_pit
TCD1304Device, 56
stop_triggers
TCD1304Device, 57
storeCoefficients
TCD1304Device_Controller_260318.ino, 180
storeIdentifier
TCD1304Device_Controller_260318.ino, 181
storeUnits
TCD1304Device_Controller_260318.ino, 182
strBool
parselib2.cpp, 79
parselib2.h, 89
strFlt
parselib2.cpp, 80
parselib2.h, 89
strFlts
parselib2.cpp, 81
parselib2.h, 90
strMatch
parselib2.cpp, 81
parselib2.h, 91
strPin
TCD1304Device_Controller_260318.ino, 182
strStartNoStart
TCD1304Device_Controller_260318.ino, 183
strSubModule
TCD1304Device_Controller_260318.ino, 184
strTimings
TCD1304Device_Controller_260318.ino, 184
strTrigger
TCD1304Device_Controller_260318.ino, 185
strUint
parselib2.cpp, 82
parselib2.h, 92
strUint16
parselib2.cpp, 83
parselib2.h, 92
strUint32
parselib2.cpp, 83
parselib2.h, 93
strUint32s
parselib2.cpp, 84
parselib2.h, 93
strUint8
parselib2.cpp, 84
parselib2.h, 94
strUint8lim
parselib2.cpp, 85
parselib2.h, 94
strUints
parselib2.cpp, 85
parselib2.h, 95
strWaitParameters
TCD1304Device_Controller_260318.ino, 186
submod
TCD1304Device::SubModule, 13
SubModule
TCD1304Device::SubModule, 11
sum_enable
TCD1304Device_Controller_260318.ino, 192
sumData
TCD1304Device_Controller_260318.ino, 187
sync_enabled
TCD1304Device, 69
SYNC_PIN
TCD1304Device2c.h, 108
sync_pin
TCD1304Device, 69
SYNC_PIN_DEFAULT
TCD1304Device2c.h, 108
syncPin
TCD1304Device_Controller_260318.ino, 192
synctoggled
TCD1304Device, 70

TCD1304_MAXCLKHZ
TCD1304Device2c.h, 108
TCD1304_MINCLKHZ
TCD1304Device2c.h, 108
TCD1304_Mode_t
TCD1304Device2c.h, 109
TCD1304Device, 14
adc_blank_counter, 62
adc_blank_isr, 20
adc_blank_running, 62
adc_offset, 62
adc_offset_done, 62

adc_raw_offset, 62
attach_isr, 20
begin_pit, 21
busytoggled, 62
check_submodule, 21
clear_busypin, 22
clear_error_flags, 22
clear_frames_completed_callback, 22
clear_framesets_completed_callback, 23
clear_ldok, 23
clear_mode, 23
clear_run, 23
clear_sync_busy_pins, 23
clear_syncpin, 24
clk, 63
close, 24
cnvst, 63
cnvst_counter, 63
cnvst_start, 24
cyccnt64_prepare_exposure, 63
cyccnt64_prepare_readout, 63
cycles64, 24
disable_irqs, 25
error_flag, 63
fill_frame_header, 25
flexpwm, 63
flexpwm_resetcounters, 26
flexpwm_running, 63
flexpwm_start, 26
flexpwm_stop, 26
flexpwm_wait, 27
force, 27
frame_counter, 63
frame_counts, 64
Frame_Header, 19
frames_complete, 64
frames_completed_callback, 64
frameset_counter, 64
frameset_counts, 64
framesets_complete, 64
framesets_completed_callback, 64
generic_wait, 27
get_next_multiple, 28
get_next_phase_tick, 28
get_next_phase_time, 28
icg, 64
icg_counter, 64
IDLE, 19
load_frames_completed_callback, 29
load_framesets_completed_callback, 29
load_submodule, 29
mode, 65
oops_flag, 65
oopsbuffer, 65
pit_begun, 65
pit_callback, 65
pit_counter, 65
pit_counts, 65
pit_cyccnt64_now, 65
pit_cyccnt64_prev, 65
pit_cyccnt64_start, 65
pit_delay_counts, 66
pit_delay_secs, 66
pit_difference_secs, 29
pit_elapsed_secs, 30
pit_interval_counts, 66
pit_interval_secs, 66
pit_isr, 30
pit_running, 66
pit_start, 31
pit_state, 66
pit_wait, 31
print_and_check_submodule, 32
print_counters, 32
print_errormsg, 33
print_pulse_parameters, 33
print_setup, 33
print_setup_diagnostic, 34
print_submodule, 34
printbits16_, 34
pulse_armed, 66
pulse_clk_secs, 66
pulse_cnvst_isr, 35
pulse_cnvst_offset_secs, 66
pulse_cnvst_secs, 67
pulse_icg_isr, 35
pulse_icg_offset_secs, 67
pulse_icg_secs, 67
pulse_interframe_min_secs, 67
pulse_loop_arm, 36
pulse_loop_armed, 67
pulse_loop_cnvst_isr, 36
pulse_loop_frame_min_secs, 67
pulse_loop_icg_isr, 37
pulse_loop_period_secs, 67
pulse_loop_running, 67
pulse_loop_sh_isr, 37
pulse_loop_start, 37
PULSE_LOOP_STATE, 19
pulse_loop_state, 67
pulse_loop_stop, 38
pulse_loop_wait, 38
pulse_reset_and_arm, 39
pulse_sh_isr, 39
pulse_sh_offset_secs, 67
pulse_sh_secs, 68
pulse_start, 40
read, 40, 41

read_adc_offset, 42
read_buffer, 68
read_callback, 68
read_counter, 68
read_counts, 68
read_expected_time, 68
read_pointer, 68
READOUT, 19
register_dump, 43
set_clock_master, 43, 44
set_clock_slave, 44
set_clock_sync, 45
set_cnt, 46
set_init, 46
set_ldok, 46
set_outen, 46
set_outen_off, 46
set_outen_on, 46
set_outenA_on, 47
set_prescale, 47
set_pulse_parameters, 47
set_run, 47
set_val0, 48
set_val1, 48
set_val2, 48
set_val3, 48
set_val4, 48
set_val5, 48
setup_digital_pins, 49
setup_pit, 49
setup_pulse, 50
setup_pulse_loop, 51
setup_submodule, 52
setup_triggers, 53
sh, 68
sh_clearing_counter, 68
sh_clearing_counts, 69
sh_counter, 69
sh_counts_per_icg, 69
sh_cyccnt64_exposure, 69
sh_cyccnt64_now, 69
sh_cyccnt64_prev, 69
sh_cyccnt64_start, 69
sh_difference_secs, 54
sh_elapsed_secs, 54
sh_exposure_secs, 54
sh_short_period_counts, 69
START_EXPOSURE, 19
start_read, 54
start_triggers, 55
stop_all, 56
stop_pit, 56
stop_triggers, 57
sync_enabled, 69
sync_pin, 69
synctoggled, 70
TCD1304Device, 20
toggle_busypin, 57
toggle_syncpin, 57
trigger_attached, 70
trigger_busy, 70
trigger_callback, 70
trigger_counter, 70
trigger_counts, 70
trigger_cyccnt64_now, 70
trigger_cyccnt64_prev, 70
trigger_cyccnt64_start, 70
trigger_difference_secs, 57
trigger_edge_mode, 70
trigger_elapsed_secs, 57
trigger_isr, 58
trigger_mode, 71
trigger_pin, 71
trigger_pin_mode, 71
triggered_read, 58, 59
update_read_buffer, 60
WAIT_EXPOSURE, 19
WAIT_FRAME, 19
wait_read, 60
wait_trigger, 61
wait_triggered_read, 61
tcd1304device
TCD1304Device_Controller_260318.ino, 192
TCD1304Device2c.h, 96
BUSY_PIN, 99
BUSY_PIN_DEFAULT, 99
CLEARCNVST, 99, 100
CLK_CHANNEL, 100
CLK_DEFAULT, 100
CLK_IRQ, 100
CLK_MASK, 100
CLK_MONITOR_PIN, 100
CLK_MUXVAL, 100
CLK_PIN, 100
CLK_SUBMODULE, 100
CMPF_MASKA_OFF, 100
CMPF_MASKA_ON, 101
CMPF_MASKA_ON_OFF, 101
CMPF_MASKB_OFF, 101
CMPF_MASKB_ON, 101
CMPF_MASKB_ON_OFF, 101
CNVST_CHANNEL, 101
CNVST_CLKS_FROM_ICG, 101
CNVST_IRQ, 101
CNVST_MASK, 101
CNVST_PIN, 101
CNVST_SECS, 102
CNVST_START_OVERHEAD, 102

CNVST_SUBMODULE, 102
COUNTER_MAX_SECS, 102
COUNTSCEIL, 102
COUNTSFLOOR, 102
CYCCNT2SECS, 102
DARKSTART, 102
DATASTART, 103
DATASTOP, 103
DEBUG, 103
DEBUGPRINTF, 103
FLEXPWM, 109
ICG_CHANNEL, 103
ICG_IRQ, 103
ICG_MASK, 103
ICG_MUXVAL, 103
ICG_PIN, 103
ICG_SUBMODULE, 104
NBITS, 104
NDARK, 104
NOTCONFIGURED, 109
NPIXELS, 104
NREADOUT, 104
PINPULLS, 104
PIT, 109
PIT_CHANNEL, 104
PIT_FREQ, 104
PITCOUNTS, 104
PITSECS, 105
PULSE, 109
PULSE_LOOP, 109
PWM_CTRL2_CLOCK_MASTER, 105
PWM_CTRL2_CLOCK_SLAVE, 105
PWM_CTRL2_CLOCK_SYNC, 105
ROUNDTO, 105
ROUNDTOMOD, 105
ROUNDUP, 105
SAR_OFFSET, 106
SECONDS, 106
SECS2CYCCNT, 106
SENDLENGTHBYTES, 106
SENDLENGTHBYTES32, 106
SENDLENGTHWORDS, 106
SENDSTART, 106
SETCNVST, 107
SH_CHANNEL, 107
SH_CLEARING_DEFAULT, 107
SH_IRQ, 107
SH_MASK, 107
SH_MUXVAL, 107
SH_PIN, 107
SH_SUBMODULE, 107
SHUTTERMIN, 107
SYNC_PIN, 108
SYNC_PIN_DEFAULT, 108
TCD1304_MAXCLKHZ, 108
TCD1304_MINCLKHZ, 108
TCD1304_Mode_t, 109
TCD1304DEVICE_LIBARY_VERSION, 108
TDIFFSECS, 108
TRIGGER_PIN, 108
USBSPEED, 108
USBTRANSFERSECS, 108
VFS, 109
VPERBIT, 109
TCD1304Device::Frame_Header_struct, 7
buffer, 7
error_flag, 7
frame_counter, 8
frame_elapsed_secs, 8
frame_exposure_secs, 8
frames_completed, 8
frameset_counter, 8
framesets_completed, 8
mode, 8
nbuffer, 8
offset, 8
offsetdone, 8
oops_flag, 9
pit_difference_secs, 9
pit_elapsed_secs, 9
rawoffset, 9
ready_for_send, 9
trigger_counter, 9
trigger_difference_secs, 9
trigger_elapsed_secs, 9
trigger_mode, 9
TCD1304Device::SubModule, 10
ctrl2_mask, 11
divider, 11
filler, 11
flexpwm, 11
intena_mask, 11
interrupt_priority, 11
invertA, 11
invertB, 12
irq, 12
isr, 12
mask, 12
muxvalA, 12
muxvalB, 12
name, 12
newvals, 12
offA_counts, 12
offB_counts, 12
offset, 13
onA_counts, 13
onB_counts, 13
period_counts, 13

period_secs, 13
pinA, 13
pinB, 13
prescale, 13
submod, 13
SubModule, 11
TCD1304Device_Controller_260318.ino, 152
adc, 188
adc_averages, 188
adc_data, 188
ADC_RESOLUTION, 156
analogPin, 188
ASCII, 156
authorstr, 188
BINARY, 156
blink, 159
buffer_index, 188
buffer_ring, 188
bufferp, 188
busyPin, 188
calculateCRC, 159
chipTemp_averages, 188
CLKPin, 189
clock_callback, 189
clock_counter, 189
clock_counts, 189
clock_isr, 159
clock_mode, 189
clock_setup, 159
clock_start, 160
clock_stop, 160
clock_timer, 189
clock_usecs, 189
CNVSTPin, 189
CONTROLLER_CSPIN, 156
CONTROLLER_CSPIN2, 156
CONTROLLER_OCPIN, 156
counter, 189
crc_enable, 189
crctable, 190
cycles64, 161
CYCLES_PER_USEC, 156
data_async, 190
dataformat, 190
diagnostic_usecs, 190
diagnostics, 190
disablePinInterrupts, 161
DRMCNELSONLAB, 157
eeErase, 161
EEPROM_COEFF_ADDR, 157
EEPROM_COEFF_LEN, 157
EEPROM_ID_ADDR, 157
EEPROM_ID_LEN, 157
EEPROM_NCOEFFS, 157
EEPROM_NUNITS, 157
EEPROM_SIZE, 157
EEPROM_UNITS_ADDR, 157
EEPROM_UNITS_LEN, 157
eeread, 161
eereadUntil, 162
eewrite, 162
eewriteUntil, 163
elapsed_usecs, 190
eraseCoefficients, 163
eraseIdentifier, 164
eraseUnits, 164
fastAnalogRead, 165
fMPin, 190
frame_callback, 165
frame_header_ring, 191
frame_index, 191
frame_send_index, 191
framep, 191
help, 166
ICGPin, 191
IMR_INDEX, 158
ISR_INDEX, 158
loop, 166
NADC_CHANNELS, 158
NBUFFERS, 158
nrcvbuf, 191
ocpinAttach, 167
ocpinattached, 191
ocpinDetach, 168
ocpinISR, 168
ocpinRead, 169
ocpinstate, 191
printCoefficients, 169
printIdentifier, 170
printTriggerSetup, 170
printUnits, 171
pulsePin, 171
rcvbuffer, 191
RCVLEN, 158
readCoefficients, 172
readIdentifier, 172
readUnits, 173
resumePinInterrupts, 174
SDIPin, 191
SDOPin, 192
send_frame, 174
send_frames, 174
send_header, 175
sendADCs, 175
sendBuffer_Binary, 176
sendBuffer_Formatted, 176
sendChipTemperature, 177
sendData, 177

sendDataCRC, 178
sendDataReady, 178
sendDataSum, 178
sendTestData, 179
sensorstr, 192
setup, 180
SHPin, 192
sndbuffer, 192
SNDLEN, 158
spi_settings, 180
srcfilestr, 192
storeCoefficients, 180
storeIdentifier, 181
storeUnits, 182
strPin, 182
strStartNoStart, 183
strSubModule, 184
strTimings, 184
strTrigger, 185
strWaitParameters, 186
sum_enable, 192
sumData, 187
syncPin, 192
tcd1304device, 192
tempmonGetTemp, 187
thisMANUFACTURER_NAME, 158
thisMANUFACTURER_NAME_LEN, 158
thisPRODUCT_NAME, 158
thisPRODUCT_NAME_LEN, 158
thisPRODUCT_SERIAL_NUMBER, 159
thisPRODUCT_SERIAL_NUMBER_LEN, 159
triggerPin, 192
usb_string_manufacturer_name, 192
usb_string_product_name, 193
usb_string_serial_number, 193
versionstr, 193
TCD1304DEVICE_LIBARY_VERSION
TCD1304Device2c.h, 108
TDIFFSECS
TCD1304Device2c.h, 108
tempmonGetTemp
TCD1304Device_Controller_260318.ino, 187
thisMANUFACTURER_NAME
TCD1304Device_Controller_260318.ino, 158
thisMANUFACTURER_NAME_LEN
TCD1304Device_Controller_260318.ino, 158
thisPRODUCT_NAME
TCD1304Device_Controller_260318.ino, 158
thisPRODUCT_NAME_LEN
TCD1304Device_Controller_260318.ino, 158
thisPRODUCT_SERIAL_NUMBER
TCD1304Device_Controller_260318.ino, 159
thisPRODUCT_SERIAL_NUMBER_LEN
TCD1304Device_Controller_260318.ino, 159
toggle_busypin
TCD1304Device, 57
toggle_syncpin
TCD1304Device, 57
trigger_attached
TCD1304Device, 70
trigger_busy
TCD1304Device, 70
trigger_callback
TCD1304Device, 70
trigger_counter
TCD1304Device, 70
TCD1304Device::Frame_Header_struct, 9
trigger_counts
TCD1304Device, 70
trigger_cyccnt64_now
TCD1304Device, 70
trigger_cyccnt64_prev
TCD1304Device, 70
trigger_cyccnt64_start
TCD1304Device, 70
trigger_difference_secs
TCD1304Device, 57
TCD1304Device::Frame_Header_struct, 9
trigger_edge_mode
TCD1304Device, 70
trigger_elapsed_secs
TCD1304Device, 57
TCD1304Device::Frame_Header_struct, 9
trigger_isr
TCD1304Device, 58
trigger_mode
TCD1304Device, 71
TCD1304Device::Frame_Header_struct, 9
TRIGGER_PIN
TCD1304Device2c.h, 108
trigger_pin
TCD1304Device, 71
trigger_pin_mode
TCD1304Device, 71
triggered_read
TCD1304Device, 58, 59
triggerPin
TCD1304Device_Controller_260318.ino, 192

update_read_buffer
TCD1304Device, 60
usb_string_descriptor_struct_manufacturer, 71
bDescriptorType, 71
bLength, 71
wString, 71
usb_string_descriptor_struct_product, 72
bDescriptorType, 72
bLength, 72

wString, 72
usb_string_descriptor_struct_serial_number, 72
bDescriptorType, 73
bLength, 73
wString, 73
usb_string_manufacturer_name
TCD1304Device_Controller_260318.ino, 192
usb_string_product_name
TCD1304Device_Controller_260318.ino, 193
usb_string_serial_number
TCD1304Device_Controller_260318.ino, 193
USBSPEED
TCD1304Device2c.h, 108
USBTRANSFERSECS
TCD1304Device2c.h, 108

versionstr
TCD1304Device_Controller_260318.ino, 193
VFS
TCD1304Device2c.h, 109
VPERBIT
TCD1304Device2c.h, 109

WAIT_EXPOSURE
TCD1304Device, 19
WAIT_FRAME
TCD1304Device, 19
wait_read
TCD1304Device, 60
wait_trigger
TCD1304Device, 61
wait_triggered_read
TCD1304Device, 61
wordLength
parselib2.cpp, 86
parselib2.h, 95
wString
usb_string_descriptor_struct_manufacturer, 71
usb_string_descriptor_struct_product, 72
usb_string_descriptor_struct_serial_number, 73